# Reinforcement Learning for Congestion Control

A Systematic Survey of Designs, Evaluation Practice, and Barriers to Deployment

Gaurab Baral Nityashree Vangala

CS 6043: Networking
Fall 2026

**Abstract**

Reinforcement learning (RL) promises congestion controllers that learn the control law instead of hard-coding it, and a decade of work has produced many designs for the Internet, wireless, satellite, IoT and data-center networks. We report a systematic survey of this literature following PRISMA 2020. From 997 records retrieved from OpenAlex, arXiv, USENIX proceedings and forward and backward citation searching, we include 301 primary studies (331 reports) published from 2016 to October 2026. We code each study's setting, control point, state, action, reward, algorithm, training environment, highest evaluation level, generalization testing, fairness evaluation and code availability, and we release every decision with its evidence. Designs have shifted from clean-slate neural policies to hybrids in which learning tunes, selects or bounds a classic algorithm: 35% of designs since 2023, against 24% before. The evidence base is weak and is not improving. Among studies whose evaluation environment is known, 57% are evaluated only in simulation and 24% reach a real network, and the real-network share has fallen since 2021. Only 11% of design studies test outside their training distribution, and 8% release code. A reproducible case study shows a PPO controller that achieves 96% utilization in distribution but drops to 29% under 10 ms of RTT jitter. Broader training helps only within the range it covers, and it made training less reliable. The deployments that exist either keep a classic algorithm or safeguard in the loop or learn from the operator's own logs, are run by operators who own the endpoints, and optimize a narrow service metric. We map nine deployment barriers to the evidence and mitigations in the corpus, review the emerging use of large language models to design and run controllers, and propose an evaluation checklist and a research agenda.

# Contents


**1 Introduction and Background** **3**
- 1.1 Motivation . . . 3
- 1.2 Research Questions and Contributions . . . 3
- 1.3 Congestion Control in Brief . . . 4
- 1.4 Reinforcement Learning in Brief . . . 5
- 1.5 Congestion Control as an RL Problem . . . 6
- 1.6 Review Method . . . 7
  - 1.6.1 Eligibility criteria . . . 7
  - 1.6.2 Information sources and search . . . 8
  - 1.6.3 Selection process and data collection . . . 8
  - 1.6.4 Data items . . . 9
  - 1.6.5 Study appraisal . . . 9
  - 1.6.6 Synthesis . . . 9
- 1.7 Related Surveys and Positioning . . . 9
- 1.8 Overview of the Corpus . . . 11

**2 RL Congestion Control for the Wide-Area Internet** **15**
- 2.1 Clean-Slate Learned Controllers . . . 15
- 2.2 Hybrid Designs: Learning on Top of Classic Control . . . 16
- 2.3 Data-Driven and Offline Designs . . . 17
- 2.4 Multiple Objectives and Application Preferences . . . 18
- 2.5 Fairness and Multi-Agent Learning . . . 18
- 2.6 Multipath Transport . . . 18
- 2.7 Real-Time Media . . . 18
- 2.8 QUIC, NDN, SDN and Other Transports . . . 19
- 2.9 Network-Side Control: AQM and ECN . . . 19
- 2.10 Summary . . . 20

**3 Wireless, Cellular, Satellite and IoT Networks** **21**
- 3.1 Cellular and Mobile Links . . . 21
- 3.2 High-Speed Rail and Vehicular Transport . . . 22
- 3.3 Wi-Fi, UAV and Other Wireless Links . . . 22
- 3.4 Multipath over Heterogeneous Radios . . . 22
- 3.5 Satellite and Space Networks . . . 23
- 3.6 IoT, Sensor and Ad Hoc Networks . . . 23
- 3.7 Summary . . . 24

**4 Data-Center Networks** **25**
- 4.1 Learning to Tune ECN at the Switch . . . 25
- 4.2 End-Host and NIC Rate Control . . . 25
- 4.3 Training Infrastructure . . . 26
- 4.4 Summary . . . 26

**5 Training and Testing: From Simulators to Real Networks** **27**
5.1 The Evaluation Ladder . . . 27
5.2 Where the Corpus Stands . . . 28
5.3 Evaluation Practices . . . 29
5.4 Evidence on Generalization . . . 30
5.5 Case Study: How the Training Distribution Shapes Behavior Outside It . . . 30
5.6 Recommendations for Evaluating Learned Controllers . . . 33

**6 Deployment: Barriers, LLM-Designed Controllers, and Open Problems** **34**
6.1 What the Deployed Systems Have in Common . . . 34
6.2 Barriers to Deployment . . . 34
6.3 LLM-Designed and LLM-Driven Controllers . . . 37
6.4 Research Agenda . . . 39
6.5 Threats to Validity . . . 39
6.6 Conclusion . . . 40

**A Search Strings and Sources** **41**

**B Additional Tables and Figures** **43**

**C Replication Package** **56**

**D PRISMA 2020 Checklist** **57**

**Bibliography** **58**

# 1 Introduction and Background

## 1.1 Motivation

Congestion control (CC) decides how fast a sender may inject data into a shared network. It is one of the oldest control problems in networking: after the Internet's congestion collapses of 1986, Jacobson's slow start and additive-increase/multiplicative-decrease (AIMD) rules restored stability by reacting to packet loss [1, 2]. Almost four decades later the problem takes many forms. Today's flows cross gigabit fiber, shared Wi-Fi, cellular links whose capacity changes every few milliseconds, satellite constellations with frequent handovers, and data-center fabrics where a full round trip takes tens of microseconds. Each environment breaks some assumption of the classic designs. Loss is no longer a reliable congestion signal on wireless links, delay is noisy under ACK aggregation, and buffers range from a few kilobytes in data-center switches to seconds of queueing in cellular base stations. The result is a long line of hand-designed algorithms, each tuned to one environment: Vegas, CUBIC, BBR and Copa for the Internet [3–6], Sprout and Verus for cellular links [7, 8], and DCTCP, DCQCN, TIMELY, HPCC and Swift for data centers [9–13].

Reinforcement learning (RL) offers a different route: instead of writing the control law, let a sender *learn* it. In RL-based congestion control, an agent at the sender (or in the network) watches signals such as delay, throughput, and packet loss. Every short interval it raises or lowers the sending rate or the congestion window, and it receives a reward that is high for fast delivery and low for long delays or lost packets. Over many trials it learns which action works best in each network state. Remy [14] and PCC [15, 16] showed that machine-generated or online-learned controllers can beat hand-tuned ones. Aurora [17] then cast Internet congestion control explicitly as a deep RL problem in 2018–2019, and the field has grown quickly since. Our searches retrieved 997 records, and 301 primary studies met our eligibility criteria (Section 1.6). However, learned controllers that look excellent in a simulator sometimes fail on links they never saw. Few of them run inside an operating-system kernel, and fewer still have been deployed by an operator.

This survey takes stock of that literature. It focuses only on RL (including imitation learning, offline RL, bandits and multi-agent RL), covers controllers published from 2016 through October 2026, and records for every study *where its results come from*: simulation, emulation, or a real network. That last dimension is our main new contribution. Without it, a reader cannot tell whether a reported gain reflects the network or the simulator.

## 1.2 Research Questions and Contributions

We organize the survey around three research questions.

**RQ1** **Design.** How do RL methods control congestion: what do they observe, what do they change, and what reward do they learn from?

**RQ2** **Testing.** How are these methods trained and tested, and do they still work on networks they did not see in training?

**RQ3** **Deployment.** What stops operators from using RL-based congestion control in real networks, and what research could fix these problems?

Our contributions are:

1. A **systematic, reproducible corpus** of 301 RL/IL congestion-control studies (331 reports), identified with PRISMA 2020 [18] from open scholarly indexes, USENIX proceedings, and forward and backward

citation searching. Every screening decision and extracted field is released with a reason or an evidence snippet.

2. A **design taxonomy** (**RQ1**) that codes each study's setting, control point, state signals, action, reward terms and RL algorithm family (Chapters 2–4).
3. An **evaluation-practice analysis** (**RQ2**, Chapter 5). We code the highest evaluation level reached by every study, the tools used, whether generalization to unseen conditions was tested, and whether code was released. We complement this with a small, fully reproducible case study that shows how the choice of training distribution shapes a learned controller's behavior outside that distribution.
4. A **deployment-barrier analysis and research agenda** (**RQ3**, Chapter 6). It maps nine barriers to the studies that address them and reviews the newest direction: congestion controllers designed, tuned or executed by large language models (LLMs).

**Roadmap.** The rest of this chapter gives the background needed for the remaining chapters (Sections 1.3 to 1.5), describes the review method (Section 1.6), positions this survey against earlier ones (Section 1.7), and gives an overview of the corpus (Section 1.8). Chapters 2–4 survey designs by setting: the wide-area Internet; wireless, cellular and satellite networks; and data centers. Chapter 5 covers training and testing, and Chapter 6 covers deployment barriers, LLM-designed controllers, and open problems.

## 1.3 Congestion Control in Brief

**Goals.** A congestion controller should (i) use the available bottleneck capacity (*efficiency*), (ii) avoid standing queues and loss (*low delay*), (iii) share capacity among competing flows (*fairness*, often measured with Jain's index [19]), (iv) converge quickly and stay stable when conditions change, and (v) coexist with the deployed base, which in practice means not starving CUBIC and BBR flows (*friendliness*). Network utility maximization frames these goals as each flow maximizing a concave utility of its rate [20]. More recent work argues that deployability should instead be judged by the *harm* a new algorithm inflicts on existing traffic [21].

**Signals and knobs.** End-to-end controllers infer congestion from what they can measure: packet loss, round-trip time (RTT) and its trend, delivery rate, and Explicit Congestion Notification (ECN) marks set by routers [22]. They act through two knobs. The *congestion window* (cwnd) caps the data in flight, so the sending rate is self-clocked by returning ACKs. The *pacing rate* spaces packets in time directly.

**Classic end-host algorithms.** Table 1.1 summarizes the families that RL work compares against. Loss-based AIMD (Reno [23]) grows cwnd by one packet per RTT and halves it on loss. Its throughput falls as $1/\sqrt{p}$ with random loss rate $p$ [24]. CUBIC [4, 25], the Linux default, grows cwnd as a cubic function of the time since the last loss. Delay-based designs such as Vegas [3] and Copa [6] back off when the RTT rises above its minimum. Model-based BBR [5] estimates the bottleneck bandwidth and minimum RTT and paces at the estimated bandwidth, but its interaction with loss-based flows can be unfair [26]. Two lines of work anticipate RL directly. Remy [14] searched offline for a rule table that maximizes an objective under assumed network conditions. A follow-up study showed that such learned controllers degrade when the assumptions are wrong [27], the generalization problem this survey revisits. PCC Allegro and Vivace [15, 16] run online micro-experiments at slightly different rates and move toward the rate with higher utility, a function of throughput and loss (and, in Vivace, latency).

**Table 1.1:** Classic congestion-control families used as baselines by the RL literature.

| Family | Examples | Control law (simplified) |
|---|---|---|
| Loss-based AIMD | Reno/NewReno [23], CUBIC [4] | Grow cwnd until loss; multiplicative decrease on loss. |
| Delay-based | Vegas [3], Copa [6], Swift [13] | Keep queueing delay near a target inferred from RTT minus its minimum. |
| Model-based | BBR [5] | Estimate bottleneck bandwidth and propagation RTT; pace at the estimate and cap in-flight data. |
| Online learning | PCC Allegro/Vivace [15, 16] | Compare a utility of throughput and loss (Vivace: also latency) across rate micro-experiments and move toward the better rate. |
| Offline optimization | Remy [14] | Search a state-to-action table offline for an objective under an assumed network model. |
| ECN-based (DC) | DCTCP [9], DCQCN [10] | Switches mark above a queue threshold; senders cut in proportion to the marked fraction. |
| AQM | RED [28], CoDel [29], PIE [30] | Router drops or marks early, based on queue length or sojourn time. |

**Network-assisted control.** Routers and switches influence congestion through active queue management (AQM), which drops or marks packets before the buffer fills: RED [28], CoDel [29], PIE [30], and the L4S architecture with shallow ECN marking [31]. In data centers, DCTCP [9] reacts to the *fraction* of ECN-marked packets, DCQCN [10] controls RDMA traffic with ECN-driven rate cuts, TIMELY and Swift [11, 13] use precise delay, and HPCC [12] uses in-band network telemetry (INT). Their performance hinges on parameters, most prominently the ECN marking threshold, that operators tune by hand, which is why several RL studies learn these parameters instead of the end-host control law (Chapter 4).

## 1.4 Reinforcement Learning in Brief

RL formalizes sequential decision making as a Markov decision process (MDP) $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$: at each step the agent observes a state $s_t$, takes an action $a_t \sim \pi(\cdot \mid s_t)$, receives a reward $r_t$, and the environment moves to $s_{t+1}$. The goal is a policy that maximizes the expected discounted return $\mathbb{E}\big[\sum_t \gamma^t r_t\big]$ [32]. A congestion controller never sees the whole network, only its own feedback, so the problem is formally a *partially observable* MDP. Most designs compensate by feeding the agent a short history of observations. We group the methods used in the corpus as follows; Fig. 1.4 gives the overall picture.

- **Value-based methods** learn the action-value function $Q(s, a)$. Tabular Q-learning and SARSA suit small discrete state spaces. The Deep Q-Network (DQN) [33] and its variants (double, dueling, Rainbow) approximate $Q$ with a neural network and pick from a discrete set of cwnd or rate adjustments.
- **Policy-gradient methods** optimize $\pi$ directly. Proximal Policy Optimization (PPO) [34], TRPO [35] and A3C/A2C [36] are the common choices, and they handle continuous rate multipliers naturally.
- **Off-policy actor-critic methods** such as DDPG [37], TD3 [38] and SAC [39] reuse past experience from a replay buffer. Distributed variants such as IMPALA [40] decouple acting from learning.
- **Offline RL** learns from a fixed dataset of logged interactions without exploring online, e.g., CQL [41], CRR [42], and sequence-modelling approaches such as the Decision Transformer [43].
- **Imitation learning (IL)** trains a policy to copy an expert, either by behavior cloning or by interactive methods such as DAgger [44]. In congestion control the expert can be an oracle with full knowledge of a simulated link or an existing heuristic.

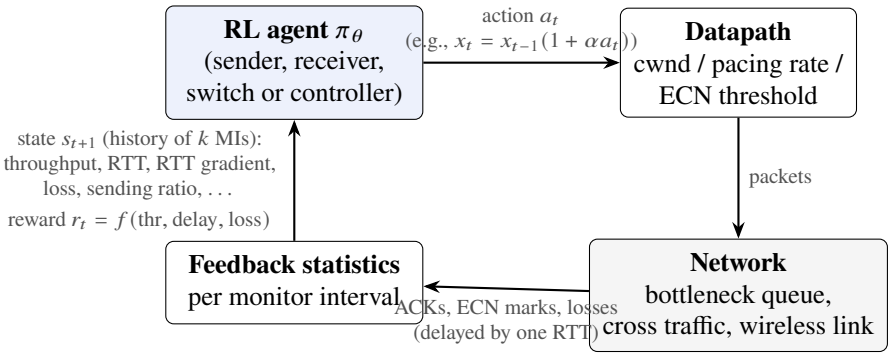



**Figure 1.1:** The canonical RL congestion-control loop. The agent acts once per monitor interval and observes the consequences one RTT later. Designs differ in where the agent runs, what it observes, what it changes, and how the reward trades throughput against delay and loss.

- **Bandits and online learning** treat each decision interval as a single-step problem (no state transitions). They are the closest relatives of PCC-style utility ascent.
- **Multi-agent RL (MARL)** trains several interacting agents, for example competing flows or switches, using centralized-training/decentralized-execution methods such as MADDPG [45] and MAPPO [46].

## 1.5 Congestion Control as an RL Problem

Figure 1.1 shows the canonical formulation, which Aurora made popular [17]. Time is divided into *monitor intervals* (MIs). At the start of each MI the agent picks an action, which is applied for the whole interval. When ACKs for that interval return, the sender computes statistics and the reward. Designs vary along five dimensions.

1. **Placement.** Most agents run at the sender. Some run at the receiver, which is common for real-time video (Section 2.7). Network-side agents sit in switches and routers, where they tune ECN or AQM parameters, and a few run in a centralized SDN controller.
2. **Observation.** Typical observations are a short history of per-interval statistics: throughput or delivery rate, RTT and its minimum, RTT gradient, loss rate, ECN fraction, the current cwnd or rate, and sometimes cross-layer radio information or in-network telemetry. Aurora deliberately chose *scale-free* features (latency gradient, latency ratio and sending ratio) to help generalization across link speeds [17].
3. **Action.** The action can directly set the cwnd or rate, usually as a bounded multiplicative change. It can tune the parameters of a classic algorithm (an initial window, a BBR pacing gain, a CUBIC parameter). It can select among existing algorithms. Or it can set network parameters such as ECN thresholds, AQM drop probabilities and buffer sizes. Discrete actions suit value-based methods; continuous actions suit policy-gradient and actor-critic methods.
4. **Objective.** Most rewards are weighted combinations of throughput, delay and loss. Aurora, for example, uses $10 \cdot \text{throughput} - 1000 \cdot \text{latency} - 2000 \cdot \text{loss}$ with throughput in packets per second [17]. Others use Kleinrock's *power* (throughput over delay), PCC-style utility functions, fairness terms, or application-level quality of experience (QoE). Because the reward fixes the operating point, it is effectively the controller's specification.
5. **Decision frequency.** Per-ACK decisions are too expensive for neural inference at line rate. Most designs act once per MI, which ranges from milliseconds to one or more RTTs. Several hybrid designs

**Table 1.2:** Eligibility criteria.

| | |
|---|---|
| **Inclusion** (all must hold) | |
| IC1 | Packet-switched data network (Internet/WAN, wireless/cellular/satellite, data center, IoT/ad hoc, ICN). |
| IC2 | An RL/IL method (incl. bandits, offline RL, MARL) is the controller, or the object of evaluation, and its decision affects how fast sources send or how congestion is signalled. |
| IC3 | Primary study: new controller, evaluation/analysis of RL-based CC, or a training/testing framework built for it. |
| IC4 | Published 2016–2026 in English as a peer-reviewed paper or a preprint with sufficient technical detail. |
| **Exclusion** (any) | |
| EC1 | "Congestion" outside packet networks (road or air traffic, power grids, on-chip interconnects, crowds). |
| EC2 | RL used for another networking task (routing, traffic engineering, load balancing, scheduling, caching, resource allocation, ABR video, MAC/channel access, V2X beacon-load control). |
| EC3 | No RL/IL component (supervised prediction, heuristics, fuzzy logic, metaheuristics, Bayesian optimization). |
| EC4 | Secondary study (survey, review, tutorial, position paper, editorial overview). |
| EC5 | Not a primary research report (thesis or book chapter duplicating published work, proceedings front matter, machine-generated or retracted text, non-English full text). |

let a classic algorithm react per ACK while the agent adjusts it at a coarser time scale (Section 2.2).

Two properties make congestion control harder than the games on which deep RL was popularized. First, the consequences of an action arrive *one RTT later* and are mixed with the effects of other flows' actions, so credit assignment is noisy and the environment is non-stationary. Second, the controller is deployed on networks whose statistics differ from any training environment. Performance under distribution shift therefore says more about a controller than in-distribution reward does (Chapter 5).

## 1.6 Review Method

We followed the PRISMA 2020 reporting guideline [18], with the search and snowballing steps adapted to computer science as recommended by Kitchenham and Charters [47] and Wohlin [48]. The whole pipeline (search scripts, raw API responses, every screening decision with its reason, the retrieval log, evidence snippets, and the extraction table) is in the replication package that accompanies this survey (`review/`). Every number in this section is generated from those files.

### 1.6.1 Eligibility criteria

Table 1.2 lists the criteria. In short, we include primary studies from 2016–2026 in which a reinforcement-learning or imitation-learning method (including bandits, offline RL and multi-agent RL) controls congestion in a packet network. The method may set a sending rate, a congestion window or pacing rate, the parameters of a congestion-control algorithm, which algorithm runs, or a network-side congestion signal (ECN marking, AQM, buffer sizing). Studies that *evaluate* such controllers (robustness, fairness, verification, interpretability) and frameworks built specifically for training or testing them are included as well, because they bear directly on **RQ2** and **RQ3**. Surveys are not primary studies; they are listed separately and discussed in Section 1.7.

We made two boundary decisions. First, "congestion control" in vehicular networks usually means *channel-load* control of periodic safety beacons (decentralized congestion control, DCC). That is a MAC/facilities-layer problem with different objectives, so we exclude it (EC2) and mention it only in Chapter 3. Second, adaptive-bitrate video streaming over TCP is excluded (EC2), but rate control for *real-time* media, which replaces WebRTC's Google Congestion Control, is congestion control and is included.

### 1.6.2 Information sources and search

The protocol query was

```
("reinforcement learning" OR "imitation learning") AND
("congestion control" OR "congestion window" OR "ECN")
```

applied to titles and abstracts of works published 2016–2026. All searches were run on 31 October 2026. Search strings in each source's syntax are given in Appendix A.

- **OpenAlex** (515 records). OpenAlex indexes the records of IEEE Xplore, the ACM Digital Library, Elsevier, Springer, Wiley, MDPI, USENIX and arXiv, and supports Boolean title/abstract search with an open API. We use it as the reproducible stand-in for the subscription databases named in our protocol and attribute each record to its database of origin by DOI prefix: 232 IEEE Xplore records (DOI prefix 10.1109) and 26 ACM records (10.1145) among the deduplicated set.
- **arXiv** (48 records). The arXiv API's phrase operator under-retrieves (it returned 44 records for the two core phrases against 239 for the same words unquoted). We therefore retrieved on single terms (474 records) and applied the exact Boolean phrase query locally, treating hyphens as spaces as the subscription databases do.
- **USENIX NSDI and ATC proceedings** (4 records). We scraped the title, authors and abstract of all 1558 papers in NSDI 2016–2026 and USENIX ATC 2016–2025 (there are no ATC 2026 proceedings) and applied the query locally.
- **Citation searching.** Forward snowballing collected every 2016–2026 work that cites one of ten seed reports of nine landmark studies (Aurora, in its preprint and ICML versions, Orca, Pantheon/Indigo, MVFST-RL, Sage, QTCP, Eagle, TCP-RL and DRL-CC for MPTCP) and mentions "congestion" in its title or abstract (446 works, 275 not already found). Backward snowballing screened the reference lists of sixteen recent included reports with long related-work sections (151 new records in range). Finally, 4 records known to the reviewers but missed by every search were added and labelled as such.

**Deviation from the protocol.** Our proposal named IEEE Xplore, the ACM Digital Library and Scopus as sources. Their programmatic search interfaces require institutional API keys, and their web interfaces block scripted queries, so we did not query them directly. OpenAlex covers the same publisher records under an open license and makes the search reproducible by anyone, but it may miss records whose abstracts are absent from its index. Forward and backward citation searching was added to offset this risk. It recovered several well-known studies that the Boolean query cannot match because their abstracts never use the exact phrases: MVFST-RL's abstract, for instance, spells "ReinforcementLearning" without a space. The query strings for the three subscription databases are provided in Appendix A so that the search can be repeated with institutional access.

### 1.6.3 Selection process and data collection

Records were deduplicated by DOI, arXiv identifier and normalized title (fuzzy similarity above 0.93), and then screened on title and abstract against Table 1.2. Records whose eligibility could not be decided from the abstract went on to full-text assessment. Every decision is stored with a reason code and a one-line justification. For full-text assessment and data collection we retrieved open-access versions (arXiv, author copies, open-access publisher PDFs) for 87 of the 374 reports sought. For the remaining reports, eligibility and data items were assessed from the abstract, the publisher metadata and, where available, a machine-generated summary; data items those sources do not report are coded *not reported* (NR) rather than guessed. Screening and extraction used extensive tool support, namely keyword pre-filtering, automatic mining of candidate

evidence (algorithm names, simulator and testbed names, code links, and the sentences that define state, action and reward) and an AI assistant. Every coded value is stored with the source it was coded from (full text, abstract, or reviewers' knowledge), so that it can be audited. Section 6.5 discusses the resulting limitations. Multiple reports of the same study (for example a conference paper and its journal extension) were linked and counted once as a *study*.

### 1.6.4 Data items

For each study we coded the items listed in our protocol: the **setting** (Table 1.4); the **control point** (sender, receiver, network device, central controller); the **RL method** (algorithm and family); the **state** signals, the **action** type and the **reward** terms; the **training setup**; the **evaluation environment** at three levels, namely *simulation* (a discrete-event or fluid model, e.g., ns-3, OMNeT++, custom Python simulators), *emulation* (real protocol stacks over emulated links, e.g., Mahimahi, Mininet, NetEm), and *real network* (real links or production traffic, including hardware testbeds); whether **generalization** to conditions outside the training distribution was explicitly tested; whether **fairness** or friendliness was evaluated; **code availability**; and the headline **result**. Each study's *highest* evaluation level is the most realistic of the three it reports.

### 1.6.5 Study appraisal

PRISMA asks reviews to assess the risk of bias of included studies. Tools for that purpose, such as those for clinical trials, do not fit engineering studies whose evidence is an experiment the authors designed. We therefore appraise each design study on five *evidence practices* that determine how far its results can be trusted outside the authors' setup. They are: (i) the evaluation environment is reported; (ii) the evaluation includes emulation with real stacks or a real network; (iii) generalization is tested on conditions outside the training range; (iv) fairness or friendliness is evaluated; and (v) code or artifacts are released. The sum is a study's *practice score* from 0 to 5. It appraises the reported evidence, not the merit of the idea, and it is a lower bound for studies coded from abstracts. Section 5.3 reports the results.

### 1.6.6 Synthesis

We synthesize narratively by setting (Chapters 2–4) and quantitatively across the corpus by tabulating the coded items (Section 1.8 and Chapter 5). We do not pool effect sizes. Studies use incomparable baselines, links and metrics, so a meta-analysis of reported gains would be misleading. That incomparability is itself one of our findings.

**Registration, funding and competing interests.** The review was not registered. Its protocol is our course project proposal, and Section 1.6.2 reports the deviations from it. The review received no funding, and the authors declare no competing interests. Appendix D maps each PRISMA 2020 item to the place where it is reported.

## 1.7 Related Surveys and Positioning

Our searches identified 74 secondary studies that touch on learning-based congestion control. Table 1.3 compares the most relevant of them with this survey, using the scope each one states for itself. Broad surveys of machine learning (ML) or deep RL for networking [49–52] treat congestion control as one application among many. Magazine-style overviews [53–55] discuss a handful of prominent designs and the challenges of standardizing learned controllers. Jiang et al. [56] give the most detailed pre-2021 summary and add an experimental comparison of learning-based schemes. Huang et al. [57] take a broad view of congestion

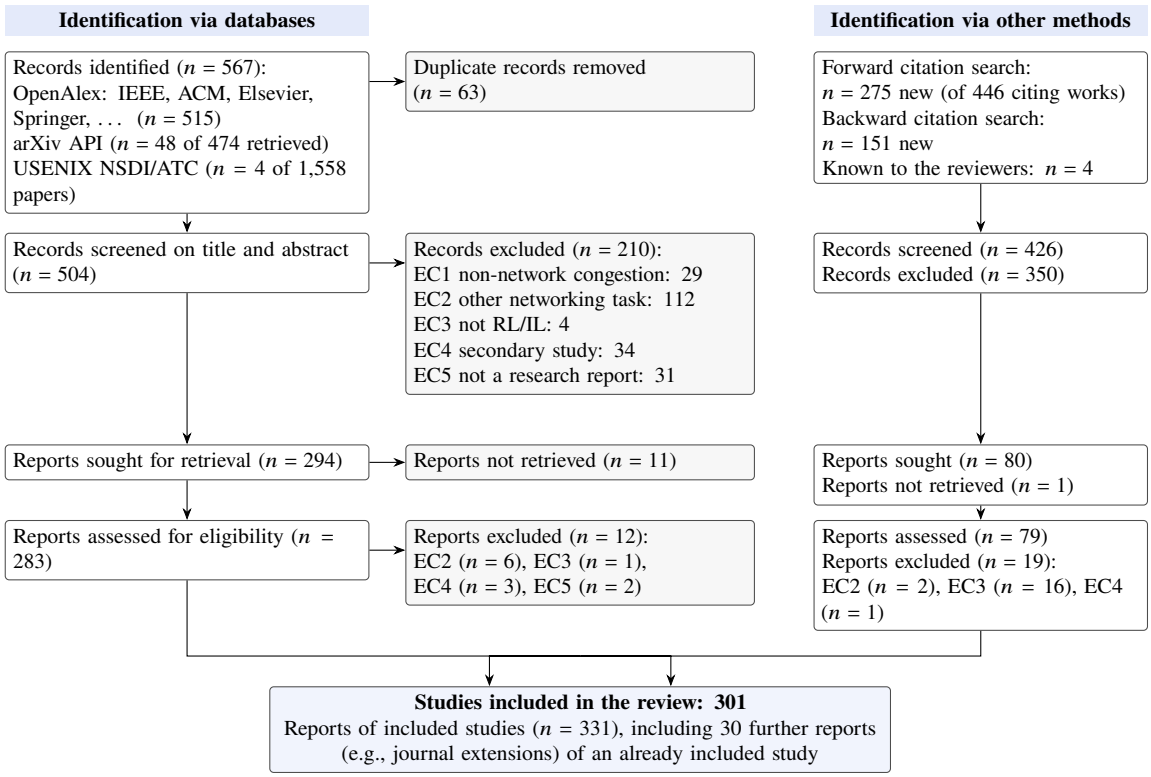



**Figure 1.2:** PRISMA 2020 flow diagram. Left: identification via databases and registers. Right: identification via other methods (citation searching and records known to the reviewers). "Reports assessed" counts full texts where openly available and abstracts plus metadata otherwise (Section 1.6).

**Table 1.3:** Comparison with prior surveys that cover learning-based congestion control, as described by each survey's stated scope. ✓: covered; ○: partially or as one topic among many; –: not covered.

| Survey | Year | Focus | RL-specific | Wireless/DC | Eval. level | LLMs |
|---|---|---|---|---|---|---|
| Luong et al. [51] | 2019 | Deep RL in communications and networking | ✓ | ○ | – | – |
| Zhang et al. [53] | 2019 | ML for Internet CC; standardization challenges | ○ | – | – | – |
| Zhang & Mao [54] | 2020 | Selected review of ML for end-to-end CC | ○ | ○ | – | – |
| Jiang et al. [56] | 2021 | Learning-based CC survey with experimental comparison | ○ | ○ | ○ | – |
| Wei et al. [55] | 2021 | Prominent ML-based CC redesigns compared | ○ | – | – | – |
| Huang et al. [57] | 2021 | ML for broad-sense congestion control and avoidance | ○ | ○ | – | – |
| Tang et al. [52] | 2021 | ML for end-to-end communication toward 6G | ○ | ○ | – | – |
| Lakshmi & Arrama [58] | 2022 | RL-based CC in wired networks | ✓ | – | – | – |
| Maliha et al. [59] | 2023 | MPTCP CC and scheduling, ML vs. classic | ○ | ○ | – | – |
| Toopchinezhad & Ahmadi [60] | 2025 | ML for active queue management | ○ | ○ | – | – |
| Vielhaus et al. [64] | 2026 | How CC algorithms are evaluated (not ML-specific) | – | ✓ | ✓ | – |
| Li et al. [62] | 2025 | Deep RL for CC | ✓ | – | – | – |
| Abedini et al. [63] | 2027 | RL-based CC: algorithms, state/action/reward, platforms | ✓ | ○ | ○ | – |
| **This survey** | 2026 | Systematic review of RL/IL CC across all settings | ✓ | ✓ | ✓ | ✓ |

control and avoidance that also covers routing and bandwidth reservation. Later reviews narrow the scope, to wired networks [58], multipath TCP [59], active queue management [60], or fairness [61]. The two most recent reviews are RL-specific [62, 63]. Abedini et al. compare RL algorithms, state/action/reward definitions, simulation platforms and reported outcomes, which is closest to our **RQ1**. Finally, Vielhaus et al. [64] survey *how* congestion-control algorithms in general are evaluated, which complements our **RQ2**.

This survey differs from them as follows. (i) It is *systematic*: the search, deduplication, screening and extraction are scripted and released with every decision, so the corpus can be audited and updated. (ii) It codes the *evaluation level* (simulation, emulation or real network) of every study, along with generalization testing and code availability, so that claims can be weighed by their evidence. (iii) It covers all settings (Internet, wireless, cellular, satellite, IoT and data centers) through 2026. (iv) It analyzes deployment barriers, including the emerging use of LLMs to design controllers, and it adds a reproducible case study on training-distribution shift.

## 1.8 Overview of the Corpus

The 301 included studies (331 reports) span 2016 to October 2026. Table B.2 in Appendix B lists every study with its setting, action, method, highest evaluation level and code availability. Figure 1.3 shows how quickly the field has grown. Only 44 studies first appeared before 2021, whereas 61% appeared in 2023 or later,

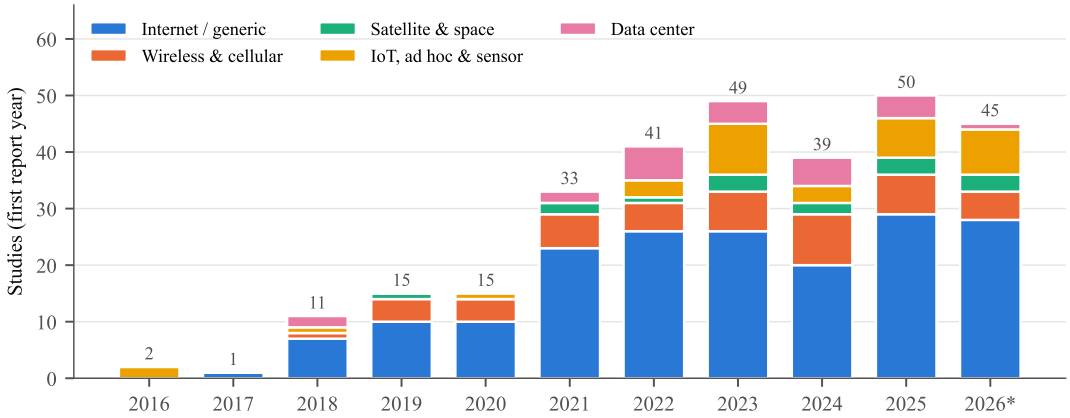



**Figure 1.3:** Included studies by year of first report and setting ($n$ = 301). A study with several reports is counted once, in the year of its earliest report. *2026 covers January to October.

**Table 1.4:** Settings coded in this survey, with the number of included studies (all contribution types) and the chapter that synthesizes them.

| Setting | What it covers | Studies | Chapter |
|---|---|---|---|
| Internet / WAN | Paths across the public Internet or a wide-area network; real-time media over the Internet; Pantheon-style evaluations | 77 | Chapter 2 |
| Generic | A wired bottleneck not tied to a specific network, usually a simulated dumbbell; AQM at a generic router | 103 | Chapter 2 |
| Wireless & cellular | Wi-Fi, LTE/5G/mmWave, vehicular and high-speed-rail links, radio fronthaul, multipath over heterogeneous radios | 48 | Chapter 3 |
| Satellite & space | LEO/GEO/HEO satellite links, satellite–ground and space–air–ground integrated networks, deep-space and delay-tolerant links | 15 | Chapter 3 |
| IoT, ad hoc & sensor | Constrained IoT protocols (CoAP, MQTT, 6LoWPAN), wireless sensor networks, MANETs, mesh, smart-grid and industrial IoT | 34 | Chapter 3 |
| Data center | Data-center and HPC fabrics, RDMA, ECN tuning at switches, NIC-resident control | 24 | Chapter 4 |

with a peak of 50 studies in 2025. The 2026 bar covers ten months only. The Internet/generic group has contributed 20 or more studies every year since 2021 and accounts for most of the growth. IoT and ad hoc studies became noticeably more frequent from 2023, whereas data-center studies remain a small stream of at most six per year.

**Settings.** Table 1.4 defines the six settings we coded and the chapter that covers each. The largest group consists of studies whose setting is the wide-area Internet (77) or a generic wired bottleneck that the authors do not tie to a particular network (103). Such "generic" studies typically evaluate on a dumbbell topology in ns-3 or Mininet. We discuss both kinds in Chapter 2, because they share designs, baselines and benchmarks.

**Contribution types.** Most studies (241) propose a new controller. A further 17 contribute a *method* that applies to existing controllers, such as curriculum learning, trace sampling, distillation, reward correction or kernel deployment, and 8 report a *deployment* in production or an A/B test. The remaining studies do not propose controllers: 26 *analyze* existing ones (evaluation, verification, interpretability, attacks) and 9 build *frameworks* for training or testing them. We call the 266 studies in the first three groups *design studies*. They are the denominator for the design statistics below.

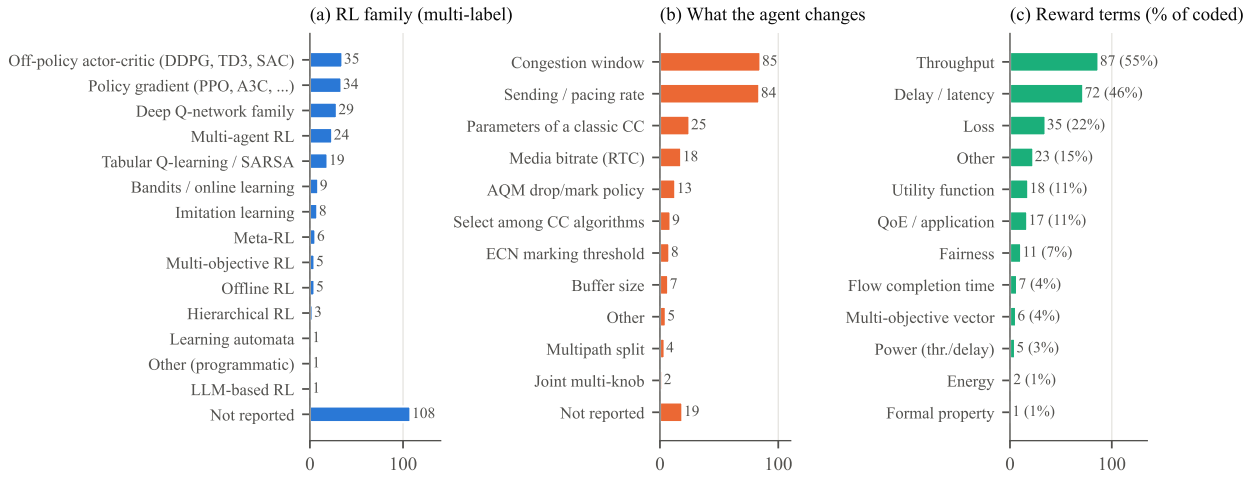



**Figure 1.4:** Design items across the 266 design studies. (a) RL family, multi-label: a study combining, e.g., imitation learning and PPO is counted in both. "Not reported" means the study does not name its algorithm. (b) What the agent changes. (c) Reward terms, as a share of the 158 studies whose reward is reported.

**What the agent changes, and with which algorithm.** Figure 1.4 summarizes the coded design items. The two dominant actions are setting the congestion window (85 studies) and setting the sending or pacing rate (84). A substantial minority tune the *parameters* of a classic algorithm (25), select among existing algorithms (9), or act inside the network on ECN thresholds, AQM policies or buffer sizes (28 studies across all settings). Algorithm families are coded only when a study names its algorithm, which 158 of the 266 design studies do. Among those, 79% use deep RL, and tabular Q-learning or SARSA account for 13%. Off-policy actor-critic methods (DDPG, TD3, SAC), policy-gradient methods (mostly PPO) and the DQN family are used about equally often. Multi-agent RL (24 studies) is now common, especially for fairness and in-network control. Imitation, offline and meta-RL remain niche, although several of the most influential designs belong to these families. The agent runs at the sender in 198 design studies, inside the network in 38, at a central (usually SDN) controller in 16, and at the receiver in 12. Receiver-side agents are mostly bandwidth estimators for real-time video; the rest are NDN consumers and individual RDMA and Wi-Fi designs.

**What the agent observes and optimizes.** The state is reported in enough detail to code its signals in 108 design studies. Throughput or delivery rate (50), delay or RTT (37), loss (24) and queue occupancy (23) dominate. Cross-layer radio information (10), in-network telemetry (9) and ECN marks (3) are rarer. Rewards are reported in 158 studies. They most often reward throughput (87) and penalize delay (72) and, less often, loss (35). Others use utility functions, QoE or flow completion time. Only 11 rewards include an explicit fairness term; Chapter 6 discusses the consequences.

**Hybrid designs.** We call a design *hybrid* when the learned policy works through, alongside, or as a fallback to a classic algorithm rather than replacing it. Examples are an agent that scales CUBIC's window, tunes BBR's gains or a switch's ECN threshold, selects among existing algorithms, or hands over to a heuristic when it is uncertain. Learning *from* classic algorithms only during training, for example by imitating them or by offline RL on their logs, does not make a design hybrid, because the deployed policy then runs alone. Overall, 30% of design studies are hybrid, rising from 24% of those first reported before 2023 to 35% since. We return to this trend in Section 2.2 and Chapter 6.

**Evidence base.** For 78 studies the coding rests on the full text, and for 3 well-known studies whose full text we could not retrieve it rests on the reviewers' knowledge of the paper. The remaining 220 studies were coded from abstracts and metadata. Many items are therefore coded "not reported", because the abstracts omit them. Chapter 5 therefore reports evaluation practice both for the whole corpus and for the full-text subset, where the coding is complete.

# 2 RL Congestion Control for the Wide-Area Internet

The Internet is where RL-based congestion control started and where most of it is still done. This chapter covers the 180 studies whose setting is the wide-area Internet (77) or a generic wired bottleneck (103). Of these, 151 are design studies and 29 analyze existing controllers or provide frameworks for them. We first follow the main design lineage, from clean-slate learned controllers (Section 2.1) through hybrids that learn on top of classic algorithms (Section 2.2) to data-driven and offline designs (Section 2.3). We then turn to multi-objective control (Section 2.4), fairness and multi-agent learning (Section 2.5), multipath transport (Section 2.6), real-time media (Section 2.7), other transports (Section 2.8), and network-side control (Section 2.9). Table 2.1 summarizes the designs that later work most often builds on or compares against.

## 2.1 Clean-Slate Learned Controllers

**Q-learning beginnings.** The earliest Internet studies in the corpus use Q-learning with simple function approximation. QTCP [77, 78] approximates the Q-function with (generalized) Kanerva coding and reports 59.5% higher throughput than rule-based TCP. RL-TCP [79] learns cwnd adjustments for shallow buffers in ns-2. These designs showed that a learned cwnd policy could beat NewReno in simulation. However, their state spaces were small and hand-quantized, and none reports an evaluation beyond simulation. Later work showed that QTCP flows are selfish when they compete with each other. Wrapping the learned decision in an AIMD rule restores fairness without losing throughput [80].

**Aurora and the deep-RL formulation.** Aurora [17] established the formulation most later work adopts (Section 1.5): PPO, monitor intervals, a history of scale-free features, and a reward that trades throughput against latency and loss. Trained on simulated links of 1.2–6 Mbit/s with 50–500 ms latency and 0–5% random loss, it kept near-full utilization in emulation from 1 to 128 Mbit/s, more than 20× its training range, and over wide ranges of buffer size and loss, comparing well with CUBIC and PCC Vivace. It performed poorly only at 1 ms latency, where about 1 ms of packet-processing noise that was absent from training dominated the latency signal. Its authors explicitly flagged fairness, safety and generalization as open problems. Seven years later, those three problems are still the main research agenda (Chapters 5 and 6). Aurora's open code made it the default testbed for methods: curriculum learning [71], teacher–student training [81], scenario-based reward shaping [82], interpretation [83], symbolic distillation [84], formal verification [85–87], meta-RL [88] and fairness shaping [89] have all been demonstrated on Aurora.

**Imitation learning and training at scale.** In parallel, Indigo [65], released with the Pantheon testbed, trained an LSTM by imitation learning. Its oracle knew the ideal congestion window of each emulated link, and Pantheon's emulators were calibrated against real paths so that this training would transfer. MVFST-RL [66] addressed a systems problem that synchronous simulators hide: in a real stack, the network does not pause while the agent computes. It runs the policy asynchronously inside Facebook's QUIC implementation and trains with IMPALA's off-policy correction. Other studies learn online or directly on real networks. ARC [90] trains asynchronously in real networks, Rax [91] learns online within minutes, and MARLIN [92] trains SAC on a real network and then specializes it for tactical satellite and UHF links.

**The broad middle of the literature.** Many Internet design studies follow the Aurora template with a different algorithm, feature set or network architecture. Examples are DQN-family controllers such as TCP-Drinc [93], RLCC [94], TCP-DQN [95], TCP-QNCC [96], KerDqn [97] and others [98, 99]; PPO

**Table 2.1:** Landmark Internet designs: where the agent acts, what it observes, what it learns from, and the most realistic environment in which it was evaluated. Details are from the full texts except where marked † (abstract and reviewers' knowledge). NR: not reported in the material we could access. Level: highest evaluation environment.

| Study | Action | Observation | Learning signal | Algorithm | Level |
|---|---|---|---|---|---|
| Indigo [65] | cwnd step every 10 ms | EWMAs of queueing delay, sending and receiving rate; current cwnd; previous action | imitation of an oracle that knows the emulated link's ideal cwnd | IL, LSTM | Real |
| Aurora [17] | rate multiplier per MI | history of latency gradient, latency ratio, sending ratio | 10 thr − 1000 lat − 2000 loss | PPO | Emu |
| MVFST-RL [66] | cwnd ∈ $\{\cdot, /2, -10, +10, \times 2\}$ | ACK statistics over 100 ms plus action history | throughput–delay trade-off | IMPALA (async.) | Emu |
| Orca [67] | cwnd = $2^{2a} \cdot$ CUBIC's cwnd, once per monitoring period | throughput, delay, loss statistics | throughput minus loss penalty, scaled by a delay factor | TD3 | Real |
| Eagle† [68] | rate | network statistics with LSTM memory | reward plus guidance from BBR as an expert | DRL + IL | Emu |
| Owl [69] | cwnd (Linux kernel module) | end-to-end statistics plus in-network signals | a utility function | DQN | Emu |
| MOCC [70] | rate | network statistics plus objective weights | preference-weighted throughput, delay, loss | PPO + preference sub-net | Real |
| Genet [71] | (training method) | Aurora's | Aurora's | PPO + curriculum | Real |
| Sage† [72] | cwnd | a general representation of many path statistics | logged behavior of existing heuristics | offline RL | Real |
| Spine† [73] | sub-policy for a light CC executor | NR | NR | hierarchical DRL | Real |
| Astraea [74] | rate | local, global (multi-flow) and historical statistics | throughput, delay, loss, fairness | multi-agent actor-critic | Real |
| Canopy [75] | as Orca | as Orca | Orca's reward plus verifier feedback on properties | TD3 + certificates | Real |
| Mutant [76] | select a kernel CC scheme | raw transport statistics | power (throughput over delay) | LinUCB bandit | Real |

controllers [100–103]; and SAC or other actor-critic controllers [104–106]. Typical claims are double-digit throughput or delay gains over NewReno or CUBIC on a few scenarios. Such gains are difficult to compare, because the scenarios, baselines and metrics differ from study to study (Chapter 5). A smaller group studies the formulation itself, and its findings are more transferable than any single controller. The length of the monitor interval and the RTT estimator change QoS noticeably [107]. Discrete actions learn more easily than continuous ones, and fourteen candidate rewards produce visibly different operating points [108]. Rainbow's DQN extensions do not behave in congestion control as they do in Atari [109]. Normalizing the reward by an estimated throughput ceiling makes it scale-invariant, so that one policy can serve links of different speeds [110, 111]. Explicit probing mechanisms or intrinsic probing rewards help an agent discover spare bandwidth [112, 113].

## 2.2 Hybrid Designs: Learning on Top of Classic Control

Hybrid designs are the clearest trend in the corpus. They grow from 24% of design studies before 2023 to 35% since, and 45 of the 151 Internet design studies are hybrid. They rest on a common premise: a neural policy acting at packet timescales is too slow, too expensive and too unpredictable, while classic algorithms react quickly and safely but cannot adapt their operating point. The designs differ in how they divide the work.

**The agent scales a classic window.** Orca [67] is the reference design. CUBIC runs per ACK in the kernel, and once per monitoring period a TD3 agent multiplies CUBIC's window by $2^{2a}$, $a \in [-1, 1]$. Orca was evaluated on emulated links and on global testbeds and found to be more consistent than clean-slate learned controllers at a fraction of their overhead. Canopy [75] keeps Orca's architecture but adds *quantitative certificates* from a verifier to the reward. The verifier measures how close the policy is to satisfying operator-specified safety and robustness properties, such as robustness to measurement noise. This raises worst-case property satisfaction by up to 1.4× while preserving or improving average performance. Other designs build the agent into BBR: ABBR infers BBR's pacing rate with DRL while staying compatible with vanilla BBR [114], and Marten guides a DRL agent with BBR and shields it at run time [115]. Eagle uses BBR differently, as a teacher during training only [68]. PBQ splits the knobs: PPO sets the window while BBR sets the pacing rate [116]. HAN falls back to a classic controller when the policy's uncertainty is high [117].

**The agent tunes parameters.** A lighter variant leaves the control law intact and tunes its parameters: CUBIC's $\beta$ and $C$ [118], CUBIC's window thresholds [119], LEDBAT++'s delay target [120], or several explicit-feedback parameters [121]. Spine [73] generalizes the idea hierarchically. A coarse-grained DRL agent emits *sub-policies* for a lightweight executor that reacts at fine granularity, which cuts control overhead by at least 80% relative to DRL controllers and makes it comparable to CUBIC.

**Classic and learned candidates compete.** In Libra [122, 123], a classic algorithm and an RL policy each propose a rate. Libra briefly tries both candidates in an evaluation stage and commits to whichever achieves the higher utility. It reports 1.2× Orca's throughput on dynamic networks with good fairness and convergence. Anole [124] extends this with candidate selection and deprecation.

**The agent selects the algorithm.** The coarsest hybrid learns *which* existing algorithm to run. TCP-RL [125], deployed at a large search engine, combines group-based RL for the initial window with deep RL that selects among 14 algorithms. Floo [126] selects algorithms for a production mobile web service and switches between them smoothly. Mutant [76] uses a LinUCB contextual bandit to switch among a pool of kernel protocols in real time, with negligible harm to competing flows. Similar selectors use offline-trained DRL [127], contextual Thompson sampling [128] or meta-evolutionary RL [129].

**Finding 2.1.** Hybrid designs trade some of the theoretical flexibility of a clean-slate policy for the reaction speed, low overhead and predictable failure behavior of classic algorithms. The Internet controllers in the corpus that report production use are either hybrids with a classic algorithm or safeguard in the loop (TCP-RL, Loki, Floo, Marten, and OnRL in Chapter 3) or policies imitated from logs or an oracle (Concerto, DuGu). None is a clean-slate policy trained only by exploring a simulator (Chapter 6).

## 2.3 Data-Driven and Offline Designs

Online RL needs an environment to explore, and for congestion control that environment is almost always a simulator or emulator, which is the root of the generalization problem. A newer line instead learns from *logged* data. Sage [72] trains offline on trajectories produced by a pool of existing heuristic schemes. Its authors present it as the first purely data-driven Internet design, and in Internet and controlled tests it learned a policy that performs better than the heuristics it observed. POCC [130] adds pessimism (critic ensembles) to guard against extrapolation from multi-algorithm data. A decision-transformer controller [131] treats control as sequence modelling. Earlier, DuGu [132] trained a network to imitate an optimal solver running in an omniscient emulator and validated it with real-world A/B tests. Offline learning also dominates real-time media, where operators have logs from millions of calls (Section 2.7).

## 2.4 Multiple Objectives and Application Preferences

A single reward hard-codes one operating point, yet applications need different ones: throughput for bulk transfers, low delay for interactive applications, and yielding for scavenger traffic. MOCC [70] conditions a PPO policy on a preference vector. One model then competes with the best specialized algorithm for each objective, and transfer learning lets it adapt to new objectives 14.2× faster than prior work. Zhang et al. train a family of near-Pareto policies with DDPG and select one per weighting without retraining [133], and other studies use constrained RL with Lagrangian weights [134], multi-objective RL distilled into a decision tree [135], or a client–server architecture that accepts arbitrary application objectives [105]. Deadline-aware variants [136, 137] and a learned less-than-best-effort controller [120] target specific application classes. Reward design itself is now a research topic. Zygos corrects the reward distribution with meta-gradients [138], and scenario-based modelling lets engineers inject rules into the reward [82].

## 2.5 Fairness and Multi-Agent Learning

A policy trained alone on a link learns to be greedy. When several copies share a bottleneck, nothing in a single-agent reward pushes them toward an equal share. Fairness is evaluated in 36 of the 151 Internet design studies, and the corpus offers three remedies. The first is *multi-agent training*. Pareto trains communication-free agents in stages and refines them online [139]. Astraea trains flows jointly in a "faithful" multi-flow environment with a fairness term in the reward, reaching near-optimal fairness with up to 8.4× faster convergence [74]. MAPPO-based [140] and meta-RL [141] variants follow. The second is to *decouple* fairness from the learned policy. Jury transforms network signals into a view that is common to all competing flows, and then post-processes the policy's sending rate according to each flow's estimated share of the bandwidth. Large flows thus behave more conservatively and small flows more aggressively, and fairness holds even in conditions unseen in training [142]. The third is *theory*. Learning algorithms with swap-regret guarantees provably converge to correlated equilibria among competing senders [143, 144]. Independent evaluations are less favorable. Giacomoni and Parisis find public RL controllers unfair in almost all setups [145], and Mazilu et al. show that a fairness term in the reward works in training conditions but does not generalize [146]. Fairness against the *deployed* base (CUBIC, BBR) is evaluated far less often than fairness among copies of the same learned controller.

## 2.6 Multipath Transport

Multipath TCP and QUIC add a second decision, namely how to split traffic over subflows, and a coupling constraint, namely not to take more than a single-path flow at a shared bottleneck. Learned multipath controllers appear in 21 studies across all settings (wireless ones in Chapter 3). On generic wired paths, DRL-CC [147] uses an LSTM actor-critic over all subflows and remains TCP-friendly. Partner [148] aligns the controller's reward with the scheduler's. DQL-MPTCP [149] combines deep Q-learning with policy gradients and an LSTM, and DDPG-MPCC [150] combines DDPG with online convex optimization in a kernel implementation. Bandit designs are also common. MP-OL runs a bandit per subflow that can fall back to a traditional controller [151], and MPLibra brings Libra's classic/learned competition to multipath [152]. Maggey trains a single-subflow model and composes it with non-blocking mode transitions [153].

## 2.7 Real-Time Media

Interactive video (video calls, cloud gaming, live streaming) is the application area where learned congestion control has been deployed most. 25 studies target it. The controller is a *bandwidth estimator* whose output

sets the encoder's target bitrate, replacing or tuning WebRTC's Google Congestion Control (GCC). In many designs it runs at the receiver, which estimates the bandwidth and feeds it back to the sender. Rewards are QoE terms such as video quality, frame delay and stalls.

**From simulation to production.** QARC [154] paired A3C with a video-quality predictor in trace-driven simulation. Fang et al. at Microsoft [155] trained PPO for real calls and documented the gap between simulated and real reward. Concerto [156] used imitation learning on logged sessions and was incorporated into a production video-telephony application. OnRL [157] moved learning online. Each user runs an agent, the agents' experience is aggregated into one model, and a rule-based safeguard takes over when the policy misbehaves. In a month-long production evaluation it reduced stalling by 14.2% at similar quality. Loki [158] fuses the RL model with a reverse-engineered rule-based model and was evaluated in a commercial RTC service. DuGu [132] validated imitation-learned control of video upload with A/B tests.

**Hybrids and benchmarks.** Several later designs are hybrids around GCC. HRCC [159] lets an agent set a gain on GCC's estimate, BoB warm-starts PPO with a heuristic [160], HybridRTS switches between rule-based and RL control [161], and DynA learns inside GCC with a curriculum [162]. Shared infrastructure has made this subfield unusually comparable. OpenNetLab provides a gym environment plus distributed real nodes [163]. ReCoCo trains on 162 traces and tests on more than 3,000 unseen ones [164], and GenRL-CC adds GAN-synthesized traces to it [165]. The MMSys'24 grand challenge [166] released logged Microsoft Teams data for *offline* RL, and the best entries matched the strongest behavior policy in the logs.

**Estimation accuracy.** Song et al. [167] showed that a two-stage regressor, trained by imitation and then refined online, outperforms several RL bandwidth estimators. They conclude that the accuracy of the bandwidth estimate, not reward-driven learning, determines QoE. Together with the offline-RL challenge results, this suggests that much of the benefit in real-time media comes from learning from data rather than from RL's exploration.

## 2.8 QUIC, NDN, SDN and Other Transports

QUIC's user-space implementation makes it a convenient host for learned controllers. 13 studies use it, from MVFST-RL [66] to SAC-based control [106], PPO with BBR pacing [116], multicast QUIC [168], XQUIC [169], and video-oriented controllers that report VMAF and rebuffering gains [170, 171]. Named Data Networking (NDN) changes the control point. Consumers pace Interests, and routers see per-content state, which learned controllers exploit at the receiver [172, 173] or with edge-aided congestion degrees [174]. In software-defined networks a central controller can allocate rates directly, either with tabular or deep RL [175–177] or with multi-agent cooperative Q-learning [178]. A central agent can also periodically guide end-to-end controllers instead of replacing them [179].

## 2.9 Network-Side Control: AQM and ECN

13 studies learn an *active queue management* policy at the router instead of the sender's control law. They range from early actor-critic and Q-learning AQMs [180–182] to DQN- and DDPG-based AQMs, some of which tune the parameters of RED or BLUE [183–185]. Some select the queue discipline itself, from FIFO, WFQ, RED and CoDel [186]. Network-side learning has two practical advantages: it needs no end-host changes, and it can see the queue directly. Two studies exploit this with modern data planes. An A3C agent tunes the L4S DualQ coupled AQM in FreeBSD [187], and DESiRED tunes a P4 AQM using in-band telemetry [188]. LFQ learns per-flow buffer sizes for fair queueing [189]. MACC trains a transport agent

and an AQM agent to cooperate [190]. Network-side learning is far more common in data centers, where operators control the switches (Chapter 4).

## 2.10 Summary

- **Design (RQ1).** Internet designs act on the rate (52) or the window (42), usually from a short history of throughput, delay and loss statistics, with a weighted throughput–delay–loss reward. The field has moved from clean-slate policies to hybrids that delegate fast reaction to a classic algorithm, and from purely simulated exploration toward learning from logs.
- **Evidence (RQ2).** Of the 151 Internet design studies, 29 reach a real network, 26 stop at emulation, 49 at simulation, and 47 do not report their environment in the material we could access. Only 22 explicitly test outside their training distribution, and 17 release code.
- **Deployment (RQ3).** Production use is concentrated in real-time media and in algorithm selection or parameter tuning (TCP-RL, Concerto, OnRL, Loki, Floo, DuGu). Most of these keep a classic algorithm or a safeguard in the loop; the two that do not (Concerto, DuGu) were imitated from logs or from an optimal solver rather than learned by exploration.

# 3 Wireless, Cellular, Satellite and IoT Networks

Wireless links break the assumptions behind loss- and delay-based congestion control in several ways at once. Losses come from fading and interference as well as from queue overflow. Capacity on a cellular link can change by an order of magnitude within a second, and the base station's deep per-user buffers turn any overshoot into seconds of delay [7]. Wi-Fi aggregation and scheduling make delay noisy. LEO satellite paths see abrupt changes in RTT and capacity, partly caused by frequent satellite handovers [191]. Constrained IoT devices have little memory or energy for a large model. These problems are why learned controllers are attractive here. An agent can learn the link's pattern instead of a fixed rule, and wireless systems often expose *cross-layer* information (signal quality, scheduling, handover notices) that a learned policy can use.

This chapter covers 97 studies: 48 on wireless and cellular links, 15 on satellite and space networks, and 34 on IoT, sensor and ad hoc networks. Of these, 92 are design studies. The evaluation evidence is weaker than for the Internet. Half of the design studies (46) stop at simulation, only 10 reach a real network, and 4 release code (Section 3.7).

**Scope note: vehicular "congestion control".** In vehicular networking, *decentralized congestion control* (DCC) refers to controlling the load that periodic safety beacons place on the shared radio channel, by adapting each vehicle's beacon rate, transmit power or data rate. Many RL studies address it, and our query retrieves them because they use the phrase "congestion control". DCC is a medium-access problem with different objectives (channel busy ratio, awareness range) and no end-to-end flows. We therefore excluded it (EC2, Section 1.6) and include vehicular studies only when they control a transport flow, as in Section 3.2.

## 3.1 Cellular and Mobile Links

**Learning on top of existing TCP.** The most influential cellular design is a hybrid. Abbasloo et al.'s DeepCC [192] is a DDPG *plug-in* that caps the congestion window of an existing TCP (CUBIC, BBR, Westwood, Illinois) so that the flow meets an application's target delay. It beat both cellular-specific schemes and clean-slate learned controllers, and it shares its plug-in philosophy with Orca, by the same authors (Section 2.2). Later designs tune BBR rather than replace it. BBR-FIT doubles BBR's link utilization on 5G without extra delay by letting an agent set BBR's gain [193], and in MIQA a learned bandwidth predictor steers TCP and QUIC controllers on a mmWave testbed [194]. A digital twin supplies an actor-critic agent with a model of the 5G network for tuning QUIC [195]. Muses [196] takes a different route to deployability. It imitation-learns an LSTM controller and then distils it into a small decision tree for each environment, which is cheap to run per flow.

**Using radio information.** DRL-3R [197] feeds radio-access-network information into the state and redistributes delayed reward to the actions that caused it. Flecto [198] runs a DQN-family agent inside QUIC on a 5G testbed and reports about 6% higher throughput than CUBIC with lower RTT (21.8 vs. 27.6 ms). A cross-layer mmWave controller first classifies line-of-sight versus non-line-of-sight conditions [199]. At the network side, RLRBM sizes the RAN buffer itself [200]. Radio fronthaul is a special case: Martins et al. train TD3 for C-RAN fronthaul congestion and then distil the policy into depth-limited decision trees with sub-microsecond inference [201], or into closed-form expressions by symbolic regression that match the RL policy's utilization, delay and fairness [202]. That is a rare example of a learned controller made fast enough for a hardware datapath.

**The training pipeline.** Two cellular studies examine the learning pipeline itself. Naqvi et al.'s ablation [203] finds that clipping and scaling of inputs help, that initialization affects results, and that *a higher training reward does not imply better network performance*. Network-segmented model specialization trains separate models for segments of the condition space and raises the QoS of existing DRL controllers [204]. Mobile real-time media is covered in Section 2.7. OnRL [157], the largest production evaluation in this setting, belongs there, and so do the Arsenal evaluation platform [205], the Gymir5G simulation platform [206] and 5G WebRTC uplink controllers [207, 208].

## 3.2 High-Speed Rail and Vehicular Transport

Trains and vehicles combine high mobility with frequent handovers. On high-speed rail, DRL controllers trained on field-measured conditions beat traditional TCP in simulation [209], a handover-aware design improves goodput [210], and a space–ground multipath design keeps throughput through frequent handovers [211]. For V2X transport flows, ICCDRL improves throughput, convergence, friendliness and fairness in ns-3 [212], and DARA forecasts per-path state with a Transformer and lets a DQN split traffic across vehicular paths [213]. Apart from DARA, which uses emulation, these studies report simulation results only or do not report their environment.

## 3.3 Wi-Fi, UAV and Other Wireless Links

**Loss differentiation.** A classic wireless problem is that a loss-based controller cannot tell a corruption loss from a congestion loss. Several RL designs learn the distinction from features of the loss event and then choose the window response, from an early design that learns wireless versus congestion loss [214] to actor-critic and DRL controllers that report 16–40% throughput gains [215, 216]. Others combine DRL with fast recovery for high-loss links [217], or let Q-learning choose among linear, polynomial and exponential window growth [218].

**Wi-Fi.** Wi-Fi studies are recent and often address fairness. A DQN switches each flow among CUBIC, BBR and PCC to prevent extreme unfairness and starvation [219]. A TD3 agent at the home router throttles bulk torrent traffic to protect interactive flows [220]. A DRL regulator tunes BBR's pacing and window gains on a Wi-Fi 7 campus network using eBPF signals, reporting 22% more throughput than BBRv3 and a Jain index of 0.93 [221]. Nuwa tunes a receiver-side heuristic with PPO to limit throughput loss when a device switches access points [222]. Two designs move the agent off the device. Jazz and TCCO run a Transformer-based DRL agent on an edge server that controls the window of nearby Wi-Fi clients, and both were evaluated on real Wi-Fi testbeds [223, 224].

**UAV and contested links.** Flying ad hoc networks have capacity that swings with distance and attitude. PPO with a continuous action keeps utilization high and p95 RTT low in ns-3 [225], and an A2C controller for UAV-assisted wireless networks improves on a Q-learning baseline in simulation [226]. Under jamming, a hierarchical DRL agent jointly chooses physical-layer anti-jamming actions and the congestion response [227]. Multi-band links [228] and private wireless networks with in-band telemetry [229] have also been studied.

## 3.4 Multipath over Heterogeneous Radios

Phones and vehicles with Wi-Fi and cellular interfaces are the main use case for multipath transport. SmartCC [230] applies asynchronous Q-learning with hierarchical tile coding to MPTCP. He et al.'s

DeepCC [231] uses multi-agent DRL with self-attention and converges about 50% faster than prior DRL designs. RMTC adds pacing and considers TCP-friendliness and head-of-line blocking [232], and a two-stage design targets AR/VR traffic [233]. Leopard [234] learns offline and then fine-tunes online. In new conditions it converges 20% faster than MPCC and achieves 13% more throughput than DRL-CC on a real testbed. Multipath also appears in satellite and IoT settings below.

## 3.5 Satellite and Space Networks

Satellite links combine long and changing RTTs, non-congestion loss and, for LEO constellations, frequent handovers. AUTO [235] uses multi-objective RL with a preference-adaptation model for satellite–ground links. It beats the state of the art in emulation and on the real Internet, is robust to random loss and rapid changes, and stays fair to other algorithms. LEO-specific designs make the agent aware of handovers. Creo uses DRL while a connection is stable and cross-layer handover notifications around a handover, cutting handover recovery time by 60.5% in simulation [236]. Other designs target hybrid HEO–LEO constellations [237] or integrated space–air–ground networks with heterogeneous multi-agent [238] or hierarchical [239] agents. For non-terrestrial QUIC, RL rate control combined with streaming FEC shortens in-order delivery delay on lossy long-delay links [240]. Deep-space and delay-tolerant networks are an extreme case where end-to-end feedback barely exists. Here, RL has been combined with an incomplete-information game to cut bundle aborts in satellite IoT [241], applied to TCP-like window control with an optimistic actor-critic [242], and used for multipath deep-space video [243].

**Simulation versus emulation.** One methodological study bears on all of this work. Valentine et al. [244] ran the same controllers (CUBIC, BBR, LEO-specific designs and an RL controller) in an OMNeT++/INET simulator and in emulation with real stacks. The simulator misses stack effects such as pacing, SACK/RACK recovery and kernel timing, and these effects change the RL controller's behavior. They conclude that emulation is necessary to validate learned controllers for LEO, which matters because most satellite RL studies in the corpus report simulation results only or do not report their environment (Fig. 5.1).

**Finding 3.1.** In wireless, satellite and IoT settings, half of the design studies are evaluated in simulation only, yet simulators of these links omit stack effects that change how learned controllers behave. Results from this part of the literature should be treated as provisional until they are reproduced in emulation or on hardware.

## 3.6 IoT, Sensor and Ad Hoc Networks

**Constrained application protocols.** CoAP, the IoT's REST protocol over UDP, has its own congestion control based on retransmission timeouts. RL agents adapt its parameters [245, 246], choose how many copies of a message a node sends [247], or jointly control CoAP and QUIC with traffic-class-aware rewards that keep critical traffic below 100 ms [248]. An MQTT broker can also be controlled with DDPG [249]. One of the two earliest studies in the corpus, a Q-learning TCP for memory-constrained IoT devices [250], already used function approximation to reduce memory use. Constrained resources remain the defining concern in this setting. KONTROL offloads the DRL agents of light clients to a central congestion-control server [251], and SymDistill-CC distils PPO into symbolic decision trees for power-IoT terminals [252].

**Sensor, mesh and ad hoc networks.** In wireless sensor networks the agent often runs at intermediate nodes and controls the forwarding rate or buffer. Examples include an energy-aware Q-learning controller [253], a learning-automata controller for Zigbee that reports lower energy and delay [254], cache-aware DRL controllers [255, 256], and DLCC for 6LoWPAN, which formulates congestion control as a non-cooperative

Markov game with a Lagrangian loss constraint [257]. Hop-by-hop RL flow control appears in lossy and maritime networks [258, 259], and RL-based TCP variants appear in MANETs, mesh and ad hoc networks [260–264]. Delay-tolerant networking motivated one of the first RL congestion controllers in the corpus, Smart-DTN-CC [265].

**Smart grid and industrial IoT.** Power-grid and industrial IoT communication is a growing niche, with seven studies covering smart meters, power IoT, substations and industrial multipath transport [252, 266–271]. Its distinguishing requirements are reliability and security. One study, for example, couples congestion control with recovery from GOOSE-message flooding attacks [270]. Two IoT studies evaluate beyond simulation. TransAL-CC evaluates in Pantheon emulation [269]. Kutsevol et al. train a *semantic* congestion controller for cyber-physical systems, whose objective is the accuracy of a remote estimator rather than throughput, offline on real traces and validate it on hardware [272].

## 3.7 Summary

- **Design (RQ1).** Wireless designs act on the window more often than the rate (41 vs. 22 design studies). Cross-layer radio state appears almost only in wireless designs, and wireless designs more often place the agent inside the network or at an edge server (16 network-side and 6 central agents among 92 design studies). Hybrid designs are less common (25%) than on the Internet (30%), even though the strongest cellular results come from hybrids.
- **Evidence (RQ2).** 50% of design studies are evaluated in simulation only and 36% do not report their environment in the material we could access. Only 10 reach a real network and 4 release code. Simulators of radio links and satellite stacks miss effects that change learned controllers' behavior [244], so simulation-only results are less dependable here than in the other settings.
- **Deployment (RQ3).** Deployability work focuses on cost. Distillation into trees or closed forms [196, 201, 202, 252] and offloading to edge servers [223, 224, 251] are the main techniques. Production evidence is limited to mobile real-time media (OnRL).

# 4 Data-Center Networks

Data centers differ from the Internet in ways that change what RL is used for. Round-trip times are tens of microseconds, so a neural network cannot sit in the per-packet or even per-RTT loop of a host's transport stack. Switch buffers are shallow, and many-to-one *incast* traffic can overflow them within a few RTTs. RDMA fabrics add lossless operation through Priority Flow Control, with its own pathologies. On the other hand, a single operator controls the hosts, the NICs and the switches. Switches can mark ECN and export in-band telemetry, and workloads are measurable. The classic designs (DCTCP, DCQCN, TIMELY, HPCC, Swift; Section 1.3) therefore depend on parameters, above all the ECN marking threshold and the rate-increase and rate-decrease constants, that operators tune by hand for their traffic.

The 24 data-center studies in the corpus reflect this. More than half of the 23 design studies are hybrid (57%, the highest share of any setting). The agent sets host rates in 10 studies, but in 6 it tunes switch ECN thresholds and in 3 the parameters of an existing scheme. Multi-agent RL is the most frequently named family (6 of the 11 studies that name their algorithm), because switches are natural agents.

## 4.1 Learning to Tune ECN at the Switch

Static ECN thresholds face a trade-off. A low threshold keeps queues short and protects latency-sensitive short flows, whereas a high threshold keeps throughput high for large flows. The right setting depends on the traffic mix, which changes over time. ACC [273] made this a multi-agent RL problem. Every switch runs an agent that adjusts its ECN marking threshold at run time, the agents work in a distributed fashion, and they rely only on features of commodity switching chips. Training combines an offline phase with online refinement. In heterogeneous production environments with 300 machines, ACC achieved up to 20% higher storage IOPS and 30% lower flow completion time (FCT) than well-tuned static thresholds, and it has been applied in production data centers. ACC is the reference design for this line of work, much as Orca is for the Internet.

Its successors address ACC's independent agents and its generalization. GraphCC [274] lets switch agents exchange messages through a graph neural network. It beats ACC in every scenario tested, including unseen workloads and link failures, with up to 20% lower FCT and 38–86% lower buffer occupancy. EMPRN uses a similar message-passing design with recurrent state [275]. LAECN makes the tuner hierarchical and lightweight [276]. BBQ bounds the learned threshold by the per-port buffer to keep incast loss below 3% in shallow-buffer switches [277]. A dual-agent design couples a host-rate agent and a switch-ECN agent through a centralized critic and evaluates it on the BMv2 software switch [278]. Two further designs go beyond ECN. RSCAT classifies congestion at an SDN controller and lowers FCT for DCTCP and CUBIC without changing end hosts [279], and D$^2$T* uses DRL to tune dynamic buffer-sharing thresholds on programmable switches, cutting short-flow FCT by 29% on average and 62% at the tail [280].

## 4.2 End-Host and NIC Rate Control

**RL-CC: from simulation to NICs.** Tessler et al. [281] formulated RDMA congestion control as a multi-agent problem in which every flow runs the same policy. Their policy-gradient method uses an analytic derivative of the reward, and in simulation it beat both RL baselines and production data-center algorithms and generalized to unseen scenarios. The follow-up by NVIDIA [282] is the clearest account in the corpus of what deploying a learned controller in hardware requires. The neural policy was too slow for the NIC's microsecond budget, so it was *distilled into decision trees*, which cut inference time by a factor of 500. On

live NICs, RL-CC was the only algorithm that did well on all benchmarks across a wide range of flow counts. The path from paper to product thus required both a training method that generalizes across flow counts and a change of model class to fit the datapath.

**Other host-side designs.** For RDMA in HPC clusters, Dolce-RC runs a multi-armed bandit on a SmartNIC that selects among rate-increase and rate-decrease rules, cutting MPI message completion time by up to 68% relative to DCQCN without switch changes [283]. RDI combines online and offline learning for RDMA in edge data centers [284]. Several designs act on the window or rate with ECN or neighbor information, among them DECC [285], DuetCC [286], NCC [287] and others [288, 289]. Two designs target power-utility data centers [290, 291]. A multi-agent design tunes BBR's parameters inside the data center [292], and a hierarchical selector invokes RL-based algorithm selection only when conditions change, which reduces its overhead by 31% [293]. SDN controllers with a global view can also allocate rates centrally, with Q-learning or SARSA [294] or with DRL that anticipates incast [295].

## 4.3 Training Infrastructure

Iroko [296] was an early gym-style environment for data-center congestion control. It emulates dumbbell and fat-tree topologies and exposes them to PPO, REINFORCE and DDPG agents. Its DRL agents beat TCP New Vegas, but its authors also reported training instability and overfitting to the training topology. Those warnings anticipate the generalization work of Chapter 5. Since then, most data-center studies have trained in packet-level simulators, either custom ones or ns-3 and OMNeT++. Only 1 data-center design study releases code, so the results of this subfield are the hardest in the corpus to reproduce.

## 4.4 Summary

- **Design (RQ1).** Data-center RL more often acts at a slower timescale and inside the network than Internet RL does. It tunes ECN thresholds, buffer thresholds or the constants of existing algorithms, often with one agent per switch. The one host-side neural controller deployed on NICs had to be distilled into decision trees to fit the microsecond budget, and a SmartNIC design avoids neural inference altogether by letting a bandit choose among rule sets.
- **Evidence (RQ2).** Of the 23 design studies, 10 are evaluated in simulation only, 3 in emulation, 4 on real hardware, and 6 do not report their environment. Only 3 explicitly test generalization.
- **Deployment (RQ3).** Relative to its size, this is the setting with the strongest deployment evidence: ACC runs in production, and NVIDIA's RL-CC runs on live NICs. Both succeeded because the operator controls the whole fabric and because the learned component is either slow-timescale (ECN tuning) or distilled. Both results come from industry. Of the two, only RL-CC's code is public [282].

# 5 Training and Testing: From Simulators to Real Networks

This chapter answers **RQ2**: how RL congestion controllers are trained and tested, and whether they still work on networks they did not see. We first define the *evaluation ladder* from simulation to production (Section 5.1) and measure where the corpus stands on it (Section 5.2). We then examine evaluation practices (Section 5.3) and the evidence on generalization (Section 5.4). A small reproducible case study isolates the effect of the training distribution (Section 5.5), and we close with recommendations (Section 5.6).

## 5.1 The Evaluation Ladder

We distinguish four rungs, ordered by how much of the real system they include.

1. **Simulation** models both the network and the endpoints. Packet-level discrete-event simulators (ns-2, ns-3, OMNeT++/INET, ndnSIM) model queues and protocols in detail. Custom simulators, often fluid or packet-level Python models such as Aurora's, trade fidelity for speed. Gym bridges connect RL libraries to them: ns3-gym [297], Park [298], RayNet for OMNeT++ [299], and Gymir5G for 4G/5G [206]. Simulation is fast, controllable and repeatable. However, it leaves out the operating system's network stack: timers, pacing, segmentation offload, ACK aggregation and recovery heuristics.
2. **Emulation** runs real protocol stacks and applications over links whose behavior is imposed in software. Mininet [300] builds virtual topologies, and Mahimahi [301] replays recorded cellular or wired link traces. NetEm adds delay, loss and jitter. Pantheon's emulators are calibrated so that their results match real Internet paths [65], and iBox learns path models from traces [302, 303]. Emulation exercises the real controller code, but the link is still a model.
3. **Shared and hardware testbeds** put real stacks on real links under controlled conditions. Examples are Pantheon's nodes on wired, cellular (LTE/UMTS) and Wi-Fi networks in ten countries [65], OpenNetLab's distributed nodes for real-time media [163], 5G, Wi-Fi and satellite testbeds, and NIC or switch labs in data centers.
4. **Real networks and production** expose the controller to real paths and traffic: experiments between cloud regions, A/B tests on a fraction of users, or full deployment.

In our coding, rung 3 counts as "real network" when the links are physical, and as "emulation" when they are emulated. Table B.1 in Appendix B lists what "real" means for every study at that level.

**A note on Wireshark.** Our project proposal described the path "from simulators (Wireshark) to shared testbeds (Pantheon) and real networks". That was a mistake that we correct here. Wireshark is a *packet analyzer*: it captures and decodes traffic on a real or emulated interface. It does not generate traffic or model a network, so it is not a simulator. Packet capture is useful on every rung, for example to inspect a testbed trace, to validate an emulator against a real path, or to debug a controller's behavior in a kernel. The simulators in this literature are ns-2, ns-3, OMNeT++ and custom models, and the emulators are Mininet, Mahimahi and NetEm.

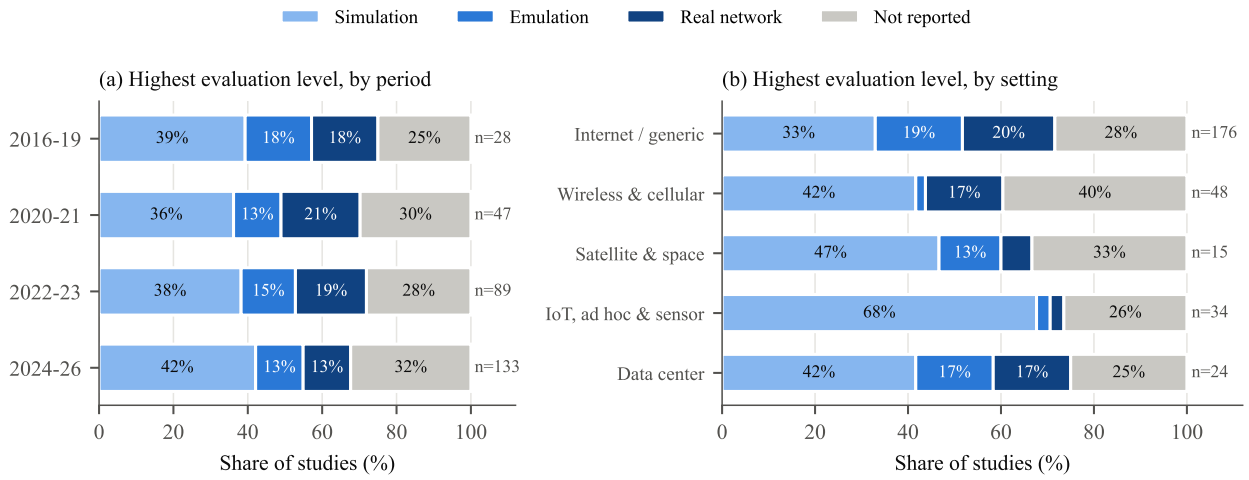



**Figure 5.1:** Highest evaluation level of the 297 empirical studies, (a) by period of first report and (b) by setting. Segments without a label are below 9%. "Not reported" means the level could not be determined from the material we could access.

## 5.2 Where the Corpus Stands

Figure 5.1 shows the highest evaluation level reached by the 297 empirical studies (all included studies except formal analyses). 118 studies (40%) stop at simulation, 41 (14%) at emulation, and 49 (16%) reach a real network. The remaining 89 (30%) do not report their environment in the material we could access. These are mostly studies coded from abstracts, which typically say "extensive experiments" without saying where. Among the 208 studies whose level is known, 57% are simulation-only and 24% reach a real network. In the subset coded from full texts, which is more complete but skewed toward prominent venues, 19% reach a real network.

**Trend over time.** Evaluations have become less realistic as the field has grown, even though tools such as Pantheon, Mahimahi and OpenNetLab have become standard (Fig. 5.1a). Studies first reported in 2020–21 reached a real network in 21% of cases and those from 2022–23 in 19%. For 2024–26 the share is 13%, while the simulation-only share rose from 36% to 42%. Simulation-only studies account for the largest part of the field's recent growth, and they dominate the IoT and wireless settings (Fig. 5.1b). By setting, Internet studies are the most realistic, and IoT, ad hoc and sensor studies the least. Two thirds of the latter are simulation-only.

**Finding 5.1.** As the field grew, the share of studies that reach a real network fell from 21% (2020–21) to 13% (2024–26). Only 11% of design studies test outside their training distribution, and only 8% release code.

**What "real" means.** Even the 49 real-network studies vary widely (Table B.1). Most run between cloud or testbed hosts over Internet paths. Deployment in production systems or user-facing tests is rare; Table 6.1 lists the nine studies that report it. Examples are OnRL's month-long evaluation on a live-streaming platform [157], Loki in a commercial RTC service [158], ACC in production data centers [273], TCP-RL at a search engine [125], and Floo in a mobile web service [126].

**Where agents are trained.** The training environment is reported by 174 of the 266 design studies. Simulators dominate (116, of which 36 use ns-3), followed by emulators (36), testbeds (12), real networks (6) and logged datasets (4). Training in simulation is reasonable, since exploration on a real network is slow and

**Table 5.1:** Evaluation practices of design studies: all 266 design studies and the 64 coded from full texts. "Tested on conditions outside the training range" requires an explicit train/test split along at least one network parameter.

| | All design studies | | Full-text subset | |
|---|---|---|---|---|
| Practice | *n* | % of 266 | *n* | % of 64 |
| Code or artifact released | 22 | 8 | 17 | 27 |
| Tested on conditions outside the training range | 30 | 11 | 13 | 20 |
| Fairness or TCP-friendliness evaluated | 54 | 20 | 24 | 38 |
| Evaluated in emulation or on a real network | 75 | 28 | 27 | 42 |
| Evaluated on a real network | 43 | 16 | 13 | 20 |
| Evaluation environment not reported | 86 | 32 | 4 | 6 |

risky. However, it makes testing outside the simulator essential, and that testing is what the corpus most often lacks.

## 5.3 Evaluation Practices

Table 5.1 reports how often design studies follow key evaluation practices, across all 266 design studies and across the 64 whose full text we coded. Because abstracts omit details, the whole-corpus figures are lower bounds; the full-text figures come from a sample that over-represents prominent venues and are better read as upper bounds.

**Code and artifacts.** Only 22 design studies (8%) release code, and even in the full-text subset only about a quarter do. In the full texts we read, comparisons with learned baselines therefore tend to involve the few controllers with public code (Aurora, Orca, Indigo, and the online-learning PCC Vivace), sometimes re-implemented. Newer learned designs are rarely compared with each other.

**Generalization testing.** Only 30 design studies (11%) explicitly test on network conditions outside their training range. Many others evaluate on held-out traces or seeds from the *same* distribution. That measures overfitting to individual traces but not robustness to distribution shift.

**Fairness and friendliness.** Fairness is evaluated in 54 design studies (20%). In the full texts we read, most of these evaluations measure Jain's index among copies of the same controller rather than the *harm* done to the deployed base [21].

**Practice scores.** Combining the five practices into the appraisal score of Section 1.6.5 shows how rarely they occur together. The median design study satisfies only 1 of the five (2 in the full-text subset). Of all design studies, 153 (58%) satisfy at most one, and only 11 satisfy four or more: Orca [67], Aurora [17], MOCC [70], Genet [71], MARLIN [92], Astraea [74], Canopy [75], Closed-form CC via symbolic regression [202], DDPG-MPCC [150], Jury [142], Mutant [76]. Of these, only Orca satisfies all five. These exemplars carry the most weight in this survey's conclusions.

**Independent evaluations.** Four independent large-scale evaluations re-run many published learned controllers under common conditions. Abbasloo et al. propose general benchmarks that rank Internet controllers and find large room for improvement [304]. Giacomoni and Parisis evaluate public RL controllers reproducibly. They find that the controllers acquire bandwidth quickly and resist random loss, but cause excess loss, are

unfair in almost all setups and lag when bandwidth or latency changes [145]. Mazilu et al. reach similar conclusions against CUBIC and BBRv3 and show that fairness terms in the reward do not generalize [146]. CCLab attacks controllers with an RL adversary that perturbs inputs or the environment within bounds. It finds learned controllers (Orca, Canopy) generally *more* robust than classic ones, and it shows that adversarial traces train more robust controllers [305]. Shared benchmarks have also emerged for real-time media, namely OpenNetLab, the MMSys'24 offline-RL challenge and ReCoCo's train/test trace split [163, 164, 166], along with automated assessment frameworks [306] and a survey of congestion-control evaluation methodology in general [64].

## 5.4 Evidence on Generalization

**Evidence of failure.** Learned controllers' sensitivity to their training assumptions was demonstrated before deep RL: Remy's machine-generated rules degraded when the network differed from the assumed model [27]. The corpus contains recent examples. Ghermezian et al. deployed an Aurora-style PPO controller, trained in simulation, on a 10 Gbit/s wide-area path. It reached 0.5% mean and 4.4% peak utilization of the path, whereas CUBIC and BBR reached up to 9.9 and 9.7 Gbit/s. The authors observe overshoot-and-collapse dynamics and call for scale-aware training [307]. Iroko's agents overfit to the training topology [296]. Fang et al. document a gap between simulated and real reward for RTC [155]. A cellular ablation shows that higher training reward need not mean better network performance [203]. Valentine et al. show that a simulator's missing stack effects change an RL controller's behavior over LEO links [244]. Adversarial studies find conditions, or competing flows, that degrade learned controllers severely [308, 309]. One input-perturbation attack cuts throughput by about half and raises latency more than 36-fold [310].

**Evidence of success.** Aurora generalized to bandwidths more than 20× above its training range, though not to links where latency noise it never saw dominated the signal [17]. Tessler et al.'s RL-CC generalized to unseen data-center scenarios and later to live NICs [281, 282], and GraphCC handled unseen workloads and link failures [274]. ReCoCo's best policies kept a 3–20% QoE advantage on more than 3,000 unseen traces [164]. The successes share a pattern: deliberately broad or adversarial training data, an architecture that limits the damage a bad decision can do, or both.

**Methods for generalization.** Table 5.2 organizes the methods proposed in the corpus into five families. The most widely used approach, hybridization, works mainly by bounding the damage a wrong decision can do rather than by making the policy itself generalize. We return to this distinction in Chapter 6.

## 5.5 Case Study: How the Training Distribution Shapes Behavior Outside It

Published studies rarely vary the training distribution while holding everything else fixed, so the corpus cannot say how much of a controller's out-of-distribution behavior is due to *what it was trained on*. We therefore ran a small controlled experiment. Everything needed to reproduce it, including the environment, training, evaluation and figures, is in the replication package (`case_study/`). Training takes about 9 minutes per seed on one CPU core.

**Setup.** The environment is a fluid model of a single drop-tail bottleneck with Aurora's formulation [17]. The agent acts every 20 ms monitor interval, and its feedback is delayed by one base RTT. It observes the latency gradient, latency ratio and sending ratio of the last ten intervals. Its action $a \in [-1, 1]$ multiplies the rate by $(1 + 0.05a)$, or divides it by $(1 - 0.05a)$ when $a < 0$, and its reward is utilization minus queueing

**Table 5.2:** Methods proposed in the corpus to make learned controllers generalize, grouped by mechanism.

| Family | Mechanism | Examples |
|---|---|---|
| Shape the training distribution | Randomize link parameters; schedule environments by difficulty or by gap to a baseline; rebalance skewed traces; synthesize or adversarially generate traces | Aurora [17], Genet [71], Plume [311], GenRL-CC [165], CCLab [305] |
| Choose invariant inputs and rewards | Scale-free features; reward normalized by a capacity estimate; signals transformed to a common view across flows | Aurora [17], Pacemaker-CC [111], Jury [142] |
| Adapt after deployment | Online learning in the field; transfer or fine-tuning; meta-RL with fast adaptation; specialized models per condition segment | OnRL [157], TRCC [312], MOCC [70], meta-RL [88, 313, 314], NMS [204] |
| Learn from real data | Offline RL or imitation from logs of deployed controllers; training directly on real networks | Sage [72], MMSys'24 [166], Concerto [156], ARC [90], MARLIN [92] |
| Bound the policy | Hybrid control with a classic algorithm; fallback under uncertainty; certified properties; verification to pick policies that generalize | Orca [67], Spine [73], HAN [117], Canopy [75], Amir et al. [87] |

delay in RTTs minus three times the loss rate. We trained PPO [34] for 5 million steps with 4 seeds under each of two training distributions:

- **Narrow**: 1–5 Mbit/s (similar to Aurora's training bandwidths), 40–60 ms base RTT, buffers of 1–2 bandwidth-delay products (BDP), with no random loss, no RTT jitter and constant capacity.
- **Broad**: 1–100 Mbit/s, 10–200 ms, 0.2–5 BDP, 0–3% random loss, 0–10 ms mean RTT jitter, and capacity that varies over time.

The baselines are a window-based, ACK-clocked Reno-like AIMD with slow start and a BBR-like controller with BBRv1's state machine and its in-flight cap of 2 BDP. We evaluated every controller on 10-second flows, ramp-up included, while varying one factor at a time around a reference link inside the narrow range (3 Mbit/s, 50 ms, 1.5 BDP).

**Results.** Figure 5.2 and Table 5.3 give the results (queueing delay is in Fig. B.1 in Appendix B).

1. **In distribution.** Both learned policies reach at least 94% utilization with queueing delay of 0.03–0.09 RTT, against 0.72 RTT for AIMD. On its training distribution, the narrow policy scores best of all four controllers on the objective it was trained for (utilization minus queueing delay), slightly ahead of BBR-like, which has higher utilization but a larger queue.
2. **RTT jitter.** With 10 ms of mean RTT jitter, which is ordinary on Wi-Fi and cellular links, the narrow policy's utilization falls to 29%. Its latency features read the noise as queue growth, and it backs off. AIMD, which ignores delay, and the BBR-like controller are unaffected (99% and 100%). Aurora's authors observed the same failure mode: their policy performed poorly at 1 ms latency because about 1 ms of packet-processing noise, absent from training, dominated the latency signal [17].
3. **Broader training.** The broad policy, which saw up to 10 ms of jitter, keeps 69% at 10 ms but drops to 49% at 40 ms, outside its range. In distribution it queues more (0.09 vs. 0.03 RTT). Training was also less reliable: one of 4 broad seeds collapsed to 3% utilization even on the reference link, whereas every narrow seed converged.

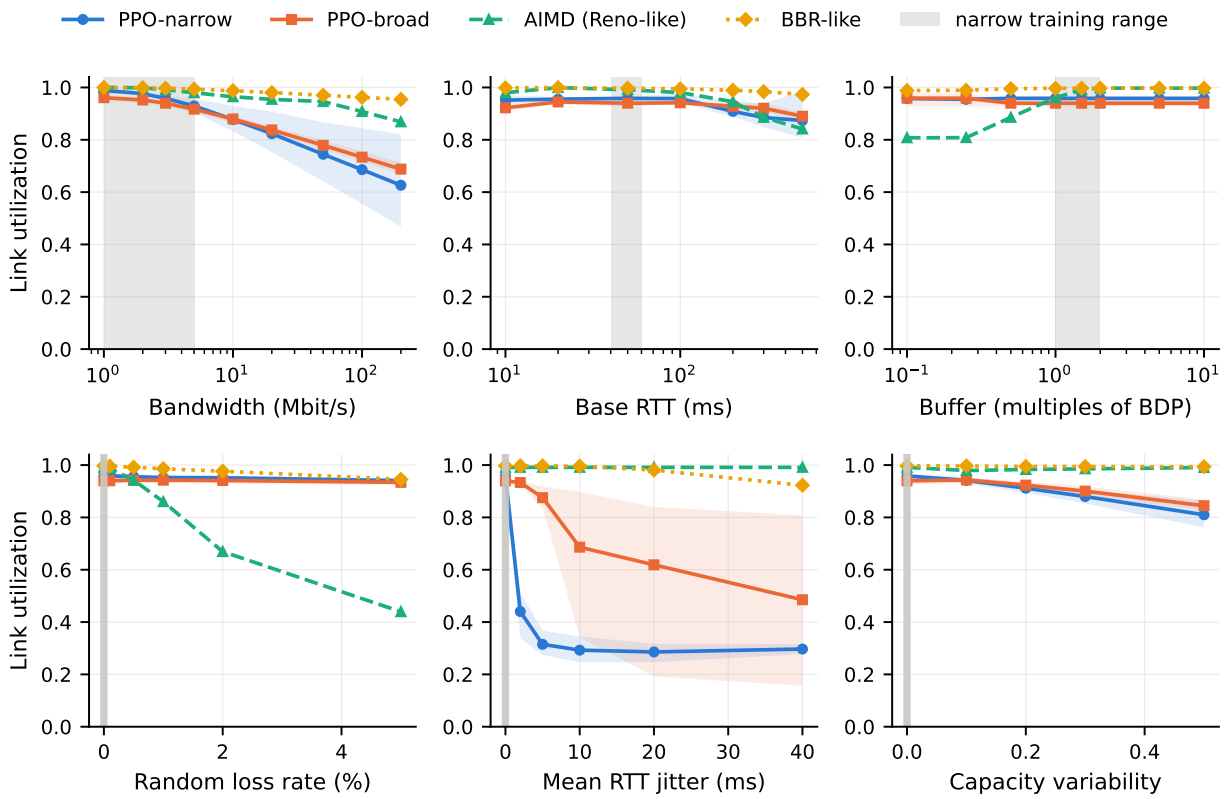



**Figure 5.2:** Case study: link utilization when one factor is varied around the reference link. Lines are means over the converged training seeds (as in Table 5.3) and shaded bands span them; the one collapsed broad seed is excluded. The gray band or line marks the *narrow* training range. The broad training range covers 1–100 Mbit/s, 10–200 ms, 0.2–5 BDP, 0–3% loss, 0–10 ms jitter and capacity variability up to 0.3.

4. **Random loss.** With 5% random loss the PPO policies keep 93–94% utilization, while AIMD falls to 44%. Their weakness is delay noise. A test suite designed around classic controllers' known weaknesses (random loss, shallow buffers) would rate the learned controller highly and miss its actual failure mode.

5. **High bandwidth.** At 200 Mbit/s both PPO policies reach only 63–69% utilization, against 95% for BBR-like, even though the broad policy trained up to 100 Mbit/s. Part of the gap is structural. An Aurora-style action changes the rate by at most 5% per interval, so ramping up over two orders of magnitude takes seconds whatever the training data.

**Finding 5.2.** A learned controller's failure modes follow from its input features. Latency-based policies that perform well in distribution can collapse under ordinary delay noise that classic controllers ignore. In our experiment, broadening the training distribution helped only within the range it covered, and it reduced in-distribution quality and training stability.

**Caveats.** The model has one flow, a fluid queue and no competing traffic, and the policies are small. The experiment isolates the effect of the training distribution; it makes no claim about any published controller.

**Table 5.3:** Case study summary: mean over converged seeds and five flows per condition. "Queue" is the mean queueing delay in multiples of the base RTT. One of the 4 broad training seeds collapsed (3% utilization even in distribution) and is excluded here; all 4 narrow seeds converged.

| | In distribution | | Utilization (%) under one shifted factor | | | |
|---|---|---|---|---|---|---|
| Controller | Util. (%) | Queue (RTT) | 200 Mbit/s | 5% loss | 10 ms jitter | Worst case |
| PPO-narrow | 96 | 0.03 | 63 | 94 | 29 | 29 (20 ms jitter) |
| PPO-broad | 94 | 0.09 | 69 | 93 | 69 | 49 (40 ms jitter) |
| AIMD (Reno-like) | 99 | 0.72 | 87 | 44 | 99 | 44 (5% loss) |
| BBR-like | 100 | 0.09 | 95 | 95 | 100 | 92 (40 ms jitter) |

## 5.6 Recommendations for Evaluating Learned Controllers

From the corpus analysis and the case study we derive a reporting checklist (Table 5.4). Several of its items echo advice from the evaluation literature [64, 145]. We add a measurement of how often the corpus follows them.

**Table 5.4:** Checklist for evaluating RL congestion controllers. The rightmost column gives the share of design studies that already satisfy the item, where the corpus coding allows it to be measured.

| # | Item | Corpus |
|---|---|---|
| C1 | State the training environment and the highest evaluation level (simulation, emulation, testbed, real, production) and name the tools. Do not call emulation "real-world". | 68% |
| C2 | Report the training distribution, and test at least one factor outside it. Include delay noise, capacity variation and competing traffic, not only random loss. | 11% |
| C3 | Before claiming deployability, evaluate with real stacks: emulation at minimum, and trace-driven emulation for wireless and satellite links. | 28% |
| C4 | Compare with the deployed base (CUBIC, BBR) and at least one public learned controller, and measure harm to competing flows, not only Jain's index among copies. | 20%[a] |
| C5 | Report results over several training seeds, including failed runs and their variance. | – |
| C6 | Report inference cost, the decision interval, and where the model runs (kernel, user space, NIC, controller). | – |
| C7 | Report worst-case and tail metrics alongside means. | – |
| C8 | Release code, trained models and traces. | 8% |

[a] Share that evaluates fairness or friendliness in any form.

# 6 Deployment: Barriers, LLM-Designed Controllers, and Open Problems

This chapter answers **RQ3**: what stops operators from deploying RL-based congestion control, and what research could fix it. We first examine the deployments that did happen (Section 6.1), because they show which barriers turned out to be surmountable. We then map nine barriers to the evidence and mitigations in the corpus (Section 6.2) and review the newest direction: controllers designed, tuned or run by large language models (Section 6.3). The chapter closes with a research agenda (Section 6.4), threats to validity (Section 6.5) and conclusions (Section 6.6).

## 6.1 What the Deployed Systems Have in Common

Table 6.1 lists the studies in the corpus that report deployment in production or tests with real users, with the scale they report. Despite their different settings, they share three properties.

1. **The learned component is bounded.** In six of the nine systems a classic algorithm, a rule-based model or a safeguard remains in the loop. The learned part sets a parameter (an initial window, an ECN threshold), chooses among vetted algorithms, or adjusts a heuristic's output. The three exceptions (Concerto, DuGu, RL-CC) replace the controller entirely. Two of them were imitated from logs or from an optimal solver rather than learned by exploration, and the third was distilled into decision trees that can be inspected.
2. **The operator owns the endpoints, and often the data.** Every deployment is by an organization that controls the sender (a search engine, a video platform, a data-center operator, a NIC vendor) and can therefore monitor, A/B test and roll back. Several train on logs of their own traffic, which removes the simulator from the training loop. No deployment of a learned controller for *general* Internet traffic, such as a default in an operating system, appears in the corpus.
3. **The objective is narrow and measurable.** Each system optimizes a concrete service metric: transfer time of short web flows, stalls in live video, storage IOPS, QoE of a mobile page. None claims to be a general-purpose congestion controller.

**Finding 6.1.** Every deployed learned controller in the corpus is run by an operator that owns the endpoints and the data, optimizes a narrow service metric, and in most cases keeps a classic algorithm or safeguard in the loop. None is a general-purpose controller for Internet traffic.

Production systems built on *non-RL* learning show the same pattern. Antelope selects among kernel algorithms with a (non-RL) machine-learning model and was evaluated in production networks [315]. Gemini pairs a white-box parameterized controller with online Bayesian optimization of its parameters and was deployed over production networks [316]. XCC, a context-aware cross-layer controller for live streaming, is fully deployed in a cloud provider's production CDN service, where A/B tests over tens of millions of sessions cut stall time by 36% [317]; a concern-decoupled design for live-video CDNs follows the same approach [318]. These systems also favor interpretable structure with learned parameters.

## 6.2 Barriers to Deployment

Table 6.2 summarizes the barriers we identified, the evidence for each in the corpus, and the mitigations proposed. The barriers are not independent. Hybrid designs, for example, address safety, overhead and

**Table 6.1:** Learned congestion controllers reported in production or user-facing tests. "Classic in the loop" marks designs where a classic algorithm, rule-based model or safeguard remains part of the control path.

| System | Setting | Learned component | Classic in loop | Reported scale |
|---|---|---|---|---|
| SmartIW / TCP-RL [125] | Web (search) | initial window per user group (group-based RL); algorithm selection (deep RL) | ✓ | a top global search engine for more than a year; 23–29% shorter transfers |
| Concerto [156] | Video calls | bitrate policy imitated from logs | – | a production video-telephony application |
| OnRL [157] | Mobile live video | per-user online RL, aggregated model | ✓ | 543 h of sessions, 151 users; 14.2% fewer stalls |
| Loki [158] | Real-time video | RL fused with a reverse-engineered rule-based model | ✓ | trained online in a commercial system; 101 million sessions |
| ACC [273] | Data center | ECN threshold per switch (MARL) | ✓ | production fabrics, 300-machine evaluation |
| DuGu [132] | Video upload | policy imitated from an optimal solver | – | real-world A/B tests |
| RL-CC [282] | RDMA NICs | rate policy distilled into decision trees | – | ConnectX-6Dx NICs in a 64-host live cluster |
| Floo [126] | Mobile web | selection among classic algorithms | ✓ | production environment; 14.3–52.7% higher QoE |
| Marten [115] | QUIC | DRL guided by BBR, safe exploration | ✓ | production network tests |

generalization at once, which explains their spread (Section 2.2).

**B1 Generalization.** This is the barrier the community discusses most, and Chapter 5 examined it in detail. Successful deployments avoided the problem rather than solving it, by training on their own traffic (OnRL, Loki, Concerto) or by limiting what a wrong decision can do (ACC, Floo, SmartIW).

**B2 Safety.** A congestion controller that collapses its rate, or floods a queue, harms users in a way that average-case metrics hide. Canopy shows a well-known learned controller entering states from which it recovers only slowly [75]. The corpus offers two remedies. *Verification* proves properties of the trained policy or finds counterexamples to them [85, 86, 319], but it scales only to small policies and properties. *Runtime assurance* monitors the policy and falls back to a safe controller when it is uncertain or misbehaves [117, 157, 320]. Most deployed systems rely on the second.

**B3 Fairness.** A learned controller must share bottlenecks with other copies of itself, with other learned controllers, and above all with CUBIC and BBR. The corpus has made progress on the first through multi-agent training and decoupled fairness modules [74, 142]. The third is rarely evaluated, and when it is, the results are poor [145]. Delay-sensitive controllers, whose features resemble those of most RL designs, can also starve under non-congestive delay [331], and our case study shows that RL policies with latency features are sensitive to exactly that kind of noise.

**B4 Overhead.** This barrier is the most thoroughly addressed. Timescale separation, distillation, lookup tables, kernel/user-space splits and asynchronous runtimes each cut overhead substantially, by tens of percent (lookup tables, kernel/user splits) up to several orders of magnitude (distillation for NICs). The remaining

**Table 6.2:** Nine barriers to deploying RL congestion control: evidence in the corpus, proposed mitigations, and our assessment of their status.

| Barrier | Evidence | Mitigations proposed | Status |
|---|---|---|---|
| B1 Generalization under distribution shift | simulator-trained PPO at 0.5% of a 10 Gbit/s path [307]; lag under changing bandwidth or latency [145, 146]; simulators miss stack effects [244]; case study (Section 5.5) | shape the training distribution [71, 305, 311]; adapt online or by meta-RL [88, 157]; learn from logs [72, 166]; bound the policy [67, 73] (Table 5.2) | partly addressed; no method validated across settings |
| B2 Safety and worst-case behavior | learned controllers enter persistently bad states [75]; verification finds counterexamples in Aurora [85, 86] | certificates in training [75]; verification [87, 319]; runtime fallback and safeguards [115, 117, 157, 320] | runtime safeguards are what deployments use; verification scales only to small policies |
| B3 Fairness and coexistence | learned flows unfair or selfish [80, 145]; fairness rewards do not generalize [146]; harm to CUBIC/BBR rarely measured [21] | multi-agent training [74, 139]; decoupled fairness module [142]; AIMD wrapping [80]; equilibrium-seeking learning [143] | improving among copies of one controller; open against the deployed base |
| B4 Overhead and datapath integration | per-ACK neural inference infeasible; NIC budget required 500× faster inference [282]; DRL control overhead [73] | timescale separation [67, 73]; distillation to trees or formulas [84, 196, 202, 282]; lookup tables [321]; kernel/user split [322, 323]; compression [324]; asynchronous runtimes [66, 325] | solved in principle; engineering per platform |
| B5 Interpretability and debugging | operators must explain failures; black-box policies resist audit [83] | decision-tree interpretation [83]; explanations [326, 327]; programmatic and symbolic policies [84, 135, 328] | research prototypes, not yet in operations |
| B6 Adversarial robustness | perturbations cut throughput by half [310]; stealthy competing flows [309]; adversarial conditions [308] | adversarial training and testing [305, 309]; robust RL [270] | early |
| B7 Objective specification | the reward fixes the operating point; higher reward ≠ better network [203]; reward choice changes trade-offs [108] | preference-conditioned and multi-objective RL [70, 133]; constrained RL [134]; reward correction [138]; LLM-written utilities (Section 6.3) | active; application-specific objectives are what deployments use |
| B8 Evaluation and reproducibility | 8% release code; 11% test out of distribution; real-network share falling (Chapter 5) | shared testbeds and benchmarks [65, 163, 164, 166]; independent evaluations [145, 146] | tools exist; adoption low |
| B9 Training cost and lifecycle | millions of interactions; online exploration is risky; retraining, monitoring and drift rarely discussed | offline RL from logs [72, 130]; distributed training [66, 299, 329]; transfer [312]; federated or aggregated online learning [157, 330] | open, especially operations |

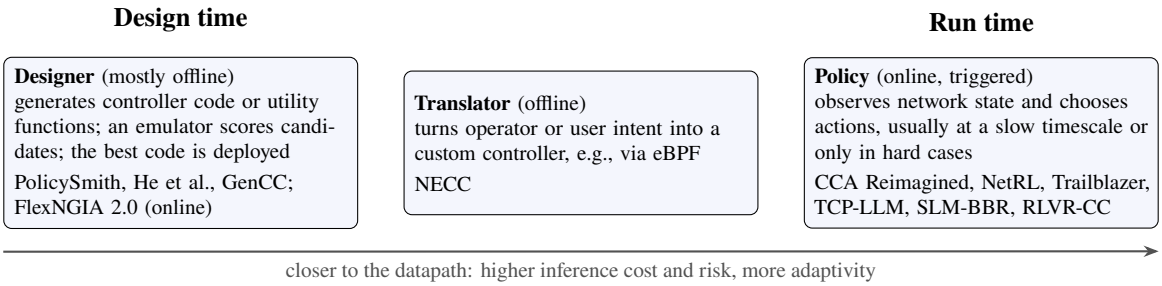



**Figure 6.1:** Roles that LLMs play in congestion control, ordered from design time to run time.

work is engineering for each platform: eBPF in the kernel, user-space QUIC stacks, NIC firmware, P4 switches.

**B5 Interpretability.** Distillation into trees or symbolic programs serves interpretability as well as efficiency [83, 84, 202, 328], and post-hoc explanation tools can expose why a controller acted [326, 327]. We found no study in which operators used such explanations, and no user study of them.

**B6 Adversarial robustness.** Learned controllers can be attacked through their inputs or by a competing flow designed to mislead them [309, 310]. Adversarial training improves robustness by roughly 20% [309]. CCLab finds learned controllers no less robust than classic ones in its threat model [305]. The threat models differ, however, and there is no shared benchmark.

**B7 Objectives.** Every RL design hard-codes a trade-off in its reward. Deployments sidestep the question by optimizing one service metric. Research designs condition on preferences [70] or constraints [134], and LLM-based work proposes to write utilities from natural-language requirements (Section 6.3).

**B8 Evaluation.** The tools for credible evaluation exist (Chapter 5), but their use is low and declining. This barrier is arguably the easiest to lower, because it requires changes in practice rather than new research.

**B9 Lifecycle.** Almost no study discusses what happens after deployment: when to retrain, how to detect drift, how to roll out a new model, or how to version models across a fleet. OnRL's per-user agents with aggregation and safeguards [157] are the most complete answer in the corpus. Federated training of bandwidth-weight agents [330] is a related idea.

## 6.3 LLM-Designed and LLM-Driven Controllers

Since 2024 a new line of work uses large language models in congestion control. Our Boolean query finds only part of it, because many of these papers do not use the phrase "reinforcement learning". We therefore searched for it separately (Appendix A). The included corpus contains one LLM-based study [332]. Here we review it together with the closely related works found by that supplementary search and organize them by the role the LLM plays (Fig. 6.1).

**LLMs as designers.** The most developed direction treats the LLM as a search operator over controller *code*, with an emulator as the fitness function. PolicySmith generates instance-specific heuristics, and for congestion control it produces safe policies that integrate directly into the Linux kernel [333]. He et al.

combine structured algorithm generation with an emulation pipeline covering a broad range of conditions and a statistical method that shortens evaluation. Their best algorithms improve on BBR by up to 27% in a production QUIC implementation [334]. GenCC asks LLMs to write the *utility functions* of PCC-style controllers for applications with conflicting objectives, and it studies how different guidance strategies affect the generated protocols on a realistic testbed [335, 336]. FlexNGIA 2.0 sketches an architecture in which LLM agents design, implement, deploy and evaluate protocols, including congestion control, at run time [337]. NADA applied the same idea to RL-based adaptive bitrate streaming, letting an LLM generate state representations and network architectures [338].

**LLMs as translators of intent.** NECC lets non-expert users model, implement and deploy customized congestion controllers by combining an LLM with the BPF interface [339]. It addresses the specification barrier (B7) directly: the objective is written in natural language and compiled into a controller.

**LLMs as policies.** Running an LLM in the control loop raises the overhead barrier (B4) to an extreme, and every design in this group works around it. CCA Reimagined first confines the LLM to congestion avoidance, studying when to invoke it, what to tell it and how to phrase its instructions, and then extends it to several phases. It reports up to 50% lower latency than traditional algorithms with under 0.3% throughput loss, and stresses that an effective *triggering* mechanism is essential [340]. NetRL fine-tunes an LLM policy with supervised learning and then with a group-relative policy-optimization variant (NetGRPO). It reports 24.1% higher network utility than Orca and generalization to unseen traces [332]. Trailblazer aligns compact LLMs to network control and invokes them only in complex conditions. In a large online A/B test of congestion control on Douyin, it outperformed a highly optimized industrial policy, with a projected reduction of about 3,145 hours of video stalls per day [341]. This is the only user-facing deployment of an LLM-based controller we found. Smaller studies adapt LLMs to TCP fairness and starvation over Wi-Fi [342], fine-tune small language models with LoRA to choose BBR pacing gains over Starlink [343], and train an LLM with RL from verifiable rewards to select rate actions [344, 345]. NetLLM provides a general framework for adapting LLMs to networking tasks [346].

**Assessment.** This line of work relates to the barriers above as follows.

1. **Hybrid structure.** Every LLM policy we found stays off the per-packet path. It is triggered only in hard cases, acts at a coarse timescale, or tunes a classic controller (BBR's gains). This is the same division of labor that Orca and Spine adopted for neural policies.
2. **Generated code.** A generated controller is source code that can be reviewed, tested, verified and versioned like any heuristic. That addresses interpretability (B5) and part of safety (B2) better than any neural policy, and it matches the interpretable structure favored in the deployments of Section 6.1. The risk moves to the search process: the generator is likely to overfit to the emulator's conditions unless held-out conditions and real-network tests are part of the loop.
3. **RL as the training method.** NetRL's NetGRPO and RLVR-CC fine-tune language models with RL, so the two lines are converging. In these systems the RL design effort moves from the policy network to the verifiable reward.
4. **Evidence.** Apart from Trailblazer's A/B test, these studies are evaluated in emulation, in simulation or on recorded traces, and He et al.'s gains are measured for a production QUIC code base under emulated conditions. Several rely on proprietary models, which limits reproducibility, and many are preprints from 2025–26.

## 6.4 Research Agenda

The evidence above points to the following research directions.

1. **Distribution-shift benchmarks.** The field needs a benchmark with explicit training and test distributions along named factors (bandwidth, RTT, buffer, loss, delay noise, capacity variation, competing traffic, wireless traces), in the spirit of ReCoCo's split for real-time media [164]. Results should be reported per factor, as in our case study, rather than as one average.
2. **Harm-based coexistence testing.** Every learned controller should be evaluated against CUBIC and BBR using harm metrics [21], and against non-congestive delay, which can starve delay-sensitive designs [331].
3. **Validated simulators and emulators for wireless and space.** Simulators should be calibrated against emulation and real traces, as Pantheon did for wired paths [65]. This matters most for satellite and IoT settings, where simulation-only evidence predominates and is known to miss stack effects [244].
4. **Runtime assurance as a design component.** Since most deployments rely on fallbacks, their design deserves study: when to switch, how quickly, and what guarantees result. Certificates in training [75] and monitors at run time [320] could be combined.
5. **Learning from logs at scale.** Offline RL from deployed controllers' logs [72, 166] removes the simulator from training. It needs shared datasets, off-policy evaluation, and pessimism against extrapolation [130].
6. **Datapath co-design.** Learned components should target eBPF, QUIC, NIC and P4 deployment from the start, through timescale separation, distillation or lookup, rather than retrofitting a neural network.
7. **LLMs at design time, with held-out evaluation.** The most promising LLM role is generating interpretable controllers. Its evaluation must guard against search overfitting through held-out conditions, real-network tests and released generated code.
8. **Lifecycle and operations.** Model updates, drift detection, rollbacks and fleet-wide versioning are unaddressed, although a controller in production needs all of them.

## 6.5 Threats to Validity

**Search.** We substituted OpenAlex for direct queries of IEEE Xplore, the ACM Digital Library and Scopus (Section 1.6.2). OpenAlex indexes their records, but it can miss records whose abstracts it lacks. Citation searching mitigated this and recovered several well-known studies, which suggests that some studies remain missing. The Boolean query requires the exact phrases of our protocol, so studies that describe RL without those words (e.g., "deep Q-network" with "TCP") can be missed unless citation searching finds them. LLM-based work is the clearest example, and we treat it separately in Section 6.3.

**Selection and coding.** Our protocol called for both authors to screen every record independently. In this version, screening and data extraction were carried out in a single pass with extensive tool support, including keyword pre-filtering, automatic mining of evidence snippets and an AI assistant, and they have not yet been duplicated independently. Every decision is recorded with its reason, and every coded value with its evidence source, so that a second reviewer can audit them. The replication package includes a blinded screening sheet and an agreement script for that purpose (Appendix C). A consistency re-check by the same coder of 25 randomly sampled design studies coded from abstracts (`review/data/audit_sample.csv`) changed no evaluation level, action or algorithm family. It flagged two borderline judgments: a trace-driven evaluation

coded as simulation, and a receiver-side placement inferred from the application. Such a re-check is not a substitute for independent coding. Until the agreement is reported, borderline inclusion decisions, especially at the boundaries with ABR video, routing and V2X channel-load control, should be read as single-reviewer judgments.

**Evidence depth.** Only 78 studies were coded from full texts, and 220 from abstracts and metadata. Items that abstracts rarely report (algorithm family, training environment, code availability) are therefore undercounted, which is why we report the full-text subset separately (Table 5.1). Results reported by the studies themselves were not re-validated.

**Time window.** The search was run on 31 October 2026, so the 2026 counts cover ten months. The LLM literature in particular is growing quickly, so this review is likely to miss recent work in it.

## 6.6 Conclusion

We surveyed 301 studies of reinforcement and imitation learning for congestion control, identified systematically from 997 records. Our answers to the three research questions follow.

**RQ1 (Design).** RL controllers observe short histories of throughput, delay and loss, act on the window or the rate once per monitor interval, and optimize a weighted throughput–delay–loss reward. The field has moved from clean-slate neural policies toward hybrids, in which learning tunes, selects or bounds classic control (35% of designs since 2023), and toward learning from logged data.

**RQ2 (Testing).** Most studies are trained and tested in simulation. Only 24% of studies whose evaluation level is known reach a real network, and the real-network share has fallen since 2021. Only 11% of design studies test outside their training distribution, and 8% release code. Our case study shows how a policy that is excellent in distribution can fail on ordinary delay noise, and how broader training trades in-distribution quality and training stability for coverage.

**RQ3 (Deployment).** The deployments that exist either keep a classic algorithm or safeguard in the loop or learn from the operator's own logs, are run by operators who own the endpoints, and optimize a narrow service metric. Overhead is largely solved. Safety relies on runtime fallbacks, fairness to the deployed base is rarely measured, and the lifecycle of a deployed model is unstudied. LLMs are a promising design-time tool for producing interpretable controllers, but their evaluation is as thin as that of early RL work.

The main obstacle to RL-based congestion control today is the gap between the conditions under which controllers are trained and tested and the networks on which they would run. Closing it depends more on better benchmarks and more realistic evaluation than on new architectures.

# A Search Strings and Sources

All searches were run on 31 October 2026 for publication years 2016–2026. The raw responses are stored in `review/data/raw/`.

**OpenAlex (executed).** Works endpoint, Boolean search over titles and abstracts, page paging, 200 records per page:

```
filter = title_and_abstract.search:("reinforcement learning" OR
         "imitation learning") AND ("congestion control" OR
         "congestion window" OR ECN),publication_year:2016-2026
```

**arXiv (executed).** Because the API's phrase operator under-retrieves, we retrieved on single terms and applied the protocol's exact phrases locally (hyphens treated as spaces; "ECN" matched as a whole word):

```
search_query = (ti:reinforcement OR abs:reinforcement OR ti:imitation
                OR abs:imitation) AND (ti:congestion OR abs:congestion
                OR ti:ECN OR abs:ECN)
                AND submittedDate:[201601010000 TO 202612312359]
```

**USENIX NSDI and ATC (executed).** The technical-sessions page of every NSDI (2016–2026) and USENIX ATC (2016–2025) was scraped for titles and abstracts, and the protocol query was applied locally.

**Subscription databases (for re-execution with institutional access).** These strings implement the same query in each database's syntax. We could not run them (Section 1.6.2).

```
IEEE Xplore (Command Search; Publication Year 2016-2026):
  (("Document Title":"reinforcement learning" OR
    "Abstract":"reinforcement learning" OR
    "Document Title":"imitation learning" OR
    "Abstract":"imitation learning") AND
   ("Document Title":"congestion control" OR
    "Abstract":"congestion control" OR
    "Document Title":"congestion window" OR
    "Abstract":"congestion window" OR
    "Document Title":ECN OR "Abstract":ECN))

ACM Digital Library (Advanced Search; Published 2016-2026):
  (Title:("reinforcement learning" OR "imitation learning") OR
   Abstract:("reinforcement learning" OR "imitation learning")) AND
  (Title:("congestion control" OR "congestion window" OR ECN) OR
   Abstract:("congestion control" OR "congestion window" OR ECN))

Scopus (Advanced Search):
  TITLE-ABS(("reinforcement learning" OR "imitation learning") AND
            ("congestion control" OR "congestion window" OR ecn))
  AND PUBYEAR > 2015 AND PUBYEAR < 2027
```

**Citation searching.** *Forward*: all 2016–2026 works citing at least one of ten seed reports and mentioning "congestion" in the title or abstract. The seeds were Aurora (arXiv 2018 and ICML 2019), Orca, Pantheon/Indigo, MVFST-RL, Sage, QTCP, Eagle, TCP-RL, and DRL-CC for MPTCP. *Backward*: the reference lists of sixteen recent included reports with long related-work sections. Sage, Orca and Astraea were among them, and the full list is stored with the raw responses. Records already identified were removed before screening.

**Supplementary search for LLM-based work (Section 6.3).** This scoped search informs the narrative of Section 6.3 only and did not enter the PRISMA flow:

```
OpenAlex (2022-2026, titles and abstracts):
  ("large language model" OR "large language models" OR LLM OR LLMs
   OR "language model" OR GPT) AND
  ("congestion control" OR "congestion window")
arXiv (2022-2026), then a local filter for "congestion control|window":
  (ti:LLM OR abs:LLM OR ti:LLMs OR abs:LLMs OR ti:"language model"
   OR abs:"language model" OR abs:"language models") AND
  (ti:congestion OR abs:congestion)
```

# B Additional Tables and Figures

**Table B.1:** The 49 studies whose evaluation reaches a real network, and what that network was. Production deployments and A/B tests are rare; most "real" evaluations use cloud or testbed hosts on Internet paths.

| Study | Year | Real-network evaluation |
|---|---|---|
| Pantheon (Indigo) [65] | 2018 | Pantheon nodes on real Internet paths (wired and cellular, several countries) |
| TCP-RL [125] | 2018 | production web servers of a top global search engine (>1 year) |
| Concerto [156] | 2019 | operational mobile video-telephony service |
| DRL-CCP [172] | 2019 | physical NDN test platform |
| RL BWE for RTC (Microsoft) [155] | 2019 | real Internet video calls |
| ARC [90] | 2020 | real testbed (training and test) |
| DeepCC (Abbasloo et al.) [192] | 2020 | real-world cellular experiments |
| OnRL [157] | 2020 | Alibaba Taobao-live (543 h, 151 users) |
| Orca [67] | 2020 | global Internet testbeds (cloud servers across continents) |
| ACC [273] | 2021 | production DCs (300 machines, storage service) |
| AUTO [235] | 2021 | real Internet paths (cloud) |
| Libra [122] | 2021 | live Internet paths |
| Loki [158] | 2021 | commercial real-time video system (101M sessions) |
| MOCC [70] | 2021 | real-world Internet experiments |
| RLCC [94] | 2021 | Pantheon nodes on AWS |
| BBR-FIT [193] | 2022 | real 5G/WiFi testbed |
| DeepCC (Zhang et al.) [133] | 2022 | inter-continental real-world paths |
| DuGu [132] | 2022 | real-world A/B tests in a short-video upload service |
| Genet [71] | 2022 | real-world network paths |
| HybridRTS [161] | 2022 | real-world WebRTC experiments |
| MP-OL [151] | 2022 | real networks |
| OpenNetLab [163] | 2022 | distributed real-network testbed nodes |
| RL-CC on NVIDIA NICs [282] | 2022 | live cluster with NVIDIA ConnectX NICs |
| Spine [73] | 2022 | real-world Internet experiments |
| ZiXia [347] | 2022 | global real-world paths |
| Dolce-RC [283] | 2023 | commodity SmartNIC testbed |
| Floo [126] | 2023 | production mobile web service |
| Leopard [234] | 2023 | real-world experiments |
| MARLIN [92] | 2023 | real network with background traffic (training and test) |
| MIQA / MIA [194] | 2023 | mmWave immersive testbed |
| Marten [115] | 2023 | production network (QUIC) |
| Sage [72] | 2023 | Internet paths |
| Astraea [74] | 2024 | inter-continental real-world paths (cloud) |
| Canopy [75] | 2024 | global testbed |
| MMSys'24 offline-RL BWE challenge [166] | 2024 | real-world RTC evaluation |
| Maggey [153] | 2024 | live experiments |
| SAC-MWL (QUIC) [106] | 2024 | worldwide real network endpoints |
| Anole [124] | 2025 | real-world wired and Wi-Fi networks |
| D2T* [280] | 2025 | P4-programmable switch |

| Study | Year | Real-network evaluation |
|---|---|---|
| EchoCC [321] | 2025 | live Internet |
| Flecto [198] | 2025 | 5G testbed (OpenAirInterface + USRP B210) |
| GO RL TL (semantic CC) [272] | 2025 | hardware tests |
| Jazz [223] | 2025 | dual-band (5/6 GHz) Wi-Fi testbed |
| Jury [142] | 2025 | real-world Internet paths |
| Mona adversarial policy [309] | 2025 | real-world network deployments |
| Mutant [76] | 2025 | inter-continental real-world paths |
| ICCP [325] | 2026 | real-world experiments |
| PPO CC on a 10 Gb/s WAN [307] | 2026 | real 10 Gbit/s high-BDP WAN path |
| TCCO [224] | 2026 | dual-band Wi-Fi testbed |

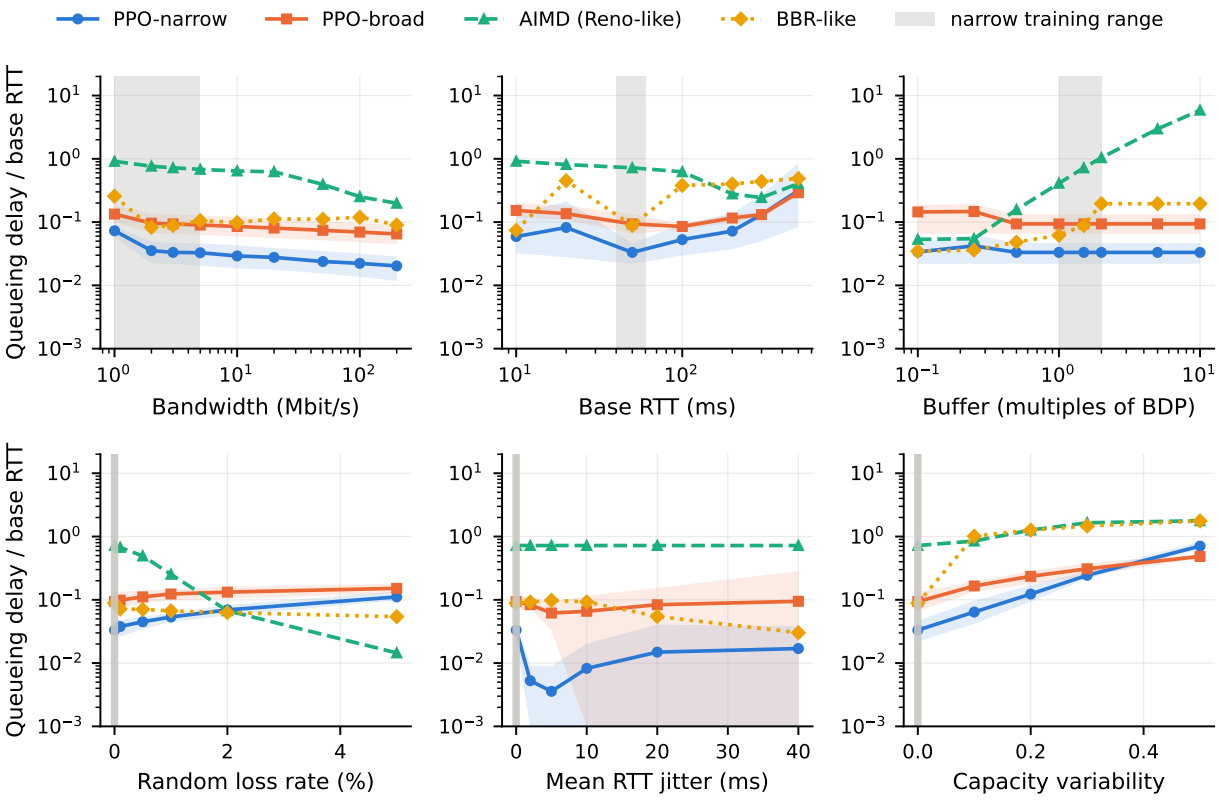



**Figure B.1:** Case study: mean queueing delay (in multiples of the base RTT, log scale) when one factor is varied around the reference link. Lines are means over the converged training seeds (as in Table 5.3) and shaded bands span them; the gray band or line marks the narrow training range. Both PPO policies keep queues far shorter than AIMD and usually shorter than BBR-like. Under capacity variability all controllers queue more, the classic ones most.

**Table B.2:** All 301 included studies, ordered by year of first report. Setting: WAN = Internet/wide-area, Gen. = unspecified/generic, Wless = wireless/cellular, Sat. = satellite/space, IoT = IoT/ad hoc/sensor, DC = data center. Level: highest evaluation environment reported (Sim/Emu/Real; NR = not reported; – = formal analysis only). Code: ✓ = artifact released. Linked reports of the same study are cited together.

| Study | Year | Set. | Action | Method | Level | Code |
|---|---|---|---|---|---|---|
| Q-learning TCP for memory-constrained IoT [250] | 2016 | IoT | cwnd | Q-learning with function approximation | Sim | |
| Smart-DTN-CC [265, 348] | 2016 | IoT | other | RL | Sim | |
| Kanerva-coding RL TCP [78] | 2017 | Gen. | cwnd | Q-learning + generalization-based Kanerva cod. . . | NR | |
| Actor-critic AQM [180] | 2018 | Gen. | AQM | actor-critic | Sim | |
| Aurora [17, 349] | 2018 | WAN | rate | PPO | Emu | ✓ |
| Iroko [296] | 2018 | DC | rate | PPO; REINFORCE; DDPG | Emu | ✓ |
| Muses [196, 350] | 2018 | Wless | cwnd | imitation-learned LSTM distilled into per-env. . . | NR | |
| Pantheon (Indigo) [65] | 2018 | WAN | cwnd | imitation learning from emulator oracles | Real | ✓ |
| QARC [154] | 2018 | WAN | bitrate | A3C + video-quality prediction network | Sim | |
| QTCP [77] | 2018 | Gen. | cwnd | Q-learning + generalized Kanerva coding | NR | |
| RL-TCP [79] | 2018 | Gen. | cwnd | Q-learning | Sim | |
| SDN-DC Q-learning/SARSA CC [294] | 2018 | DC | rate | Q-learning; SARSA | NR | |
| TCP-RL [125, 351] | 2018 | WAN | param., select | group-based RL + deep RL | Real | |
| eqCC [253] | 2018 | IoT | rate | Q-learning | Sim | |
| Concerto [156] | 2019 | WAN | bitrate | custom imitation learning on >1M logged sessi. . . | Real | |
| DRL fronthaul CC [352] | 2019 | Wless | rate | several DRL methods | Sim | |
| DRL initial window for 5G MEC [353] | 2019 | Wless | param. | DRL | Sim | |
| DRL-CC (MPTCP) [147] | 2019 | Gen. | cwnd | deterministic policy gradient actor-critic +. . . | Emu | |
| DRL-CCP [172] | 2019 | WAN | rate | DRL | Real | |
| Intelligent AQM with ECN [181] | 2019 | Gen. | AQM | Q-learning | Emu | |
| LGFCC [241] | 2019 | Sat. | other | RL + incomplete-information game | Sim | |
| MVFST-RL [66] | 2019 | WAN | cwnd | IMPALA with delayed actions | Emu | ✓ |
| RL BWE for RTC (Microsoft) [155] | 2019 | WAN | bitrate | PPO | Real | |
| RL loss-differentiating TCP [214] | 2019 | Wless | cwnd | RL | NR | |
| Rax [91] | 2019 | Gen. | cwnd | online deep RL with Partial Action Learning | NR | |
| Robustifying protocols with adversarial examples [308] | 2019 | Gen. | – | RL-generated adversarial network conditions | Sim | |
| SmartCC [230] | 2019 | Wless | cwnd | Q-learning + hierarchical tile coding | NR | |
| TCP-Drinc [93] | 2019 | Gen. | cwnd | DQN with CNN+LSTM | Sim | |

| Study | Year | Set. | Action | Method | Level | Code |
|---|---|---|---|---|---|---|
| Verily [85] | 2019 | Gen. | – | formal verification of DRL policies | – | |
| ARC [90] | 2020 | WAN | cwnd | asynchronous RL trained in real networks | Real | |
| DeepCC (Abbasloo et al.) [192] | 2020 | Wless | param. | DDPG plug-in capping the cwnd of existing TCP | Real | ✓ |
| Distributed RL in-network CC [354] | 2020 | Gen. | rate | CTDE with centralized critic | Sim | |
| Eagle [68] | 2020 | WAN | rate | DRL with LSTM guided by BBR as expert | Emu | |
| Hd-TCP [209] | 2020 | Wless | cwnd | DRL | Sim | |
| LFQ [189] | 2020 | Gen. | buffer | online DRL | Sim | ✓ |
| Metis [83] | 2020 | Gen. | – | decision-tree and hypergraph interpretation o. . . | Sim | ✓ |
| Multi-task DRL CC (SDN) [176] | 2020 | Gen. | rate | multi-task DRL | NR | |
| OnRL [157] | 2020 | Wless | bitrate | online RL: per-user agents, aggregated model. . . | Real | |
| Orca [67] | 2020 | WAN | cwnd | TD3 | Real | ✓ |
| Partner [148] | 2020 | Gen. | cwnd | distributed DRL | NR | |
| RLAQM [182] | 2020 | Gen. | AQM | Q-learning | Sim | |
| RTCP (HSR) [210] | 2020 | Wless | cwnd | DRL | Sim | |
| SDN actor-critic MPTCP for IoT [355] | 2020 | IoT | cwnd | actor-critic with fuzzy normalized NN | Sim | |
| TCP-PPCC [100] | 2020 | Gen. | cwnd | PPO | Sim | |
| ACC [273] | 2021 | DC | ECN thr. | multi-agent DRL per switch | Real | |
| AIMD-QTCP [80] | 2021 | Gen. | cwnd | Q-learning + AIMD window rule | Sim | |
| AQM meets DRL [356] | 2021 | Gen. | AQM | DRL | NR | |
| AUTO [235] | 2021 | Sat. | rate | multi-objective RL + preference-adaptation mo. . . | Real | |
| Arsenal [205] | 2021 | Wless | – | evaluation platform for AI-driven video trans. . . | Sim | |
| CRL-CC [134] | 2021 | WAN | rate | constrained RL | Emu | |
| DQL-MPTCP [149] | 2021 | Gen. | cwnd, split | deep Q-learning with policy gradients, LSTM. . . | Emu | |
| DRL-3R [197, 357] | 2021 | Wless | cwnd | DRL with reward redistribution | NR | |
| DRL-CC (deadline-aware) [136] | 2021 | Gen. | rate | DRL | NR | |
| DeepCC (He et al., MPTCP) [231] | 2021 | Wless | cwnd | multi-agent DRL with self-attention | NR | |
| HRCC [159] | 2021 | WAN | param. | RL agent sets a gain on the heuristic bandwid. . . | Sim | |
| HSR-CC (space-ground MPTCP) [211] | 2021 | Sat. | cwnd | DRL | NR | |
| LearningCC [358] | 2021 | Gen. | cwnd | multi-armed bandit over cwnd update rules | Sim | |
| Libra [122, 123] | 2021 | WAN | rate | classic and RL candidates compared in explora. . . | Real | |
| Loki [158] | 2021 | WAN | bitrate | RL model fused with a reverse-engineered rule. . . | Real | |
| MARL + transfer TCP CC [359] | 2021 | Gen. | cwnd | transfer-learning-assisted RL; coordinated MA. . . | NR | |

| Study | Year | Set. | Action | Method | Level | Code |
|---|---|---|---|---|---|---|
| MOCC [70, 360] | 2021 | WAN | rate | PPO with preference sub-network + transfer le. . . | Real | |
| MOCC (Xia et al.) [361] | 2021 | WAN | rate | multi-objective RL with preferences | NR | |
| MPLibra [152, 362] | 2021 | Gen. | select | classic and learned MPTCP decisions compared. . . | Sim | |
| Owl [69, 363] | 2021 | WAN | cwnd | DQN in a Linux kernel module | Emu | |
| Policy distillation (fronthaul) [201] | 2021 | Wless | rate | TD3 distilled into decision trees / C++ rules | Sim | ✓ |
| RL-PSD [233] | 2021 | Wless | cwnd | two-stage RL | NR | |
| RL-explore [112] | 2021 | Gen. | cwnd | RL + bandwidth-probing mechanism | NR | |
| RLCC [94] | 2021 | WAN | cwnd | DQN | Real | |
| Rainbow vs DQN for TCP CC [109] | 2021 | Gen. | cwnd | Rainbow and DQN extensions | Sim | |
| SDN Q-learning rate allocation [175] | 2021 | Gen. | rate | Q-learning | Emu | |
| TCP-DQN [95] | 2021 | WAN | cwnd | DQN | NR | |
| TCP-NeuRoc [364] | 2021 | Gen. | cwnd | DRL + online changepoint detection; cold-star. . . | Emu | |
| TCP-RL in an SDN data centre [289] | 2021 | DC | cwnd | RL | NR | |
| TD actor-critic TCP [215] | 2021 | Wless | cwnd | actor-critic TD learning | NR | |
| Teacher-Student robust DRL [81, 365] | 2021 | Gen. | rate | teacher advice + reward shaping + prioritized. . . | Sim | |
| Zwei [366] | 2021 | WAN | bitrate | self-play RL using the actual requirement as. . . | Sim | |
| whiRL 2.0 [86] | 2021 | Gen. | – | DNN verification + model checking of DRL poli. . . | – | |
| Adversarial attack on RL CC [310] | 2022 | Gen. | – | state-estimating adversarial perturbation att. . . | Sim | |
| BBR-FIT [193] | 2022 | Wless | param. | RL tunes BBR gain | Real | |
| BoB [160] | 2022 | WAN | bitrate | PPO + heuristic warm start | Emu | ✓ |
| CLCC [367] | 2022 | WAN | bitrate | RL with frame-level features | Sim | |
| Coefficient-free RL rate control [288] | 2022 | DC | rate | RL with coefficient-free objective | NR | |
| DQN TCP v2 fairness [368] | 2022 | Gen. | cwnd | DQN | Sim | |
| DQN-tuned AQM [183] | 2022 | Gen. | AQM | DQN | Sim | |
| DRL bidirectional-node WSN CC [369] | 2022 | IoT | rate | DRL | Sim | |
| DRL-CC ablation (cellular) [203] | 2022 | Wless | rate | ablation of DRL training choices | Sim | |
| DeepCC (Zhang et al.) [133] | 2022 | WAN | rate | DDPG-based multi-objective offline training +. . . | Real | |
| DuGu [132] | 2022 | WAN | rate | NN imitates an optimal solver in an omniscien. . . | Real | |
| Federated DRL bandwidth weights [330] | 2022 | WAN | param. | federated multi-agent DRL | NR | |
| GDMD [370] | 2022 | Gen. | rate | RL CC + block scheduler | NR | |
| Genet [71, 371] | 2022 | WAN | rate | curriculum learning promoting environments wh. . . | Real | ✓ |
| Glider [372] | 2022 | WAN | rate | DRL | Emu | |

| Study | Year | Set. | Action | Method | Level | Code |
|---|---|---|---|---|---|---|
| HybridRTS [161] | 2022 | WAN | bitrate | switching between rule-based and RL rate cont. . . | Real | |
| ICCDRL [212] | 2022 | Wless | cwnd | DRL | Sim | |
| IEACC [174] | 2022 | WAN | rate | DRL with edge-aided congestion degrees | Sim | |
| LiteFlow [322, 373] | 2022 | Gen. | rate | kernel fast path for NN inference + userspace. . . | Emu | |
| MACC (cross-layer) [190, 374] | 2022 | Gen. | cwnd, AQM | two cooperating DQN agents | Sim | |
| MP-OL [151] | 2022 | Gen. | rate | multi-armed bandit per subflow, switchable to. . . | Real | |
| NCC [287] | 2022 | DC | rate | RL | Emu | |
| Nuwa-RL [222, 375] | 2022 | Wless | param. | PPO-LSTM tunes receiver-side heuristic | NR | |
| OAC-TCPCC [242] | 2022 | Sat. | cwnd | optimistic actor-critic | Sim | |
| OpenNetLab [163] | 2022 | WAN | bitrate | platform for training/evaluating RL RTC CC | Real | ✓ |
| Pareto [139] | 2022 | WAN | rate | communication-free MARL, staged training + on. . . | NR | |
| Plume (lightweight DRL CC) [324] | 2022 | Gen. | rate | compressed DRL framework; reward re-parameter. . . | NR | |
| Q-learning adaptive TCP CC [376] | 2022 | Gen. | cwnd | Q-learning | NR | |
| RL local flow control (lossy networks) [258] | 2022 | IoT | rate, buffer | RL + portfolio-based buffer allocation | Sim | |
| RL-CC (Tessler et al.) [281] | 2022 | DC | rate | policy gradient with analytic reward derivati. . . | Sim | |
| RL-CC on NVIDIA NICs [282] | 2022 | DC | rate | RL-CC policy distilled into decision trees | Real | ✓ |
| RLRBM [200] | 2022 | Wless | buffer | DRL | Sim | |
| RSCAT [279, 377] | 2022 | DC | param. | actor-critic RL + congestion classification | NR | |
| SAC-CC [104] | 2022 | Gen. | rate | SAC with GRU + self-attention | NR | |
| SDN-DC incast DRL [295] | 2022 | DC | rate | DRL | Sim | |
| Scenario-based modelling for DRL (Aurora) [82, 378] | 2022 | WAN | rate | PPO with scenario-based reward shaping | Sim | |
| Spine [73, 379] | 2022 | WAN | param. | hierarchical: light CC executor + coarse-grai. . . | Real | |
| Symbolic distillation [84] | 2022 | WAN | rate | Aurora PPO policy distilled into symbolic tre. . . | Emu | ✓ |
| TCP-CFCC [101] | 2022 | WAN | rate | PPO | Emu | |
| Transformer-BBR (power IoT) [267] | 2022 | IoT | cwnd | DRL with Transformer agent in BBR probing | Sim | |
| ZiXia [347] | 2022 | WAN | rate | DRL with adjusted ranking reward | Real | |
| A2C UAV bottleneck control [226] | 2023 | Wless | rate | A2C | Sim | |
| ABBR [114] | 2023 | WAN | rate | DRL with BBR domain knowledge | Emu | |
| ACC-RL (power DC) [290] | 2023 | DC | rate | deterministic policy gradient | Sim | |
| ACCT [121] | 2023 | Gen. | param. | actor-critic with multiple reward functions | Sim | |

| Study | Year | Set. | Action | Method | Level | Code |
|---|---|---|---|---|---|---|
| ACoCo [245] | 2023 | IoT | param. | RL adapts CoAP CC parameters | Sim | |
| CARLA [254] | 2023 | IoT | rate | learning automata | Sim | ✓ |
| DECC [285] | 2023 | DC | cwnd | DRL + ECN with decoupled cwnd adjustment | Sim | |
| DLCC [257] | 2023 | IoT | rate | DRL in a non-cooperative Markov game with Lag. . . | Sim | |
| DNN+DRL wireless CC [216] | 2023 | Wless | cwnd | DRL | NR | |
| DRL CC for SDN/NDN satellites [380] | 2023 | Sat. | NR | DRL | NR | |
| DRLFcc [217] | 2023 | Wless | cwnd | DRL combined with fast recovery | Sim | |
| DeepFC [381] | 2023 | IoT | param., cwnd | DRL predicts transmission-unit size coupled w. . . | NR | |
| Distributed RL training system for CC [329] | 2023 | Gen. | – | distributed sample generation + online analys. . . | NR | |
| Dolce-RC [283] | 2023 | DC | param. | multi-armed bandit over rate-increase/decreas. . . | Real | |
| Floo [126] | 2023 | WAN | select | RL-based CCA selection with smooth switching | Real | |
| Hierarchical MARL queue policies (maritime) [259] | 2023 | IoT | rate, other | hierarchical multi-agent RL | Sim | |
| Implementability of DRL CC (cellular) [382] | 2023 | Wless | NR | DRL | NR | |
| Internet CC benchmarking [304] | 2023 | WAN | – | general benchmarks ranking Internet CCs incl. . . | Emu | |
| Interval configuration for DRL CC [107] | 2023 | WAN | rate | PPO with different MI / RTT estimators | Emu | |
| Inverse-RL CC [383] | 2023 | Gen. | NR | inverse RL with parallel training | NR | |
| IoUT CoAP copies [247] | 2023 | IoT | param. | RL at the sink sets per-node copy count | Sim | |
| LRCC [384] | 2023 | WAN | bitrate | attention-LSTM bandwidth memory + RL | Sim | |
| LUC (bandit-feedback game CC) [143] | 2023 | Gen. | rate | swap-regret-minimizing no-regret learning | Emu | |
| Leopard [234] | 2023 | Wless | cwnd | offline RL + online fine-tuning | Real | |
| MARL BBR parameter tuning (DC) [292] | 2023 | DC | param. | multi-agent RL tuning BBR parameters | Emu | |
| MARL SDN CC [178] | 2023 | Gen. | other | improved cooperative Q-learning | NR | |
| MARLIN [92, 385] | 2023 | WAN | cwnd | SAC, trained on a real network | Real | ✓ |
| MIQA / MIA [194, 386] | 2023 | Wless | rate | RL bandwidth predictor steering TCP/QUIC CC | Real | |
| Marten [115, 387] | 2023 | WAN | rate | DRL guided by BBR expert: entropy-based explo. . . | Real | |
| Multicast QUIC RL CC [168] | 2023 | WAN | rate | actor-critic with decaying epsilon-greedy | Sim | |
| PBQ-enhanced QUIC [116] | 2023 | WAN | cwnd | PPO sets cwnd, BBR sets pacing | Sim | |
| PPO cwnd strategy [103] | 2023 | Gen. | cwnd | PPO with advantage estimation | NR | |
| PeaceKeeper [120] | 2023 | Gen. | param. | RL adjusts LEDBAT++ delay target | Sim | |
| Plume (prioritized trace sampling) [311] | 2023 | Gen. | rate | trace clustering and prioritized sampling | Sim | ✓ |

| Study | Year | Set. | Action | Method | Level | Code |
|---|---|---|---|---|---|---|
| Q-learning/DQN MPTCP (deep-space video) [243] | 2023 | Sat. | cwnd | Q-learning and DQN | Sim | |
| RL CC + streaming FEC (NTN) [240] | 2023 | Sat. | rate | RL rate control with streaming FEC in quic-go | Emu | |
| RL protocol comparison (ns-3) [388] | 2023 | IoT | – | comparison of RL-based protocols incl. QTCP | Sim | |
| RLECN [389] | 2023 | Gen. | ECN thr. | RL via SDN controller | NR | |
| RMTC [232] | 2023 | Wless | cwnd, split | real-time multi-agent DRL + pacing | NR | |
| ReCoCo [164, 390] | 2023 | WAN | bitrate | PPO/DDPG/TD3/SAC trained on 162 traces, teste. . . | Sim | ✓ |
| STOP [137] | 2023 | Gen. | param., rate | curriculum-guided DRL | NR | |
| Sage [72] | 2023 | WAN | cwnd | offline RL learning from a pool of existing h. . . | Real | ✓ |
| TCP-PPO2 [102] | 2023 | Gen. | cwnd | PPO | Emu | |
| TCP-QNCC [96] | 2023 | Gen. | cwnd | DQN | NR | |
| TCP-RLLD [260] | 2023 | IoT | cwnd | RL-based loss differentiation steering TCP's. . . | NR | |
| TRCC [312] | 2023 | Gen. | cwnd | transfer RL | NR | |
| Verifying OOD generalization [87, 391] | 2023 | Gen. | – | agreement of independently trained DNNs check. . . | – | |
| Weighted-ensemble DRL CC [392] | 2023 | Gen. | NR | weighted ensemble DRL | NR | |
| iCoCoA [246] | 2023 | IoT | param. | DRL | NR | |
| A3C-tuned L4S AQM (FreeBSD) [187] | 2024 | Gen. | AQM | A3C tuning DualQ coupled AQM | Emu | ✓ |
| Astraea [74, 393, 394] | 2024 | WAN | rate | multi-agent DRL with a faithful multi-flow tr. . . | Real | |
| BBQ [277] | 2024 | DC | ECN thr. | RL ECN tuning bounded by per-port buffer | NR | |
| Canopy [75] | 2024 | WAN | cwnd | Orca trained with quantitative certificates f. . . | Real | ✓ |
| Closed-form CC via symbolic regression [202] | 2024 | Wless | rate | TD3 policy distilled by deep symbolic regress. . . | Sim | ✓ |
| CrystalBox [326] | 2024 | Gen. | – | post-hoc future-based explanations via decomp. . . | Sim | ✓ |
| DDPG-MPCC [150] | 2024 | Gen. | rate | DDPG + online convex optimization | Emu | ✓ |
| DDQN power-IoT CC [268] | 2024 | IoT | NR | double DQN | NR | |
| DESiRED (P4 AQM) [188] | 2024 | Gen. | AQM | DQN with in-band network telemetry | Emu | ✓ |
| DQN UDP CC for smart grid [266] | 2024 | IoT | rate | DQN | Sim | |
| DRL CCA switching for Wi-Fi fairness [219] | 2024 | Wless | select | DQN switching among CUBIC/BBR/PCC | Sim | |
| DRL WebRTC uplink rate control (5G) [207] | 2024 | Wless | bitrate | online model-free DRL | Sim | |
| DRL deep-space DTN CC [395] | 2024 | Sat. | NR | DRL | NR | |
| DRL-CaCC (cache-aware WSN CC) [255, 396] | 2024 | IoT | cwnd | DRL | NR | |
| DRL-RED [184] | 2024 | Gen. | AQM | DQN tunes RED queue weight | Sim | |

| Study | Year | Set. | Action | Method | Level | Code |
|---|---|---|---|---|---|---|
| Digital-twin actor-critic QUIC CC [195, 397] | 2024 | Wless | param. | actor-critic with a digital-twin network model | Sim | |
| EMPRN [275] | 2024 | DC | ECN thr. | MARL with message-passing GNN + GRU | Sim | |
| Game-theoretic bandits (swap regret) [144] | 2024 | Gen. | rate | online-mirror-descent multi-agent bandit | Emu | |
| GraphCC [274] | 2024 | DC | ECN thr. | MARL + GNN message passing between switch age. . . | Sim | |
| Gymir5G [206] | 2024 | Wless | bitrate | OMNeT++/Simu5G WebRTC platform with a DRL bri. . . | Sim | ✓ |
| Hierarchical DRL anti-jamming PHY + CC [227] | 2024 | Wless | multi | hierarchical DRL | Sim | |
| KerDqn [97] | 2024 | Gen. | cwnd | DQN with kernel CC + user-space learning fram. . . | NR | |
| LACC / HACC (hierarchical CCA selection) [293, 398] | 2024 | DC | select | RL-based CCA selection triggered only when ne. . . | Sim | |
| LAECN [276] | 2024 | DC | ECN thr. | lightweight hierarchical DRL ECN tuning | Sim | |
| MMSys'24 offline-RL BWE challenge [166] | 2024 | WAN | bitrate | offline RL on logged Microsoft Teams data wit. . . | Real | ✓ |
| Maggey [153] | 2024 | Gen. | cwnd | single-subflow RL model + non-blocking mode t. . . | Real | |
| Multi-context RL INT CC [229] | 2024 | Wless | cwnd | RL | NR | |
| NVC-aware RL CC [399] | 2024 | WAN | bitrate | online RL exploiting neural-codec loss tolera. . . | Sim | |
| Network-assist MAB [400] | 2024 | Gen. | rate | network-assisted MAB with discrete and contin. . . | Sim | |
| Q-NDN [173] | 2024 | WAN | cwnd | Q-learning | Sim | |
| Q-learning CC (multi-band) [228] | 2024 | Wless | rate | Q-learning | NR | |
| RL CC for LEO (Inf. Sci.) [401] | 2024 | Sat. | NR | RL | NR | |
| RL-TCP vs QTCP comparison (ns-2) [402] | 2024 | Gen. | – | comparison of RL-TCP, QTCP and classic TCPs | Sim | |
| RL-TCP+ (FCT vs sharing) [403] | 2024 | Gen. | cwnd | modified RL-TCP | NR | |
| RayNet [299] | 2024 | Gen. | cwnd | OMNeT++ + Ray/RLlib platform | Sim | ✓ |
| SAC-MWL (QUIC) [106] | 2024 | WAN | cwnd | SAC with monitor-window limit | Real | |
| Systematic evaluation of RL CC [145] | 2024 | Gen. | – | reproducible large-scale evaluation of public. . . | Emu | ✓ |
| TCP-RLACC [218] | 2024 | Wless | cwnd | Q-learning selecting linear/polynomial/expone. . . | Sim | |
| Zygos [138] | 2024 | Gen. | NR | distribution-based reward correction + meta-g. . . | NR | |
| ACKurate-DQN [110] | 2025 | Gen. | cwnd | DQN with scale-invariant reward and throughpu. . . | Sim | |
| AERO [170, 404] | 2025 | WAN | rate | learning-based plug-and-play QUIC CC | Sim | |
| AIRL (blockchain SDN) [405] | 2025 | Gen. | NR | active-inference RL | NR | |

| Study | Year | Set. | Action | Method | Level | Code |
|---|---|---|---|---|---|---|
| ASCRL [105] | 2025 | Gen. | cwnd | DRL with client-server architecture | Sim | |
| Agua [327] | 2025 | Gen. | – | concept-based explanations for learned contro. . . | NR | ✓ |
| Anole [124] | 2025 | WAN | rate | hybrid classic + learning-based CC with selec. . . | Real | |
| Auto (MORL CC service) [135] | 2025 | Gen. | rate | multi-objective RL; policy translated into a. . . | NR | |
| CUBIC-Learn [119] | 2025 | Gen. | param. | Q-learning adjusts CUBIC cwnd thresholds | Sim | |
| Creo [236] | 2025 | Sat. | cwnd | DRL in connected phase + handover-aware cross. . . | Sim | |
| Cross-layer mmWave CC [199] | 2025 | Wless | cwnd | DRL environment evaluation + LOS/NLOS classif. . . | NR | |
| D2T* [280] | 2025 | DC | buffer | dual-threshold buffer sharing + DRL threshold. . . | Real | |
| DNCCQ-PPO (XQUIC) [169] | 2025 | WAN | NR | PPO | NR | |
| DQN TCP CC (ns3-gym) [98] | 2025 | Gen. | cwnd | DQN via ns3-gym | Sim | |
| DQN_CC multi-hop mobile wireless [264] | 2025 | IoT | rate | DQN | Sim | |
| DRL CC for HEO-LEO [237] | 2025 | Sat. | cwnd | DRL | Sim | |
| DRL CC for wireless (LNNS) [406] | 2025 | Wless | NR | DRL | NR | |
| DRL-ABS [407] | 2025 | IoT | buffer | Dueling DQN | Sim | |
| Debunking RL for RTC BWE [167] | 2025 | WAN | bitrate | two-stage regressor benchmarked against RL BWE | Emu | |
| Design principles for RL CC environments [108] | 2025 | Gen. | cwnd | comparison of state, action and 14 reward des. . . | Sim | |
| Dual-agent Dueling DQN (SDN) [177] | 2025 | Gen. | rate | Dueling DQN vs Rainbow, A2C, DDPG | Emu | |
| DuetCC [286] | 2025 | DC | rate | DRL sub-strategies executed through model map. . . | Sim | |
| EE-AOBF-MPTCP-DDPG [408] | 2025 | Gen. | param. | DDPG optimizes the MPTCP balancing factor | Sim | |
| EPPO (power emergency RTC) [208] | 2025 | Wless | bitrate | PPO with parallel sampling, GAE and entropy o. . . | NR | |
| EchoCC [321] | 2025 | WAN | rate | WordBook lookup replacing learned-CCA inferen. . . | Real | |
| Expert-driven jumpstart RL CC [409] | 2025 | Gen. | NR | jump-start RL from an expert policy | NR | |
| Flecto [198] | 2025 | Wless | cwnd | DQN-family agent in QUIC | Real | |
| GO RL TL (semantic CC) [272] | 2025 | IoT | other | RL trained offline on real traces | Real | |
| HTPCCS [128] | 2025 | Gen. | select | offline profiling + contextual Thompson-sampl. . . | NR | |
| Jazz [223] | 2025 | Wless | cwnd | Transformer DRL agent on an edge server, deco. . . | Real | |
| Jury [142] | 2025 | WAN | rate | DRL + post-processing fairness module | Real | |
| KONTROL [251] | 2025 | IoT | cwnd | DRL agents on a central CC server | Sim | |

| Study | Year | Set. | Action | Method | Level | Code |
|---|---|---|---|---|---|---|
| Learning-based vs human-derived CC study [146] | 2025 | Gen. | – | reproducible large-scale comparison with CUBI. . . | Emu | ✓ |
| MACC (MAPPO) [140] | 2025 | WAN | rate | MAPPO with centralized training, decentralize. . . | Emu | |
| MHA-DRL-CC (SAGIN) [238] | 2025 | Sat. | NR | heterogeneous multi-agent DRL + state predict. . . | Sim | |
| Meta-RL CC protocol [88] | 2025 | WAN | rate | meta-RL with latent task inference on an Auro. . . | Sim | |
| Meta-diffusion BWE policy [314] | 2025 | WAN | bitrate | offline DRL with diffusion meta-policy | Sim | |
| MetaCon [313] | 2025 | WAN | rate | meta-RL | NR | |
| Mona adversarial policy [309] | 2025 | Gen. | – | adversarial-policy attack via a stealthy comp. . . | Real | |
| Mutant [76, 410] | 2025 | WAN | select | online contextual bandit over a pool of exist. . . | Real | ✓ |
| NMS + Galaxy [204] | 2025 | Wless | cwnd | network-segmented model specialization + clea. . . | NR | |
| Pisces [141] | 2025 | WAN | rate | multi-agent meta-RL | NR | |
| ProCC [328] | 2025 | Gen. | cwnd | programmatic RL: MCTS over a DSL of if/else p. . . | Sim | |
| PulseQUIC [171] | 2025 | WAN | rate | DRL-guided adaptive pacing | NR | |
| Q-AIMD [411] | 2025 | Gen. | cwnd | Q-learning over AIMD actions | NR | |
| RDI [284] | 2025 | DC | rate | RL with online + offline learning | Sim | |
| RL-BNCC (power DCs) [291] | 2025 | DC | NR | DRL + Bayesian network | NR | |
| SAB-RL [412] | 2025 | IoT | buffer | lightweight RL allocating switch buffer | NR | |
| TCP-Int (mesh) [261] | 2025 | IoT | cwnd | RL via ns3-gym | Sim | |
| TransAL-CC [269] | 2025 | IoT | cwnd, split | Transformer + asynchronous actor-learner RL | Emu | |
| WHCC (home Wi-Fi) [220] | 2025 | Wless | rate | TD3 regulates torrent bandwidth at the router | NR | |
| AHMA [129] | 2026 | Gen. | select | Bayesian Transformer loss classifier + meta-e. . . | Sim | |
| ALCHEMY [127] | 2026 | WAN | select | offline-trained DRL selects expert CCAs with. . . | Emu | |
| AutoNetFlow [413] | 2026 | Gen. | AQM | RL agent optimizing packet-handling strategies | Sim | |
| C3 [179] | 2026 | Gen. | cwnd | centralized RL agent periodically guiding end. . . | Sim | |
| CCLab [305] | 2026 | Gen. | – | RL adversary perturbing inputs or environment. . . | Emu | |
| Cross-layer dual-agent RL (P4) [278] | 2026 | DC | rate, ECN thr. | host-rate and switch-ECN agents with a centra. . . | Emu | |
| DARA [213] | 2026 | Wless | split | DQN + Transformer path-state forecasting | Emu | |
| DDPG MQTT-broker CC [249] | 2026 | IoT | rate | DDPG | Sim | |
| DRL + least-squares congestion minimization [414] | 2026 | Gen. | NR | DRL with least-squares congestion term | NR | |

| Study | Year | Set. | Action | Method | Level | Code |
|---|---|---|---|---|---|---|
| DRL-BLUE [185] | 2026 | Gen. | AQM | DDPG tunes BLUE queue weight | Emu | |
| DTCC [131] | 2026 | Gen. | NR | decision transformer | NR | |
| Delay-based RL jitter-aware CC [415] | 2026 | Gen. | NR | delay-based RL | NR | |
| Digital-twin robust RL (substation GOOSE flooding) [270] | 2026 | IoT | rate | digital-twin-assisted robust RL | NR | |
| DynA [162] | 2026 | WAN | param. | dual-policy continuous-action RL with curricu. . . | NR | |
| Emulation vs simulation (LEO CC) [244] | 2026 | Sat. | – | same CCs in OMNeT++/INET and Mininet | Emu | ✓ |
| Explainable cache-aware DRL CC (robotic sensor nets) [256] | 2026 | IoT | cwnd | explainable DRL | NR | |
| Fair-Aurora [89] | 2026 | WAN | rate | PPO with reward shaping / observation augment. . . | Sim | |
| Fuzzy + RL AQM [416] | 2026 | Gen. | AQM | fuzzy logic + RL | NR | |
| GenRL-CC [165] | 2026 | WAN | bitrate | TD3/SAC on ReCoCo with GAN-synthesized traini. . . | Sim | |
| HAN [117] | 2026 | Gen. | rate | DRL with uncertainty-triggered fallback to cl. . . | NR | |
| ICCP [325] | 2026 | Gen. | – | decoupled runtime hosting heuristic and RL CC. . . | Real | |
| Learn-to-Probe [113] | 2026 | WAN | rate | DRL with intrinsic probing reward + Bayesian. . . | NR | |
| MADRL MPTCP for IIoT [271] | 2026 | IoT | cwnd | multi-agent DRL | NR | |
| MARL CC for wireless ad hoc networks [263] | 2026 | IoT | rate | multi-agent RL + traffic prioritization | Sim | |
| Multi-agent DQN CC (high-speed) [417] | 2026 | Gen. | rate | multi-agent deep Q-learning | NR | |
| NetRL [332] | 2026 | WAN | rate | LLM policy: SFT + NetGRPO | NR | ✓ |
| POCC [130] | 2026 | WAN | NR | pessimistic offline RL with critic ensembles. . . | Sim | |
| PPO CC on a 10 Gb/s WAN [307] | 2026 | WAN | rate | Aurora-style PPO via a user-space PCC shim | Real | |
| PPO CoAP-QUIC cross-layer CC [248] | 2026 | IoT | cwnd, param. | PPO with traffic-class-aware reward weights | Sim | |
| PPO TCP CC for FANETs [225] | 2026 | Wless | cwnd | PPO | Sim | |
| PPO-tuned CUBIC (beta, C) [118] | 2026 | Gen. | param. | PPO within safe parameter ranges | Sim | |
| PRO-SAC [418] | 2026 | WAN | bitrate | SAC with lower-bounded entropy temperature | Emu | |
| Pacemaker-CC [111] | 2026 | Gen. | cwnd | DQN with ceiling-normalized reward | Sim | ✓ |
| Q-learning queue-discipline selection [186] | 2026 | Gen. | AQM | Q-learning selects FIFO/WFQ/RED/CoDel | Sim | |
| QoS-aware hierarchical CC (SAGSIN) [239] | 2026 | Sat. | NR | hierarchical intelligent CC | NR | |
| RL-HCC (6G) [419] | 2026 | Wless | cwnd, rate | RL-assisted hybrid CC | Sim | |
| RL-HKD [99] | 2026 | Gen. | cwnd | DQN | Sim | |

| Study | Year | Set. | Action | Method | Level | Code |
|---|---|---|---|---|---|---|
| SAC-BBR [221] | 2026 | Wless | param. | DRL regulator of BBR pacing/cwnd gains | NR | |
| SCC-RT [420] | 2026 | Gen. | bitrate, buffer | hierarchical DRL for semantic encoding and bu. . . | NR | |
| SymDistill-CC [252] | 2026 | IoT | rate | PPO distilled into symbolic decision trees | Sim | |
| Syntra [421] | 2026 | WAN | multi | minimax-synthesized strategies distilled by i. . . | NR | |
| TCCO [224] | 2026 | Wless | cwnd | Transformer DRL agent in an external decision. . . | Real | |
| TCP-RLA [262] | 2026 | IoT | cwnd | DQN with rule-assisted discrete actions | Sim | |
| TT&C microburst ECN (space) [422] | 2026 | Sat. | ECN thr. | deep-learning threshold optimizer with RL aux. . . | Sim | |
| diffRL symbolic properties [319] | 2026 | Gen. | – | symbolic property verification of DRL agents | – | |

# C Replication Package

The replication package contains everything needed to regenerate every number, table and figure in this survey. Its `README.md` gives the exact commands.

**`review/scripts/`** The pipeline. Search (`search_openalex.py`, `search_arxiv.py`, `search_usenix.py`, `search_snowball.py`, `search_backward.py`, `search_llm.py`), deduplication (`merge_dedupe.py`), retrieval and evidence mining (`fulltext_*.py`, `evidence.py`), study linking (`studies.py`), corpus and bibliography (`build_corpus.py`, `assemble_bib.py`), and statistics, figures and tables (`make_numbers.py`, `make_tables.py`).

**`review/data/`** Records, every screening decision with its reason (`screening/`), the retrieval log, the coded extraction files (`extraction/coded_b0*.psv`, one line per report with the evidence source), the study table, and the generated statistics.

**`review/data/screening/second_screener/`** A blinded sample of title/abstract records for an independent second screener, and `review/scripts/agreement.py`, which reports Cohen's $\kappa$ [423] against the original decisions.

**`case_study/`** Environment (`ccenv.py`), training (`train.py`), evaluation (`evaluate.py`), figures and tables (`plot_case_study.py`), and the eight trained models.

**Case-study configuration.** Table C.1 lists the settings used in Section 5.5.

**Table C.1:** Case-study configuration.

| Item | Setting |
|---|---|
| Environment | fluid single-bottleneck drop-tail queue; 20 ms monitor intervals; feedback delayed by $\lceil \mathrm{RTT}/20\,\mathrm{ms} \rceil$ intervals; per-packet random loss; exponential RTT jitter; capacity as a bounded multiplicative random walk |
| Observation | latency gradient, latency ratio, sending ratio for each of the last 10 intervals (30 values) |
| Action | $a \in [-1, 1]$; rate $\times(1 + 0.05a)$ if $a \geq 0$, else rate $/(1 - 0.05a)$ |
| Reward | utilization − queueing delay/base RTT $-3\times$ loss rate |
| Algorithm | PPO (Stable-Baselines3): 8 parallel environments, 512 steps each per update, minibatch 256, 10 epochs, learning rate $3 \times 10^{-4}$, $\gamma = 0.99$, GAE $\lambda = 0.95$, clip 0.2, entropy coefficient 0.005, MLP $64 \times 64$, reward normalization |
| Training | 5 million steps per run, 4 seeds per distribution, episodes of 400 intervals (8 s), about 9 minutes per run on one CPU core |
| Evaluation | 10 s flows including ramp-up, 5 flows per condition, deterministic policy; reference link 3 Mbit/s, 50 ms, 1.5 BDP; one factor varied at a time |
| Baselines | Reno-like AIMD (window-based, ACK-clocked, slow start, halving at most once per RTT); BBR-like (BBRv1 state machine, gain cycling, in-flight cap of 2×BDP) |

# D PRISMA 2020 Checklist

Table D.1 maps the PRISMA 2020 items [18] to the place where this survey reports them. Items that assume a quantitative meta-analysis (effect measures, certainty of pooled estimates) are marked as not applicable, because the synthesis is narrative and tabular (Section 1.6.6).

**Table D.1:** PRISMA 2020 checklist.

| # | Item | Where reported |
|---|---|---|
| 1 | Title | Title page: identified as a systematic survey |
| 2 | Abstract | Abstract |
| 3 | Rationale | Sections 1.1 and 1.7 |
| 4 | Objectives | Section 1.2 (**RQ1**–**RQ3**) |
| 5 | Eligibility criteria | Section 1.6.1, Table 1.2 |
| 6 | Information sources | Section 1.6.2; search date 31 October 2026 |
| 7 | Search strategy | Appendix A (executed strings and strings for the subscription databases) |
| 8 | Selection process | Section 1.6.3; decisions in `review/data/screening/` |
| 9 | Data collection process | Section 1.6.3; evidence and coding files in `review/data/` |
| 10a–b | Data items | Section 1.6.4 |
| 11 | Study risk-of-bias assessment | Section 1.6.5 (evidence-practice appraisal) |
| 12 | Effect measures | Not applicable: no effect sizes are pooled (Section 1.6.6) |
| 13a–f | Synthesis methods | Section 1.6.6; tabulation by setting, period and coded item (Section 1.8) |
| 14 | Reporting-bias assessment | Section 6.5 (search coverage, abstract-only coding) |
| 15 | Certainty assessment | Evidence level of every study (Section 5.2) and practice scores (Section 5.3) |
| 16a | Study selection | Figure 1.2 |
| 16b | Excluded studies | Reason codes for every excluded record in `review/data/screening/` and `extraction.csv` |
| 17 | Study characteristics | Table B.2; Section 1.8 |
| 18 | Risk of bias in studies | Section 5.3 (practice scores), Table 5.1 |
| 19 | Results of individual studies | Tables B.1 and B.2; Chapters 2 to 4 |
| 20a–d | Results of syntheses | Section 1.8; Chapters 2 to 6 |
| 21 | Reporting biases | Section 6.5 |
| 22 | Certainty of evidence | Sections 5.2 to 5.4 |
| 23a | Interpretation | Chapter 6, Section 6.6 |
| 23b | Limitations of the evidence | Sections 5.2 and 5.3 |
| 23c | Limitations of the review | Section 6.5 |
| 23d | Implications | Sections 5.6 and 6.4 |
| 24a–c | Registration and protocol | Not registered; protocol is the course proposal; deviations in Section 1.6.2 |
| 25 | Support | None (Section 1.6.6, final paragraph) |
| 26 | Competing interests | None declared (Section 1.6.6, final paragraph) |
| 27 | Availability of data, code and materials | Appendix C; replication package |

# Bibliography


[1] V. Jacobson, "Congestion avoidance and control," in *Symposium proceedings on Communications architectures and protocols*. ACM, Aug. 1988, pp. 314–329. [Online]. Available: http://dx.doi.org/10.1145/52324.52356

[2] D.-M. Chiu and R. Jain, "Analysis of the increase and decrease algorithms for congestion avoidance in computer networks," *Computer Networks and ISDN Systems*, vol. 17, no. 1, pp. 1–14, 1989. [Online]. Available: http://dx.doi.org/10.1016/0169-7552(89)90019-6

[3] L. Brakmo and L. Peterson, "TCP Vegas: end to end congestion avoidance on a global Internet," *IEEE Journal on Selected Areas in Communications*, vol. 13, no. 8, pp. 1465–1480, 1995. [Online]. Available: http://dx.doi.org/10.1109/49.464716

[4] S. Ha, I. Rhee, and L. Xu, "CUBIC: a new TCP-friendly high-speed TCP variant," *ACM SIGOPS Operating Systems Review*, vol. 42, no. 5, pp. 64–74, 2008. [Online]. Available: http://dx.doi.org/10.1145/1400097.1400105

[5] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, "BBR: Congestion-Based Congestion Control: Measuring bottleneck bandwidth and round-trip propagation time," *Queue*, vol. 14, no. 5, pp. 20–53, Oct. 2016. [Online]. Available: http://dx.doi.org/10.1145/3012426.3022184

[6] V. Arun and H. Balakrishnan, "Copa: Practical Delay-Based Congestion Control for the Internet," in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2018, pp. 329–342.

[7] K. Winstein, A. Sivaraman, and H. Balakrishnan, "Stochastic Forecasts Achieve High Throughput and Low Delay over Cellular Networks," in *10th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2013, pp. 459–471.

[8] Y. Zaki, T. Pötsch, J. Chen, L. Subramanian, and C. Görg, "Adaptive Congestion Control for Unpredictable Cellular Networks," in *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*. ACM, Aug. 2015, pp. 509–522. [Online]. Available: http://dx.doi.org/10.1145/2785956.2787498

[9] M. Alizadeh, A. Greenberg, D. A. Maltz, J. Padhye, P. Patel, B. Prabhakar, S. Sengupta, and M. Sridharan, "Data center TCP (DCTCP)," in *Proceedings of the ACM SIGCOMM 2010 conference*. ACM, Aug. 2010, pp. 63–74. [Online]. Available: http://dx.doi.org/10.1145/1851182.1851192

[10] Y. Zhu, H. Eran, D. Firestone, C. Guo, M. Lipshteyn, Y. Liron, J. Padhye, S. Raindel, M. H. Yahia, and M. Zhang, "Congestion Control for Large-Scale RDMA Deployments," in *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*. ACM, Aug. 2015, pp. 523–536. [Online]. Available: http://dx.doi.org/10.1145/2785956.2787484

[11] R. Mittal, V. T. Lam, N. Dukkipati, E. Blem, H. Wassel, M. Ghobadi, A. Vahdat, Y. Wang, D. Wetherall, and D. Zats, "TIMELY: RTT-based Congestion Control for the Datacenter," in *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*. ACM, Aug. 2015, pp. 537–550. [Online]. Available: http://dx.doi.org/10.1145/2785956.2787510

[12] Y. Li, R. Miao, H. H. Liu, Y. Zhuang, F. Feng, L. Tang, Z. Cao, M. Zhang, F. Kelly, M. Alizadeh, and M. Yu, "HPCC: high precision congestion control," in *Proceedings of the ACM Special Interest Group on Data Communication*. ACM, Aug. 2019, pp. 44–58. [Online]. Available: http://dx.doi.org/10.1145/3341302.3342085

[13] G. Kumar, N. Dukkipati, K. Jang, H. M. G. Wassel, X. Wu, B. Montazeri, Y. Wang, K. Springborn, C. Alfeld, M. Ryan, D. Wetherall, and A. Vahdat, "Swift: Delay is Simple and Effective for Congestion Control in the Datacenter," in *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*. ACM, 2020, pp. 514–528. [Online]. Available: http://dx.doi.org/10.1145/3387514.3406591

[14] K. Winstein and H. Balakrishnan, "TCP ex machina: computer-generated congestion control," in *Proceedings of the ACM SIGCOMM 2013 conference on SIGCOMM*. ACM, Aug. 2013, pp. 123–134. [Online]. Available: http://dx.doi.org/10.1145/2486001.2486020

[15] M. Dong, Q. Li, D. Zarchy, P. B. Godfrey, and M. Schapira, "PCC: Re-architecting Congestion Control for Consistent High Performance," in *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2015, pp. 395–408.

[16] M. Dong, T. Meng, D. Zarchy, E. Arslan, Y. Gilad, B. Godfrey, and M. Schapira, "PCC Vivace: Online-Learning

Congestion Control," in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2018, pp. 343–356.

[17] N. Jay, N. H. Rotman, P. B. Godfrey, M. Schapira, and A. Tamar, "A Deep Reinforcement Learning Perspective on Internet Congestion Control," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, vol. 97, 2019, pp. 3050–3059.

[18] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, L. Shamseer, J. M. Tetzlaff, E. A. Akl, S. E. Brennan, R. Chou, J. Glanville, J. M. Grimshaw, A. Hróbjartsson, M. M. Lalu, T. Li, E. W. Loder, E. Mayo-Wilson, S. McDonald, L. A. McGuinness, L. A. Stewart, J. Thomas, A. C. Tricco, V. A. Welch, P. Whiting, and D. Moher, "The PRISMA 2020 statement: an updated guideline for reporting systematic reviews," *BMJ*, p. n71, Mar. 2021. [Online]. Available: http://dx.doi.org/10.1136/bmj.n71

[19] R. Jain, D.-M. Chiu, and W. R. Hawe, "A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems," Digital Equipment Corporation, Tech. Rep. DEC-TR-301, 1984.

[20] F. P. Kelly, A. K. Maulloo, and D. K. H. Tan, "Rate control for communication networks: shadow prices, proportional fairness and stability," *Journal of the Operational Research Society*, vol. 49, no. 3, pp. 237–252, Mar. 1998. [Online]. Available: http://dx.doi.org/10.1057/palgrave.jors.2600523

[21] R. Ware, M. K. Mukerjee, S. Seshan, and J. Sherry, "Beyond Jain's Fairness Index: Setting the Bar For The Deployment of Congestion Control Algorithms," in *Proceedings of the 18th ACM Workshop on Hot Topics in Networks*. ACM, Nov. 2019, pp. 17–24. [Online]. Available: http://dx.doi.org/10.1145/3365609.3365855

[22] K. Ramakrishnan, S. Floyd, and D. Black, "The Addition of Explicit Congestion Notification (ECN) to IP," RFC 3168, IETF, 2001.

[23] M. Allman, V. Paxson, and E. Blanton, "TCP Congestion Control," RFC 5681, IETF, 2009.

[24] M. Mathis, J. Semke, J. Mahdavi, and T. Ott, "The macroscopic behavior of the TCP congestion avoidance algorithm," *ACM SIGCOMM Computer Communication Review*, vol. 27, no. 3, pp. 67–82, 1997. [Online]. Available: http://dx.doi.org/10.1145/263932.264023

[25] L. Xu, S. Ha, I. Rhee, V. Goel, and L. Eggert, "CUBIC for Fast and Long-Distance Networks," RFC 9438, IETF, 2023.

[26] R. Ware, M. K. Mukerjee, S. Seshan, and J. Sherry, "Modeling BBR's Interactions with Loss-Based Congestion Control," in *Proceedings of the Internet Measurement Conference*. ACM, Oct. 2019, pp. 137–143. [Online]. Available: http://dx.doi.org/10.1145/3355369.3355604

[27] A. Sivaraman, K. Winstein, P. Thaker, and H. Balakrishnan, "An experimental study of the learnability of congestion control," in *Proceedings of the 2014 ACM conference on SIGCOMM*. ACM, Aug. 2014, pp. 479–490. [Online]. Available: http://dx.doi.org/10.1145/2619239.2626324

[28] S. Floyd and V. Jacobson, "Random early detection gateways for congestion avoidance," *IEEE/ACM Transactions on Networking*, vol. 1, no. 4, pp. 397–413, 1993. [Online]. Available: http://dx.doi.org/10.1109/90.251892

[29] K. Nichols and V. Jacobson, "Controlling queue delay," *Communications of the ACM*, vol. 55, no. 7, pp. 42–50, 2012. [Online]. Available: http://dx.doi.org/10.1145/2209249.2209264

[30] R. Pan, P. Natarajan, F. Baker, and G. White, "Proportional Integral Controller Enhanced (PIE): A Lightweight Control Scheme to Address the Bufferbloat Problem," RFC 8033, IETF, 2017.

[31] B. Briscoe, K. D. Schepper, M. Bagnulo, and G. White, "Low Latency, Low Loss, and Scalable Throughput (L4S) Internet Service: Architecture," RFC 9330, IETF, 2023.

[32] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[33] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015. [Online]. Available: http://dx.doi.org/10.1038/nature14236

[34] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017.

[35] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, "Trust Region Policy Optimization," in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, vol. 37, 2015, pp. 1889–1897.

[36] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous

Methods for Deep Reinforcement Learning," in *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, vol. 48, 2016, pp. 1928–1937.

[37] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous Control with Deep Reinforcement Learning," in *International Conference on Learning Representations (ICLR)*, 2016.

[38] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing Function Approximation Error in Actor-Critic Methods," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, vol. 80, 2018, pp. 1587–1596.

[39] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, vol. 80, 2018, pp. 1861–1870.

[40] L. Espeholt, H. Soyer, R. Munos, K. Simonyan, V. Mnih, T. Ward, Y. Doron, V. Firoiu, T. Harley, I. Dunning, S. Legg, and K. Kavukcuoglu, "IMPALA: Scalable Distributed Deep-RL with Importance Weighted Actor-Learner Architectures," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, vol. 80, 2018, pp. 1407–1416.

[41] A. Kumar, A. Zhou, G. Tucker, and S. Levine, "Conservative Q-Learning for Offline Reinforcement Learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[42] Z. Wang, A. Novikov, K. Zolna, J. T. Springenberg, S. Reed, B. Shahriari, N. Siegel, J. Merel, C. Gulcehre, N. Heess, and N. de Freitas, "Critic Regularized Regression," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[43] L. Chen, K. Lu, A. Rajeswaran, K. Lee, A. Grover, M. Laskin, P. Abbeel, A. Srinivas, and I. Mordatch, "Decision Transformer: Reinforcement Learning via Sequence Modeling," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021.

[44] S. Ross, G. J. Gordon, and J. A. Bagnell, "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning," in *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, vol. 15, 2011, pp. 627–635.

[45] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[46] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, and Y. Wu, "The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 35, 2022.

[47] B. Kitchenham and S. Charters, "Guidelines for Performing Systematic Literature Reviews in Software Engineering," Keele University and Durham University, Tech. Rep. EBSE-2007-01, 2007.

[48] C. Wohlin, "Guidelines for snowballing in systematic literature studies and a replication in software engineering," in *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering*. ACM, May 2014, pp. 1–10. [Online]. Available: http://dx.doi.org/10.1145/2601248.2601268

[49] R. Boutaba, M. A. Salahuddin, N. Limam, S. Ayoubi, N. Shahriar, F. Estrada-Solano, and O. M. Caicedo, "A comprehensive survey on machine learning for networking: evolution, applications and research opportunities," *Journal of Internet Services and Applications*, vol. 9, no. 1, 2018. [Online]. Available: http://dx.doi.org/10.1186/s13174-018-0087-2

[50] M. Wang, Y. Cui, X. Wang, S. Xiao, and J. Jiang, "Machine Learning for Networking: Workflow, Advances and Opportunities," *IEEE Network*, vol. 32, no. 2, pp. 92–99, Mar. 2018. [Online]. Available: http://dx.doi.org/10.1109/MNET.2017.1700200

[51] N. C. Luong, D. T. Hoang, S. Gong, D. Niyato, P. Wang, Y.-C. Liang, and D. I. Kim, "Applications of Deep Reinforcement Learning in Communications and Networking: A Survey," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 4, pp. 3133–3174, 2019. [Online]. Available: http://dx.doi.org/10.1109/COMST.2019.2916583

[52] F. Tang, B. Mao, Y. Kawamoto, and N. Kato, "Survey on Machine Learning for Intelligent End-to-End Communication Toward 6G: From Network Access, Routing to Traffic Control and Streaming Adaption," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 3, pp. 1578–1598, 2021.

[53] L. Zhang, Y. Cui, M. Wang, Z. Yang, and Y. Jiang, "Machine Learning for Internet Congestion Control: Techniques and Challenges," *IEEE Internet Computing*, vol. 23, no. 5, pp. 59–64, 2019.

[54] T. Zhang and S. Mao, "Machine Learning for End-to-End Congestion Control," *IEEE Communications Magazine*, vol. 58, no. 6, pp. 52–57, 2020.

[55] W. Wei, H. Gu, and B. Li, "Congestion Control: A Renaissance with Machine Learning," *IEEE Network*, vol. 35, no. 4, pp. 262–269, 2021.

[56] H. Jiang, Q. Li, Y. Jiang, G. Shen, R. Sinnott, C. Tian, and M. Xu, "When machine learning meets congestion control: A survey and comparison," *Computer Networks*, vol. 192, p. 108033, 2021.

[57] H. Huang, X. Zhu, J. Bi, W. Cao, and X. Zhang, "Machine Learning for Broad-Sensed Internet Congestion Control and Avoidance: A Comprehensive Survey," *IEEE Access*, vol. 9, pp. 31 525–31 545, 2021.

[58] M. J. Lakshmi and M. BabuArrama, "Methods of Congestion Control in Wired Networks with Reinforcement Learning – A Review," in *2022 2nd International Conference on Innovative Practices in Technology and Management (ICIPTM)*. IEEE, Feb. 2022, pp. 128–136.

[59] M. Maliha, G. Habibi, and M. Atiquzzaman, "A Survey on Congestion Control and Scheduling for Multipath TCP: Machine Learning vs Classical Approaches," in *Proceedings of the 18th Conference on Computer Science and Intelligence Systems*, vol. 35. IEEE, 2023, pp. 49–61.

[60] M. P. Toopchinezhad and M. Ahmadi, "Machine learning approaches for active queue management: A survey, taxonomy, and future directions," *Computer Networks*, vol. 262, p. 111174, May 2025.

[61] M. E. Haque, "Fairness-Aware Congestion Control- From Heuristic Protocols to Learning-Based Frameworks," *Communications on Applied Nonlinear Analysis*, vol. 32, no. 6s, pp. 735–743, 2025.

[62] H. Li, Y. Liu, S. Zhang, and Z. Yang, "A Survey of Deep Reinforcement Learning for Network Congestion Control," in *2025 8th International Conference on Communication Engineering and Technology (ICCET)*. IEEE, May 2025, pp. 1–6.

[63] E. Abedini, M. Nickray, and A. Jalaly Bidgoly, "Dynamic Network Congestion Control: A Reinforcement Learning-based Perspective Review," *International Journal of Engineering*, vol. 40, no. 3, pp. 892–904, 2027.

[64] C. L. Vielhaus, C. v. Lengerke, V. Latzko, J. Rischke, M. Reisslein, and F. H. P. Fitzek, "Evaluating Transport Layer Congestion Control Algorithms: A Comprehensive Survey," *IEEE Communications Surveys & Tutorials*, vol. 28, pp. 278–340, 2026.

[65] F. Y. Yan, J. Ma, G. D. Hill, D. Raghavan, R. S. Wahby, P. Levis, and K. Winstein, "Pantheon: The Training Ground for Internet Congestion-Control Research," in *2018 USENIX Annual Technical Conference (USENIX ATC)*, 2018.

[66] V. Sivakumar, O. Delalleau, T. Rocktäschel, A. H. Miller, H. Küttler, N. Nardelli, M. Rabbat, J. Pineau, and S. Riedel, "MVFST-RL: An Asynchronous RL Framework for Congestion Control with Delayed Actions," arXiv:1910.04054, 2019.

[67] S. Abbasloo, C.-Y. Yen, and H. J. Chao, "Classic Meets Modern: a Pragmatic Learning-Based Congestion Control for the Internet," in *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*. ACM, 2020, pp. 632–647.

[68] S. Emara, B. Li, and Y. Chen, "Eagle: Refining Congestion Control by Learning from the Experts," in *IEEE INFOCOM 2020 - IEEE Conference on Computer Communications*. IEEE, 2020, pp. 676–685.

[69] A. Sacco, M. Flocco, F. Esposito, and G. Marchetto, "Owl: Congestion Control with Partially Invisible Networks via Reinforcement Learning," in *IEEE INFOCOM 2021 - IEEE Conference on Computer Communications*. IEEE, May 2021, pp. 1–10.

[70] Y. Ma, H. Tian, X. Liao, J. Zhang, W. Wang, K. Chen, and X. Jin, "Multi-objective congestion control," in *Proceedings of the Seventeenth European Conference on Computer Systems*. ACM, Mar. 2022, pp. 218–235.

[71] Z. Xia, Y. Zhou, F. Y. Yan, and J. Jiang, "Genet: automatic curriculum generation for learning adaptation in networking," in *Proceedings of the ACM SIGCOMM 2022 Conference*. ACM, Aug. 2022, pp. 397–413.

[72] C.-Y. Yen, S. Abbasloo, and H. J. Chao, "Computers Can Learn from the Heuristic Designs and Master Internet Congestion Control," in *Proceedings of the ACM SIGCOMM 2023 Conference*. ACM, 2023, pp. 255–274.

[73] H. Tian, X. Liao, C. Zeng, D. Sun, J. Zhang, and K. Chen, "Efficient DRL-Based Congestion Control With Ultra-Low Overhead," *IEEE/ACM Transactions on Networking*, vol. 32, no. 3, pp. 1888–1903, 2024.

[74] X. Liao, H. Tian, C. Zeng, X. Wan, and K. Chen, "Astraea: Towards Fair and Efficient Learning-based Congestion

Control," in *Proceedings of the Nineteenth European Conference on Computer Systems*. ACM, Apr. 2024, pp. 99–114.

[75] C. Yang, D. Saxena, R. Dwivedula, K. Mahajan, S. Chaudhuri, and A. Akella, "Canopy: Property-Driven Learning for Congestion Control," arXiv:2412.10915, 2024.

[76] L. Pappone, A. Sacco, and F. Esposito, "Mutant: Learning Congestion Control from Existing Protocols via Online Reinforcement Learning," in *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2025.

[77] W. Li, F. Zhou, K. R. Chowdhury, and W. Meleis, "QTCP: Adaptive Congestion Control with Reinforcement Learning," *IEEE Transactions on Network Science and Engineering*, vol. 6, no. 3, pp. 445–458, 2019.

[78] W. Li, F. Zhou, W. Meleis, and K. Chowdhury, "Dynamic Generalization Kanerva Coding in Reinforcement Learning for TCP Congestion Control Design," in *Proceedings of the 16th Conference on Autonomous Agents and MultiAgent Systems (AAMAS), extended abstract*. IFAAMAS, 2017, pp. 1598–1600.

[79] Y. Kong, H. Zang, and X. Ma, "Improving TCP Congestion Control with Machine Intelligence," in *Proceedings of the 2018 Workshop on Network Meets AI & ML - NetAI'18*. ACM Press, 2018, pp. 60–66.

[80] M. Yamazaki and M. Yamamoto, "Fairness Improvement of Congestion Control with Reinforcement Learning," *Journal of Information Processing*, vol. 29, no. 0, pp. 592–595, 2021.

[81] Y. Zheng, L. Lin, T. Zhang, H. Chen, Q. Duan, Y. Xu, and X. Wang, "Enabling Robust DRL-Driven Networking Systems via Teacher-Student Learning," *IEEE Journal on Selected Areas in Communications*, vol. 40, no. 1, pp. 376–392, Jan. 2022.

[82] R. Yerushalmi, G. Amir, A. Elyasaf, D. Harel, G. Katz, and A. Marron, "Enhancing Deep Reinforcement Learning with Scenario-Based Modeling," *SN Computer Science*, vol. 4, no. 2, Jan. 2023.

[83] Z. Meng, M. Wang, J. Bai, M. Xu, H. Mao, and H. Hu, "Interpreting Deep Learning-Based Networking Systems," in *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*. ACM, 2020, pp. 154–171.

[84] S. P. Sharan, W. Zheng, K.-F. Hsu, J. Xing, A. Chen, and Z. Wang, "Symbolic Distillation for Learned TCP Congestion Control," in *Advances in Neural Information Processing Systems 35*. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2022, pp. 10 684–10 695.

[85] Y. Kazak, C. Barrett, G. Katz, and M. Schapira, "Verifying Deep-RL-Driven Systems," in *Proceedings of the 2019 Workshop on Network Meets AI & ML - NetAI'19*. ACM Press, 2019, pp. 83–89.

[86] T. Eliyahu, Y. Kazak, G. Katz, and M. Schapira, "Verifying learning-augmented systems," in *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*. ACM, Aug. 2021, pp. 305–318.

[87] G. Amir, O. Maayan, T. Zelazny, G. Katz, and M. Schapira, "Verifying the Generalization of Deep Learning to Out-of-Distribution Domains," *Journal of Automated Reasoning*, vol. 68, no. 3, Aug. 2024.

[88] M. Mwaura, S. Kiambi, and H. Nganga, "Adaptive Congestion Control Protocol based on Meta-Reinforcement Learning for Data Communication Networks," *International Journal of Computer Applications*, vol. 186, no. 65, pp. 1–7, Feb. 2025.

[89] T. Mbrice and Y. Liu, "Fair-Aurora: Comparing Fairness Strategies for Reinforcement Learning-Based Congestion Control in Multi-Flow Environments," arXiv:2605.19909, 2026.

[90] L. Zhang, K. Zhu, J. Pan, H. Shi, Y. Jiang, and Y. Cui, "Reinforcement Learning Based Congestion Control in a Real Environment," in *2020 29th International Conference on Computer Communications and Networks (ICCCN)*. IEEE, Aug. 2020, pp. 1–9.

[91] M. Bachl, T. Zseby, and J. Fabini, "Rax: Deep Reinforcement Learning for Congestion Control," in *ICC 2019 - 2019 IEEE International Conference on Communications (ICC)*. IEEE, May 2019, pp. 1–6.

[92] R. Galliera, A. Morelli, R. Fronteddu, and N. Suri, "MARLIN: Soft Actor-Critic based Reinforcement Learning for Congestion Control in Real Networks," in *NOMS 2023-2023 IEEE/IFIP Network Operations and Management Symposium*. IEEE, May 2023, pp. 1–10.

[93] K. Xiao, S. Mao, and J. K. Tugnait, "TCP-Drinc: Smart Congestion Control Based on Deep Reinforcement Learning," *IEEE Access*, vol. 7, pp. 11 892–11 904, 2019.

[94] Z. Xia, J. Wu, L. Wu, J. Yuan, J. Zhang, J. Li, and D. Wu, "RLCC: Practical Learning-based Congestion Control for the Internet," in *2021 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2021, pp. 1–8.

[95] Y. Wang, L. Wang, and X. Dong, "An Intelligent TCP Congestion Control Method Based on Deep Q Network," *Future Internet*, vol. 13, no. 10, p. 261, Oct. 2021.

[96] X. Huang, H. Zhu, L. Yang, C. Yin, J. Li, and H. Li, "TCP-QNCC: congestion control algorithm based on deep Q-network," in *Fifth International Conference on Artificial Intelligence and Computer Science (AICS 2023)*, Y. S. Shmaliy, Y. Sun, H. Zaidi, H. Meng, H. Kolivand, J. Luo, and M. Alazab, Eds. SPIE, Oct. 2023, p. 58.

[97] H. Xu, L. Wang, and F. Song, "KerDqn: Deep Reinforcement Learning Enhanced Congestion Control in Kernel," in *ICC 2024 - IEEE International Conference on Communications*. IEEE, 2024, pp. 479–484.

[98] E. Ağlamazlar, E. Eken, and H. B. Geçici, "A Deep Reinforcement Learning-Based TCP Congestion Control Algorithm: Design, Simulation, and Evaluation," arXiv:2508.01047, 2025.

[99] K. Aoki, R. Mitsuhashi, K. Iida, and Y. Takai, "Performance Enhancement of Adaptive Network Congestion Control Using Reinforcement Learning," in *2026 28th International Conference on Advanced Communications Technology (ICACT)*. IEEE, Feb. 2026, pp. 328–334.

[100] J. Li, Y. Guan, P. Ding, and S. Wang, "TCP-PPCC: Online-Learning Proximal Policy for Congestion Control," in *2020 21st Asia-Pacific Network Operations and Management Symposium (APNOMS)*. IEEE, 2020, pp. 243–246.

[101] X. Huang, M. Li, H. Liu, J. Liu, K. Bai, and S. He, "TCP-CFCC: Proximal Policy Optimization for Congestion Control," in *2022 IEEE 2nd International Conference on Power, Electronics and Computer Applications (ICPECA)*. IEEE, Jan. 2022, pp. 1188–1191.

[102] H. Shi and J. Wang, "Intelligent TCP Congestion Control Policy Optimization," *Applied Sciences*, vol. 13, no. 11, p. 6644, May 2023.

[103] Y. Yuan, "Research on TCP Congestion Control Strategy Based on Proximal Policy Optimization," in *2023 IEEE 11th Joint International Information Technology and Artificial Intelligence Conference (ITAIC)*. IEEE, Dec. 2023, pp. 1934–1938.

[104] Y. Guo and X. Gong, "Deep Reinforcement Learning Based Congestion Control for Delay-Sensitive Multimedia Streaming," in *2022 4th International Conference on Communications, Information System and Computer Engineering (CISCE)*. IEEE, May 2022, pp. 327–330.

[105] J. Xing and M. Shahzad, "A Reinforcement Learning Framework for Application-Specific TCP Congestion-Control," in *2025 IEEE International Performance, Computing, and Communications Conference (IPCCC)*. IEEE, Nov. 2025, pp. 1–10.

[106] A. Blokhin, V. Kalev, R. Pusev, I. Kviatkovsky, and D. Moskvitin, "Deep Reinforcement Learning-Based Congestion Control for File Transfer over QUIC," in *2024 IEEE International Multi-Conference on Engineering, Computer and Information Sciences (SIBIRCON)*. IEEE, 2024, pp. 25–30.

[107] H. Naqvi, M. H. Hilman, and B. Anggorojati, "Fine Tuning of Interval Configuration for Deep Reinforcement Learning Based Congestion Control," *Jurnal Ilmu Komputer dan Informasi*, vol. 16, no. 2, pp. 151–161, 2023.

[108] L. Kamau Kiarie, M. Derakhshani, and K. G. Kyriakopoulos, "Design Principles for Reinforcement Learning in Congestion Control Environments," *IEEE Access*, vol. 13, pp. 85 217–85 230, 2025.

[109] J. P. Martins, R. S. Souza, I. Almeida, and S. Lins, "Rainbow deep reinforcement learning for TCP congestion control," in *2021 IEEE Latin-American Conference on Communications (LATINCOM)*. IEEE, Nov. 2021, pp. 1–6.

[110] H. Chae, W. Lim, G. Noh, and E. Kim, "ACKurate-DQN: Adaptive Throughput-Based Congestion Control Using Deep Reinforcement Learning," in *2025 16th International Conference on Information and Communication Technology Convergence (ICTC)*. IEEE, Oct. 2025, pp. 1324–1327.

[111] ——, "Pacemaker Congestion Control: Ceiling-Normalized Reinforcement Learning for TCP," *IEEE Access*, vol. 14, pp. 43 891–43 906, 2026.

[112] M. Li, X. Huang, C. Jin, and Y. Pei, "A TCP Congestion Control Algorithm Based on Deep Reinforcement Learning Combined with Probe Bandwidth Mechanism," in *Proceedings of the 5th International Conference on Computer Science and Application Engineering*. ACM, Oct. 2021, pp. 1–7.

[113] H. Tian, W. Li, J. Zhang, X. Liao, D. Sun, D. Chen, B. Huang, W. Li, Y. Wang, and K. Chen, "Learn-to-Probe: Achieving Signal Distinguishability in Learning-based Congestion Control," in *Proceedings of the 21st European Conference on Computer Systems*. ACM, Apr. 2026, pp. 108–125.

[114] W. Pan, Y. Xu, and S. Liu, "Making TCP BBR Pacing Adaptive With Domain Knowledge Assisted Reinforcement Learning," *IEEE Transactions on Network Science and Engineering*, vol. 10, no. 4, pp. 2250–2264, 2023.

[115] J. Zhou, Z. Pan, Z. Li, G. Tyson, W. Li, X. Qiu, H. Pan, X. Zhang, and G. Xie, "Safety in DRL-Based Congestion Control: A Framework Empowered by Expert Refinement," *IEEE Transactions on Networking*, vol. 33, no. 6, pp. 2929–2944, Dec. 2025.

[116] Z. Zhang, S. Li, Y. Ge, G. Xiong, Y. Zhang, and K. Xiong, "PBQ-Enhanced QUIC: QUIC with Deep Reinforcement Learning Congestion Control Mechanism," *Entropy*, vol. 25, no. 2, p. 294, Feb. 2023.

[117] Z. Jia, C. Chen, A. Asheralieva, L. Guan, and Z. Xiao, "HAN: Adaptive DRL-Based Congestion Control via Model Uncertainty," *IEEE Transactions on Network and Service Management*, vol. 23, pp. 6022–6034, 2026.

[118] D. Sharma, V. Sharma, M. Bakshi, and V. Mehta, "Adaptive TCP Congestion Control using Deep Reinforcement Learning for TCP CUBIC Parameter Optimization," in *2026 IEEE International Conference on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI)*. IEEE, Mar. 2026, pp. 1–6.

[119] E. Abedini and M. Nickray, "CUBIC-Learn: A Reinforcement Learning Approach to CUBIC Congestion Control," *Jordanian Journal of Computers and Information Technology*, vol. 11, no. 4, p. 466, 2025.

[120] S. Tang, X. Jiang, M. Zhang, G. Jin, and H. Chen, "Combining reinforcement learning method to enhance LEDBAT++ over diversified network environments," *Journal of King Saud University - Computer and Information Sciences*, vol. 35, no. 9, p. 101730, Oct. 2023.

[121] M. Kheirkhah, J. Reis, F. Yang, D. Griffin, and M. Rio, "ACCT: An Intelligent Congestion Control Mechanism for Future Internet," in *GLOBECOM 2023 - 2023 IEEE Global Communications Conference*. IEEE, Dec. 2023, pp. 5530–5536.

[122] Z. Du, J. Zheng, H. Yu, H. Zhang, and G. Chen, "Libra: A Congestion Control Framework for Diverse Application Preferences and Network Conditions," *IEEE Transactions on Networking*, vol. 33, no. 2, pp. 510–525, Apr. 2025.

[123] Z. Du, J. Zheng, H. Yu, L. Kong, and G. Chen, "A unified congestion control framework for diverse application preferences and network conditions," in *Proceedings of the 17th International Conference on emerging Networking EXperiments and Technologies*. ACM, Dec. 2021, pp. 282–296.

[124] F. Han, Y. Wang, Y. Zhang, Q. Li, D. Zhao, and Y. Jiang, "Anole: A Pragmatic Blend of Classic and Learning-Based Algorithms in Congestion Control," *IEEE Transactions on Computers*, vol. 74, no. 7, pp. 2501–2514, 2025.

[125] X. Nie, Y. Zhao, Z. Li, G. Chen, K. Sui, J. Zhang, Z. Ye, and D. Pei, "Dynamic TCP Initial Windows and Congestion Control Schemes Through Reinforcement Learning," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 6, pp. 1231–1247, 2019.

[126] J. Zhang, Y. Zhang, E. Dong, Y. Zhang, S. Ren, Z. Meng, M. Xu, X. Li, Z. Hou, Z. Yang, and X. Fu, "Bridging the Gap between QoE and QoS in Congestion Control: A Large-scale Mobile Web Service Perspective," in *2023 USENIX Annual Technical Conference (USENIX ATC)*, 2023.

[127] Y. Li, J. S. Sidhu, and A. Bentaleb, "ALCHEMY: Reusing Congestion Control Wisdom for Adaptive QUIC Transport," in *2026 IEEE/ACM International Symposium on Quality of Service (IWQoS)*. IEEE, 2026, pp. 1–6.

[128] Z. Peng, W. Zhan, B. Li, X. Chen, and P. Liu, "HTPCCS: A Hybrid Two-Phase Learning Framework for Congestion Control Scheme Selection," in *2025 IEEE 19th International Conference on Big Data Science and Engineering (BigDataSE)*. IEEE, Nov. 2025, pp. 1–8.

[129] A. Kanungo and P. Panse, "An Adaptive Hirarchical Meta Agent for Intelligent Congestion Control in IP Networks using Machine Intelligence," *Journal of Automation, Mobile Robotics and Intelligent Systems*, pp. 126–133, 2026.

[130] Z. Jia, C. Chen, and L. Guan, "POCC: Learning Robust Congestion Control via Pessimistic Offline Reinforcement Learning," in *2026 IEEE Symposium on Computers and Communications (ISCC)*. IEEE, 2026, pp. 1–6.

[131] X. Ji, B. Han, X. Wang, R. Li, and J. Su, "DTCC: Decision Transformer-driven framework for adaptive network congestion control," *Computer Networks*, vol. 287, p. 112531, 2026.

[132] T. Huang, C. Zhou, L. Jia, R.-X. Zhang, and L. Sun, "Learned Internet Congestion Control for Short Video Uploading," in *Proceedings of the 30th ACM International Conference on Multimedia*. ACM, Oct. 2022, pp. 3064–3075.

[133] L. Zhang, Y. Cui, M. Wang, K. Zhu, Y. Zhu, and Y. Jiang, "DeepCC: Bridging the Gap Between Congestion

Control and Applications via Multiobjective Optimization," *IEEE/ACM Transactions on Networking*, vol. 30, no. 5, pp. 2274–2288, Oct. 2022.

[134] Q. Liu, P. Yang, F. Lyu, N. Zhang, and L. Yu, "Multi-Objective Network Congestion Control via Constrained Reinforcement Learning," in *2021 IEEE Global Communications Conference (GLOBECOM)*. IEEE, Dec. 2021, pp. 1–6.

[135] J. Liu, X. Li, F. Tang, P. Li, L. Chen, J. Yu, Y. Zhu, P.-A. Heng, and L. T. Yang, "An Adaptive and Interpretable Congestion Control Service Based on Multi-Objective Reinforcement Learning," *IEEE Transactions on Services Computing*, vol. 18, no. 2, pp. 586–602, Mar. 2025.

[136] R. Liao, J. Yan, T. Lin, and Y. Chen, "Learning Based Deadline Aware Congestion Control," in *5th Asia-Pacific Workshop on Networking (APNet 2021)*. ACM, 2021, pp. 9–11.

[137] R. Liao, Y. Zhang, J. Yan, Y. Cai, and N. Tao, "STOP: Joint send buffer and transmission control for user-perceived deadline guarantee via curriculum guided-deep reinforcement learning," *Journal of Network and Computer Applications*, vol. 221, p. 103787, Jan. 2024.

[138] Y. Liu, Y. Fan, J. Ren, P. Xie, L. Liu, and H. Ma, "Zygos: A Reward Correction Mechanism for Reinforcement Learning-based Congestion Control," in *2024 20th International Conference on Mobility, Sensing and Networking (MSN)*. IEEE, Dec. 2024, pp. 16–23.

[139] S. Emara, F. Wang, B. Li, and T. Zeyl, "Pareto: Fair Congestion Control With Online Reinforcement Learning," *IEEE Transactions on Network Science and Engineering*, vol. 9, no. 5, pp. 3731–3748, 2022.

[140] Z. Peng, W. Zhan, M. Han, X. Chen, and X. Chen, "MACC: Multi-Agent Reinforcement Learning for Scalable and Fair Network Congestion Control," in *2025 IEEE 19th International Conference on Big Data Science and Engineering (BigDataSE)*. IEEE, Nov. 2025, pp. 20–27.

[141] H. Bai, H. Li, J. Que, M. Zhang, Z. Hu, X. Xu, B. Lin, R. Huang, J. Qiu, and S. Deng, "Pisces: Towards Adaptive and Fair Congestion Control via Multi-Agent Meta-Reinforcement Learning," in *Proceedings of the 54th International Conference on Parallel Processing*. ACM, 2025, pp. 104–114.

[142] H. Tian, X. Liao, D. Sun, C. Zeng, Y. Jin, J. Zhang, X. Wan, Z. Wang, Y. Wang, and K. Chen, "Achieving Fairness Generalizability for Learning-based Congestion Control with Jury," in *Proceedings of the Twentieth European Conference on Computer Systems*. ACM, Mar. 2025, pp. 413–427.

[143] Z. Huang, K. Liu, and J. Pan, "End-to-End Congestion Control as Learning for Unknown Games with Bandit Feedback," in *2023 IEEE 43rd International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2023, pp. 327–338.

[144] Z. Huang and J. Pan, "Game-Theoretic Bandits for Network Optimization With High-Probability Swap-Regret Upper Bounds," *IEEE/ACM Transactions on Networking*, vol. 32, no. 6, pp. 4855–4870, Dec. 2024.

[145] L. Giacomoni and G. Parisis, "Reinforcement Learning-based Congestion Control: A Systematic Evaluation of Fairness, Efficiency and Responsiveness," in *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*. IEEE, May 2024, pp. 1451–1460.

[146] M. Mazilu, L. Giacomoni, and G. Parisis, "Learning-Based vs Human-Derived Congestion Control: An In-Depth Experimental Study," arXiv, 2025.

[147] Z. Xu, J. Tang, C. Yin, Y. Wang, and G. Xue, "Experience-Driven Congestion Control: When Multi-Path TCP Meets Deep Reinforcement Learning," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 6, pp. 1325–1336, 2019.

[148] C. Huang, J. Zhang, and T. Huang, "Objective-Oriented Resource Pooling in MPTCP: A Deep Reinforcement Learning Approach," in *2020 3rd International Conference on Hot Information-Centric Networking (HotICN)*. IEEE, Dec. 2020, pp. 175–181.

[149] S. R. Pokhrel and A. Walid, "Learning to Harness Bandwidth With Multipath Congestion Control and Scheduling," *IEEE Transactions on Mobile Computing*, vol. 22, no. 2, pp. 996–1009, Feb. 2023.

[150] S. R. Pokhrel, J. Kua, D. Satish, S. Ozer, J. Howe, and A. Walid, "DDPG-MPCC: An Experience Driven Multipath Performance Oriented Congestion Control," *Future Internet*, vol. 16, no. 2, p. 37, Jan. 2024.

[151] R. Zhuang, J. Han, K. Xue, J. Li, D. S. L. Wei, R. Li, Q. Sun, and J. Lu, "Achieving Flexible and Lightweight Multipath Congestion Control Through Online Learning," *IEEE Transactions on Network and Service Management*, vol. 20, no. 1, pp. 46–59, Mar. 2023.

[152] H. Yu, J. Zheng, Z. Du, Y. Liu, B. Quan, T. Chen, and G. Chen, "When Classic Meets Intelligence: A Hybrid Multipath Congestion Control Framework," *IEEE/ACM Transactions on Networking*, vol. 32, no. 4, pp. 3575–3590, Aug. 2024.

[153] D. Wei, J. Zhang, H. Li, Y. Liu, X. Zhang, T. Pan, and T. Huang, "Breaking the Inertial Thinking: Non-Blocking Multipath Congestion Control Based on the Single-Subflow Reinforcement Learning Model," *IEEE Transactions on Network and Service Management*, vol. 21, no. 3, pp. 2876–2887, 2024.

[154] T. Huang, R.-X. Zhang, C. Zhou, and L. Sun, "QARC: Video Quality Aware Rate Control for Real-Time Video Streaming based on Deep Reinforcement Learning," in *Proceedings of the 26th ACM international conference on Multimedia*. ACM, Oct. 2018, pp. 1208–1216.

[155] J. Fang, M. Ellis, B. Li, S. Liu, Y. Hosseinkashi, M. Revow, A. Sadovnikov, Z. Liu, P. Cheng, S. Ashok, D. Zhao, R. Cutler, Y. Lu, and J. Gehrke, "Reinforcement learning for bandwidth estimation and congestion control in real-time communications," arXiv:1912.02222, 2019.

[156] A. Zhou, H. Zhang, G. Su, L. Wu, R. Ma, Z. Meng, X. Zhang, X. Xie, H. Ma, and X. Chen, "Learning to Coordinate Video Codec with Transport Protocol for Mobile Video Telephony," in *The 25th Annual International Conference on Mobile Computing and Networking*. ACM, Oct. 2019, pp. 1–16.

[157] H. Zhang, A. Zhou, J. Lu, R. Ma, Y. Hu, C. Li, X. Zhang, H. Ma, and X. Chen, "OnRL: improving mobile video telephony via online reinforcement learning," in *Proceedings of the 26th Annual International Conference on Mobile Computing and Networking*. ACM, 2020, pp. 1–14.

[158] H. Zhang, A. Zhou, Y. Hu, C. Li, G. Wang, X. Zhang, H. Ma, L. Wu, A. Chen, and C. Wu, "Loki: improving long tail performance of learning-based real-time video adaptation by fusing rule-based models," in *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking*. ACM, Oct. 2021, pp. 775–788.

[159] B. Wang, Y. Zhang, S. Qian, Z. Pan, and Y. Xie, "A Hybrid Receiver-side Congestion Control Scheme for Web Real-time Communication," in *Proceedings of the 12th ACM Multimedia Systems Conference*. ACM, 2021, pp. 332–338.

[160] A. Bentaleb, M. N. Akcay, M. Lim, A. C. Begen, and R. Zimmermann, "BoB: Bandwidth Prediction for Real-Time Communications Using Heuristic and Reinforcement Learning," *IEEE Transactions on Multimedia*, vol. 25, pp. 6930–6945, 2023.

[161] K. Zhang, Z. Wang, H. Ma, H. Du, and W. Zhang, "HybridRTS: A Hybrid Congestion Control Framework with Rule and Reinforcement Learning for Low-Latency WebRTC Live Video Streaming," in *2022 IEEE 24th Int Conf on High Performance Computing & Communications; 8th Int Conf on Data Science & Systems; 20th Int Conf on Smart City; 8th Int Conf on Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys)*. IEEE, Dec. 2022, pp. 1757–1764.

[162] X. Wu, B. Xu, H. Chen, and Z. Ma, "DynA: Breaking Fixed-Increase Rigidity in Google Congestion Control," in *Proceedings of the 10th Asia-Pacific Workshop on Networking*. ACM, Aug. 2026, pp. 308–309.

[163] J. Eo, Z. Niu, W. Cheng, F. Y. Yan, R. Gao, J. Kardhashi, S. Inglis, M. Revow, B.-G. Chun, P. Cheng, and Y. Xiong, "OpenNetLab: Open Platform for RL-based Congestion Control for Real-Time Communications," in *Proceedings of the 6th Asia-Pacific Workshop on Networking*. ACM, 2022, pp. 70–75.

[164] D. Markudova and M. Meo, "Advancing Congestion Control for Real-Time Communications With Reinforcement Learning: The ReCoCo Framework," *IEEE Transactions on Network and Service Management*, vol. 23, pp. 3998–4008, 2026.

[165] M. R. Islam, X. Zhang, and P.-C. Huang, "GenRL-CC: A GAN-Augmented Deep Reinforcement Learning Framework for Adaptive Congestion Control in Real-Time Communication," in *2026 35th International Conference on Computer Communications and Networks (ICCCN)*. IEEE, 2026, pp. 1–9.

[166] S. Khairy, G. Mittag, V. Gopal, F. Y. Yan, Z. Niu, E. Ameri, S. Inglis, M. Golestaneh, and R. Cutler, "ACM MMSys 2024 Bandwidth Estimation in Real Time Communications Challenge," in *Proceedings of the ACM Multimedia Systems Conference 2024 on ZZZ*. ACM, Apr. 2024, pp. 339–345.

[167] T. Song and M. Meo, "Debunking Reinforcement Learning for Bandwidth Estimation in Real-Time Communications: When a Simple Regressor Suffices," in *2025 21st International Conference on Network and Service Management (CNSM)*. IEEE, Oct. 2025, pp. 1–9.

[168] Y. Chen, H. Shi, Q. Weng, and Z. Shi, "Congestion Control Design of Multicast QUIC Based on Reinforcement Learning," in *2023 International Conference on Ubiquitous Communication (Ucom)*. IEEE, 2023, pp. 232–236.

[169] W. Yu, J. Liu, X. Di, P. Xiao, and H. Qi, "DNCCQ-PPO: A dynamic network congestion control algorithm based on deep reinforcement learning for XQUIC," *Journal of Network and Computer Applications*, vol. 245, p. 104371, Jan. 2026.

[170] J. S. Sidhu and A. Bentaleb, "Towards a Pluggable and Implementation-Agnostic Learning-Based Congestion Control for QUIC," *ACM Transactions on Multimedia Computing, Communications, and Applications*, 2026.

[171] G. T. Suarez, J. S. Sidhu, and A. Bentaleb, "PulseQUIC: Enhancing QUIC-Based Video Streaming through DRL-Guided Adaptive Pacing," in *Proceedings of the 3rd Workshop on Emerging Multimedia Systems*. ACM, 2025, pp. 43–48.

[172] D. Lan, X. Tan, J. Lv, Y. Jin, and J. Yang, "A Deep Reinforcement Learning Based Congestion Control Mechanism for NDN," in *ICC 2019 - 2019 IEEE International Conference on Communications (ICC)*. IEEE, May 2019, pp. 1–7.

[173] R. Zheng, B. Zhang, X. Zhao, L. Wang, and Q. Wu, "A Receiver-Driven Named Data Networking (NDN) Congestion Control Method Based on Reinforcement Learning," *Electronics*, vol. 13, no. 23, p. 4609, Nov. 2024.

[174] J. Yang, Y. Chen, K. Xue, J. Han, J. Li, D. S. L. Wei, Q. Sun, and J. Lu, "IEACC: An Intelligent Edge-Aided Congestion Control Scheme for Named Data Networking With Deep Reinforcement Learning," *IEEE Transactions on Network and Service Management*, vol. 19, no. 4, pp. 4932–4947, Dec. 2022.

[175] L. Zhang and X. Tian, "Research on SDN Congestion Control Based on Reinforcement Learning," *Journal of Physics: Conference Series*, vol. 2010, no. 1, p. 012164, 2021.

[176] K. Lei, Y. Liang, and W. Li, "Congestion Control in SDN-Based Networks via Multi-Task Deep Reinforcement Learning," *IEEE Network*, vol. 34, no. 4, pp. 28–34, 2020.

[177] K. Boussaoud, A. En-Nouaary, and M. Ayache, "Dual-Agent Reinforcement Learning for Adaptive Bandwidth Allocation and Congestion Control in SDN," *International Journal of Intelligent Engineering and Systems*, vol. 18, no. 7, pp. 338–361, Aug. 2025.

[178] K. Boussaoud, M. Ayache, and A. En-Nouaary, "A Multi-Agent Reinforcement Learning Approach for Congestion Control in network based-SDN," in *2023 10th International Conference on Future Internet of Things and Cloud (FiCloud)*. IEEE, Aug. 2023, pp. 250–255.

[179] M. König and M. Zitterbart, "Augmenting End-to-End Congestion Control with Centralized Reinforcement Learning," in *NOMS 2026-2026 IEEE Network Operations and Management Symposium*. IEEE, May 2026, pp. 1–8.

[180] T. Xu, L. Gong, W. Zhang, X. Li, X. Wang, and W. Pan, "Network congestion control algorithm based on Actor-Critic reinforcement learning model," in *AIP Conference Proceedings*, vol. 1955. Author(s), 2018, p. 040167.

[181] C. A. Gomez, X. Wang, and A. Shami, "Intelligent Active Queue Management Using Explicit Congestion Notification," in *2019 IEEE Global Communications Conference (GLOBECOM)*. IEEE, Dec. 2019, pp. 1–6.

[182] F. Jiang, C. Feng, C. Zhu, and Y. Sun, "Performance Analysis of Active Queue Management Algorithm Based on Reinforcement Learning," *Journal Européen des Systèmes Automatisés*, vol. 53, no. 5, pp. 637–644, Nov. 2020.

[183] A. B. Yousif, H. J. Hassan, and G. M. Abdulsaheb, "Intelligent Parameter Tuning using Deep Q-network in Adaptive Queue Management Systems," *Iraqi Journal of Computer, Communication, Control and System Engineering*, pp. 62–71, 2022.

[184] M. Q. Matrood and M. H. Ali, "Adaptive Random Early Detection Using Deep Reinforcement Learning for Enhanced Congestion Control in Dynamic TCP/IP Networks," *Al-Salam Journal for Engineering and Technology*, vol. 4, no. 1, pp. 135–149, Nov. 2024.

[185] S. J. Mohammed, M. H. Ali, and M. S. shwaish, "DDPG-Enhanced Integration of the BLUE Algorithm for Adaptive Active Queue Management and Congestion Control," *International Journal of Multidisciplinary Studies and Innovative Technologies*, vol. 10, no. 1, p. 57, 2026.

[186] N. Liu, "Q-Learning-Based Adaptive Scheduling Strategy for Library Network Congestion Control," in *Proceedings of the 3rd International Conference on Machine Intelligence and Digital Applications*. ACM, Apr. 2026, pp. 1650–1656.

[187] D. Satish, J. Kua, and S. R. Pokhrel, "Active Queue Management in L4S with Asynchronous Advantage Actor-Critic: A FreeBSD Networking Stack Perspective," *Future Internet*, vol. 16, no. 8, p. 265, 2024.

[188] L. C. de Almeida, W. R. D. da Silva, T. C. Tavares, R. Pasquini, C. Papagianni, and F. L. Verdi, "DESiRED — Dynamic, Enhanced, and Smart iRED: A P4-AQM with Deep Reinforcement Learning and In-band Network Telemetry," *Computer Networks*, vol. 244, p. 110326, May 2024.

[189] M. Bachl, J. Fabini, and T. Zseby, "LFQ: Online Learning of Per-flow Queuing Policies using Deep Reinforcement Learning," in *2020 IEEE 45th Conference on Local Computer Networks (LCN)*. IEEE, Nov. 2020, pp. 417–420.

[190] J. Bai, T. Zhang, and G. Xie, "MACC: Cross-Layer Multi-Agent Congestion Control with Deep Reinforcement Learning," arXiv:2206.01972, 2022.

[191] F. Michel, M. Trevisan, D. Giordano, and O. Bonaventure, "A first look at starlink performance," in *Proceedings of the 22nd ACM Internet Measurement Conference*. ACM, Oct. 2022, pp. 130–136. [Online]. Available: http://dx.doi.org/10.1145/3517745.3561416

[192] S. Abbasloo, C.-Y. Yen, and H. J. Chao, "Wanna Make Your TCP Scheme Great for Cellular Networks? Let Machines Do It for You!" *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 1, pp. 265–279, Jan. 2021.

[193] Y. Xie, X. Jiang, G. Jin, and H. Chen, "BBR-FIT: An Intelligent BBR based on the Reinforcement Learning to Boost the Network Efficiency over Time-Varying Networks," in *2022 IEEE 24th Int Conf on High Performance Computing & Communications; 8th Int Conf on Data Science & Systems; 20th Int Conf on Smart City; 8th Int Conf on Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys)*. IEEE, Dec. 2022, pp. 1633–1640.

[194] Z. Wu, C.-Y. Huang, and P. Ramanathan, "MIQA: An Application Agent for Immersive Content Delivery Over Millimeter Waves," *IEEE Transactions on Mobile Computing*, vol. 24, no. 5, pp. 3750–3763, May 2025.

[195] T. Yang, Q. Li, C. Guo, M. Wang, C. Liang, and H. Liang, "Digital Twin-Enabled Intelligent Congestion Control for QUIC-based E2E Communication," in *2025 IEEE 102nd Vehicular Technology Conference (VTC2025-Fall)*. IEEE, Oct. 2025, pp. 1–6.

[196] Z. Zhong, W. Wang, Y. Shao, Z. Li, H. Pan, H. Guan, G. Tyson, G. Xie, and K. Zheng, "Muses: Enabling Lightweight Learning-Based Congestion Control for Mobile Devices," in *IEEE INFOCOM 2022 - IEEE Conference on Computer Communications*. IEEE, May 2022, pp. 2208–2217.

[197] M. Chen, R. Li, J. Crowcroft, J. Wu, Z. Zhao, and H. Zhang, "RAN Information-Assisted TCP Congestion Control Using Deep Reinforcement Learning With Reward Redistribution," *IEEE Transactions on Communications*, vol. 70, no. 1, pp. 215–230, Jan. 2022.

[198] C. Serra, E. Paolini, R. Immich, A. Sacco, G. Marchetto, and F. Esposito, "Flecto: Cross-Layer Adaptive Congestion Control with Reinforcement Learning," in *2025 IEEE 26th International Conference on High Performance Switching and Routing (HPSR)*. IEEE, May 2025, pp. 1–6.

[199] Q. Pan and A. Nakao, "Intelligent Cross-Layer Congestion Control for Dynamic mmWave Networks with High Throughput and Fairness," in *GLOBECOM 2025 - 2025 IEEE Global Communications Conference*. IEEE, Dec. 2025, pp. 1316–1321.

[200] H. Ma and D. Xu, "RLRBM: A Reinforcement Learning-based RAN Buffer Management Scheme," in *2022 18th International Conference on Mobility, Sensing and Networking (MSN)*. IEEE, Dec. 2022, pp. 142–149.

[201] J. P. Martins, I. Almeida, R. Souza, and S. Lins, "Policy Distillation for Real-Time Inference in Fronthaul Congestion Control," *IEEE Access*, vol. 9, pp. 154 471–154 483, 2021.

[202] J. Martins, I. Almeida, R. Souza, and S. Lins, "Closed-form congestion control via deep symbolic regression," arXiv:2405.01435, 2024.

[203] H. Naqvi and B. Anggorojati, "Ablation Study of Deep Reinforcement Learning Congestion Control in Cellular Network Settings," in *2022 25th International Symposium on Wireless Personal Multimedia Communications (WPMC)*. IEEE, Oct. 2022, pp. 80–85.

[204] J. Yao, K. Liu, G. Zhang, J. Y. B. Lee, T. A. Benson, Y. Bao, and M. Chen, "A Unified Framework for DRL-Based Congestion Control to Optimize QoS Over Mobile Networks," in *2025 IEEE/ACM 33rd International Symposium on Quality of Service (IWQoS)*. IEEE, 2025, pp. 1–10.

[205] H. Zhang, A. Zhou, R. Ma, J. Lu, and H. Ma, "Arsenal: Understanding Learning-Based Wireless Video Transport via In-Depth Evaluation," *IEEE Transactions on Vehicular Technology*, vol. 70, no. 10, pp. 10 832–10 844, Oct. 2021.

[206] N. Smirnov and S. Tomforde, "Gymir5G: A Simulation Platform to Study Data Transmission over WebRTC in 5G Networks with Deep Learning Assistance," in *2024 International Conference on Software, Telecommunications and Computer Networks (SoftCOM)*. IEEE, 2024, pp. 1–6.

[207] ——, "Real-time rate control of WebRTC video streams in 5G networks: Improving quality of experience with Deep Reinforcement Learning," *Journal of Systems Architecture*, vol. 148, p. 103066, Mar. 2024.

[208] J. Liu, Y. Chen, C. Liu, Y. Jiang, F. You, and Y. Li, "Research on Real-Time Audio and Video Communication Technology for Power Emergency Command Based on Deep Learning," in *2025 IEEE 8th International Conference on Automation, Electronics and Electrical Engineering (AUTEEE)*. IEEE, Nov. 2025, pp. 893–898.

[209] L. Cui, Z. Yuan, Z. Ming, and S. Yang, "Improving the Congestion Control Performance for Mobile Networks in High-Speed Railway via Deep Reinforcement Learning," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 6, pp. 5864–5875, 2020.

[210] J. Xu, B. Ai, L. Wu, and L. Chen, "Handover-Aware Cross-Layer Aided TCP With Deep Reinforcement Learning for High-Speed Railway Networks," *IEEE Networking Letters*, vol. 3, no. 1, pp. 31–35, Mar. 2021.

[211] J. Xu and B. Ai, "Deep Reinforcement Learning for Handover-Aware MPTCP Congestion Control in Space-Ground Integrated Network of Railways," *IEEE Wireless Communications*, vol. 28, no. 6, pp. 200–207, Dec. 2021.

[212] H. Wang, H. Li, and Y. Zhao, "An Intelligent Congestion Control Strategy in Heterogeneous V2X Based on Deep Reinforcement Learning," *Symmetry*, vol. 14, no. 5, p. 947, May 2022.

[213] G. Maglione, V. Rakocevic, M. Amend, and T. Soleymani, "Deep Adaptive Rate Allocation in Volatile Heterogeneous Wireless Networks," arXiv:2603.20926, 2026.

[214] S. V. Jansi Rani, R. S. Milton, L. Yamini, and K. Shivaani, "Reinforcement Learning Approach to Improve Transmission Control Protocol," in *2019 International Conference on Computational Intelligence in Data Science (ICCIDS)*. IEEE, Feb. 2019, pp. 1–5.

[215] R. Pradeep, A. Babu, M. K. S, and A. R. Kumar Parthiban, "Temporal Difference Learning Model for TCP End-To-End Congestion Control in Heterogeneous Wireless Networks," in *2021 12th International Conference on Computing Communication and Networking Technologies (ICCCNT)*. IEEE, 2021, pp. 1–5.

[216] K. S. Midhula and P. A. R. Kumar, "An Adaptive Congestion Control Protocol for Wireless Networks Using Deep Reinforcement Learning," *IEEE Transactions on Network and Service Management*, vol. 21, no. 2, pp. 2027–2043, Apr. 2024.

[217] Y. Cao, J. Nie, Y. Fan, X. Shao, and G. Lei, "DRLFcc: Deep Reinforcement Learning-empowered Congestion Control Mechanism for TCP Fast Recovery in High Loss Wireless Networks," in *GLOBECOM 2023 - 2023 IEEE Global Communications Conference*. IEEE, Dec. 2023, pp. 4345–4350.

[218] H. K. Molia, "Reinforcement Learning based Adaptive Congestion Control for TCP over Wireless Networks," *International Journal of Computer Networks and Applications*, vol. 11, no. 5, pp. 607–616, Oct. 2024.

[219] S. K. Shrestha, S. R. Pokhrel, and J. Kua, "On the Fairness of Internet Congestion Control over WiFi with Deep Reinforcement Learning," *Future Internet*, vol. 16, no. 9, p. 330, 2024.

[220] R. A. F. Shawish, A. Aneiba, H. Mahmoud, and D. Mi, "WiFi Network Congestion Control Using Deep Reinforcement Learning," in *2025 IEEE/CIC International Conference on Communications in China (ICCC)*. IEEE, Aug. 2025, pp. 1–6.

[221] Z. Li, N. Xu, and X. Zhang, "SAC-BBR: A Semantic-Aware and Cross-Layer Collaborative Congestion Control Mechanism for Heterogeneous Campus Networks," *Applied Sciences*, vol. 16, no. 11, p. 5587, 2026.

[222] G. Gong, X. Jiang, G. Jin, Y. Xie, and H. Chen, "Nuwa-RL: A Reinforcement Learning based Receiver-side Congestion Control Algorithm to Meet Applications Demands over Dynamic Wireless Networks," in *2022 IEEE 24th Int Conf on High Performance Computing & Communications; 8th Int Conf on Data Science & Systems; 20th Int Conf on Smart City; 8th Int Conf on Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys)*. IEEE, Dec. 2022, pp. 508–515.

[223] L. Wang, "Edge-Served Congestion Control for Wireless Multipath Transmission with a Transformer Agent," arXiv:2512.20186, 2025.

[224] Z. Zhang, T. Duan, L. Wang, Z. Fang, Z. Lin, Y. Lu, J. Wu, X. Du, M. Yang, Z. Chen, H. Cui, and J. Luo, "Transformer-Based Multipath Congestion Control: A Decoupled Approach for Wireless Uplinks," Mar. 2026.

[225] H. Cai, Z. Zhang, and Z. Li, "Deep Reinforcement Learning for TCP Congestion Control in UAV Flying Ad-Hoc Networks: A Continuous-Action PPO Approach," in *2026 7th Information Communication Technologies Conference (ICTC)*. IEEE, May 2026, pp. 358–363.

[226] Y. Feng, Y. Wang, B. Zhang, and L. Zhou, "Deep Reinforcement Learning Based TCP Congestion Control in UAV Assisted Wireless Networks," in *2023 International Conference on Wireless Communications and Signal Processing (WCSP)*. IEEE, Nov. 2023, pp. 862–867.

[227] H. Yuan, J. Chen, Y. Jiao, Z. Feng, G. Li, W. Dai, and H. Wang, "DRL-based Cross-layer Design for PHY Scheduling and Congestion Control in Anti-jamming Communications," in *GLOBECOM 2024 - 2024 IEEE Global Communications Conference*. IEEE, Dec. 2024, pp. 3164–3169.

[228] X. Meng, H. Li, and W. Han, "Multi-Band and Multi-Network Cooperative Transmission Algorithm for Wireless Resource Optimization," *IEEE Access*, vol. 12, pp. 31 916–31 929, 2024.

[229] R. V. Bhat, J. Haxhibeqiri, I. Moerman, and J. Hoebeke, "Multi-Context-aware RL approach towards INT-based Congestion Control Algorithm," in *2024 IEEE Conference on Standards for Communications and Networking (CSCN)*. IEEE, Nov. 2024, pp. 191–195.

[230] W. Li, H. Zhang, S. Gao, C. Xue, X. Wang, and S. Lu, "SmartCC: A Reinforcement Learning Approach for Multipath TCP Congestion Control in Heterogeneous Networks," *IEEE Journal on Selected Areas in Communications*, vol. 37, no. 11, pp. 2621–2633, Nov. 2019.

[231] B. He, J. Wang, Q. Qi, H. Sun, J. Liao, C. Du, X. Yang, and Z. Han, "DeepCC: Multi-Agent Deep Reinforcement Learning Congestion Control for Multi-Path TCP Based on Self-Attention," *IEEE Transactions on Network and Service Management*, vol. 18, no. 4, pp. 4770–4788, Dec. 2021.

[232] B. He, J. Wang, Q. Qi, H. Sun, J. Liao, L. Lu, and Z. Han, "Learning-Based Real-Time Transmission Control for Multi-Path TCP Networks," *IEEE Transactions on Cognitive Communications and Networking*, vol. 9, no. 5, pp. 1353–1369, Oct. 2023.

[233] C. Xu, J. Qin, P. Zhang, K. Gao, and L. A. Grieco, "Reinforcement Learning-Based Mobile AR/VR Multipath Transmission With Streaming Power Spectrum Density Analysis," *IEEE Transactions on Mobile Computing*, vol. 21, no. 12, pp. 4529–4540, Dec. 2022.

[234] Y. Liu, J. Zhang, D. Wei, H. Li, C. Li, and Y. Zhang, "Leopard: A Pragmatic Learning-Based Multipath Congestion Control for Rapid Adaptation to New Network Conditions," in *ICC 2023 - IEEE International Conference on Communications*. IEEE, May 2023, pp. 1124–1129.

[235] X. Li, F. Tang, J. Liu, L. T. Yang, L. Fu, and L. Chen, "AUTO: Adaptive Congestion Control Based on Multi-Objective Reinforcement Learning for the Satellite-Ground Integrated Network," in *2021 USENIX Annual Technical Conference (USENIX ATC)*, 2021.

[236] Y. Yan, J. Li, J. Han, Q. Long, K. Xue, H. Chen, and N. Qiao, "A Handover-Aware Congestion Control Algorithm Assisted by DRL in LEO Satellite Networks," in *ICC 2025 - IEEE International Conference on Communications*. IEEE, 2025, pp. 4720–4725.

[237] C. Liu, Y. Zhang, S. Lu, G. Yang, and C. Ji, "Intelligent Congestion Control in Satellite Networks," in *2025 IEEE 25th International Conference on Communication Technology (ICCT)*. IEEE, Oct. 2025, pp. 1200–1207.

[238] J. Gui, Z. Li, J. Zhang, X. Deng, and G. Min, "Multi-Heterogeneous-Agent DRL for Efficient Congestion Control With Reward Redistribution in Space–Air–Ground Integrated Networks," *IEEE Transactions on Network Science and Engineering*, vol. 13, pp. 3035–3052, 2026.

[239] E. Lv, X. Wang, B. Yi, L. Qiu, K. Zhang, J. Guo, M. Huang, Y. Kou, and K. Li, "A QoS-aware hierarchical intelligent congestion control framework for space–air–ground–sea integrated network," *Journal of Network and Computer Applications*, vol. 253, p. 104544, 2026.

[240] J. Yu, S. Pan, R. Gao, and Y. Li, "Low-Delay Transmission for Non-Terrestrial Networks Based on FEC and Reinforcement Learning," in *2023 IEEE/CIC International Conference on Communications in China (ICCC)*. IEEE, Aug. 2023, pp. 1–6.

[241] Z. Wang, J. Zhang, X. Zhang, and W. Wang, "Reinforcement Learning Based Congestion Control in Satellite Internet of Things," in *2019 11th International Conference on Wireless Communications and Signal Processing (WCSP)*. IEEE, Oct. 2019, pp. 1–6.

[242] A. Masood, T. Ha, D. S. Lakew, N.-N. Dao, and S. Cho, "Intelligent TCP Congestion Control Scheme in Internet

of Deep Space Things Communication," *IEEE Transactions on Network Science and Engineering*, vol. 10, no. 3, pp. 1472–1486, May 2023.

[243] T. Ha, A. Masood, W. Na, and S. Cho, "Intelligent Multi-Path TCP Congestion Control for video streaming in Internet of Deep Space Things communication," *ICT Express*, vol. 9, no. 5, pp. 860–868, Oct. 2023.

[244] A. Valentine, M. Mazilu, J. Knowles, I. Wakeman, and G. Parisis, "Emulation vs Simulation: A Case Study from Congestion Control Algorithms in Low Earth Orbit Satellite Networks," arXiv:2608.02095, 2026.

[245] J. Jayaudhaya, S. Supriya, V. A. Kandaswamy, S. P. V, S. Kamatchi, and C. P. Priya, "ACoCo: An Adaptive Congestion Control Approach for Enhancing CoAP Performance in IoT Network," in *2023 2nd International Conference on Applied Artificial Intelligence and Computing (ICAAIC)*. IEEE, May 2023, pp. 189–194.

[246] P. K. Donta, S. N. Srirama, T. Amgoth, and C. S. R. Annavarapu, "iCoCoA: intelligent congestion control algorithm for CoAP using deep reinforcement learning," *Journal of Ambient Intelligence and Humanized Computing*, vol. 14, no. 3, pp. 2951–2966, Jan. 2023.

[247] M. A. Tariq, M. T. R. Khan, M. M. Saad, M. M. Islam, and D. Kim, "AI-Enabled Reliable Delay Sensitive Communication Mechanism in IoUT Using CoAP," *IEEE Sensors Journal*, vol. 23, no. 16, pp. 18 832–18 841, Aug. 2023.

[248] S. Choi, "Energy-Efficient Dynamic Retransmission Timeouts with Enhanced Stability for Constrained Application Protocol-Based Internet of Things Networks via Edge Intelligence-Assisted Cross-Layer Congestion Control," *Electronics*, vol. 15, no. 13, p. 2884, 2026.

[249] S. P. Janani, I. J. J., and G. J. L. P., "Adaptive Markov Decision Process-Based Congestion Control Using Deep Deterministic Policy Gradient in Distributed MQTT Brokers," *Journal of Trends in Computer Science and Smart Technology*, vol. 8, no. 1, pp. 89–113, Mar. 2026.

[250] W. Li, F. Zhou, W. Meleis, and K. Chowdhury, "Learning-Based and Data-Driven TCP Design for Memory-Constrained IoT," in *2016 International Conference on Distributed Computing in Sensor Systems (DCOSS)*. IEEE, May 2016, pp. 199–205.

[251] L. Wang, H. Xu, L. Li, W. Su, and F. Song, "KONTROL: Offloading Data-Driven Congestion Control Intelligence for IoT Networks," *IEEE Internet of Things Journal*, vol. 13, no. 2, pp. 1936–1946, Jan. 2026.

[252] S. Lu, S. Q. Cong, Q. R. He, C. P. Li, and Y. Ju, "WAPI-Integrated Congestion Control and Adaptive Transmission for Power IoT," *Advanced Electromagnetics*, vol. 15, no. 3, pp. 9976–9988, Aug. 2026.

[253] B. T. Nguyen, L. Murphy, and G.-M. Muntean, "Energy-Efficient QoS-Based Congestion Control for Reliable Communications in Wireless Multimedia Sensor Networks," in *2018 IEEE International Conference on Communications Workshops (ICC Workshops)*. IEEE, May 2018, pp. 1–6.

[254] Z. He, L. Chen, F. Li, and G. Jin, "Congestion Avoidance in Intelligent Transport Networks Based on WSN-IoT through Controlling Data Rate of Zigbee Protocol by Learning Automata," *Electronics*, vol. 12, no. 9, p. 2070, Apr. 2023.

[255] M. Alipio and M. Bures, "A cache-aware congestion control mechanism using deep reinforcement learning for wireless sensor networks," *Ad Hoc Networks*, vol. 166, p. 103678, Jan. 2025.

[256] K. N. Mishra, P. Krishnamoorthy, B. S. Lakshmi, S. Zope, R. R. Chaudhuri, and V. N. Kollu, "Explainable cache aware congestion control with using deep reinforcement learning in robotic sensor networks," *Engineering Applications of Artificial Intelligence*, vol. 181, p. 115570, Oct. 2026.

[257] Y. Hou, H. He, X. Jiang, S. Chen, and J. Yang, "Deep-Reinforcement-Learning-Aided Loss-Tolerant Congestion Control for 6LoWPAN Networks," *IEEE Internet of Things Journal*, vol. 10, no. 21, pp. 19 125–19 140, Nov. 2023.

[258] P. A. Forero, P. Zhang, and D. Radosevic, "Toward Congestion Control in Lossy Networks via Local Queue-Management Policies," in *MILCOM 2022 - 2022 IEEE Military Communications Conference (MILCOM)*. IEEE, Nov. 2022, pp. 879–886.

[259] P. A. Forero, D. Radosevic, J. M. McInerney, and J. L. Chavez, "Learning Hierarchical Management Policies for Data Queues with Applications to Maritime Networking," in *OCEANS 2023 - MTS/IEEE U.S. Gulf Coast*. IEEE, 2023, pp. 1–7.

[260] H. K. Molia and A. D. Kothari, "TCP-RLLD: TCP with reinforcement learning based loss differentiation for mobile adhoc networks," *Wireless Networks*, vol. 29, no. 5, pp. 1937–1948, Feb. 2023.

[261] S. Mahajan, P. Srideviponmalar, R. Harikrishnan, and K. Kotecha, "Reinforcement Learning Based Congestion Control Technique for Wireless Mesh Networks," *International Journal of Networked and Distributed Computing*, vol. 13, no. 1, 2025.

[262] H. Li and Z. Xin, "Lightweight Adaptive Reinforcement Learning-Based TCP Congestion Control for Multi-Hop Ad Hoc Networks," *Electronics*, vol. 15, no. 5, p. 947, Feb. 2026.

[263] M. Vazaios, L. J. d. l. C. Llopis, J. P. A. Leon, and J. Parra-Arnau, "A Multi-Agent Reinforcement Learning Congestion Control Protocol for Wireless Networks," *IEEE Open Journal of the Communications Society*, vol. 7, pp. 11 500–11 523, 2026.

[264] A. R. Andrade-Zambrano, L. Lemus Cárdenas, J. P. Astudillo León, M. Eugenio Morocho-Cayamcela, and L. J. De La Cruz Llopis, "Performance Evaluation of DQN-Based Reinforcement Learning Congestion Control in Mobile Multi-Hop Wireless Networks," in *2025 International Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems (MSWiM)*. IEEE, Oct. 2025, pp. 562–569.

[265] A. P. Silva, K. Obraczka, S. Burleigh, J. M. Nogueira, and C. M. Hirata, "A congestion control framework for delay- and disruption tolerant networks," *Ad Hoc Networks*, vol. 91, p. 101880, Aug. 2019.

[266] A. R. Andrade-Zambrano, J. P. A. León, M. E. Morocho-Cayamcela, L. L. Cárdenas, and L. J. de la Cruz Llopis, "A Reinforcement Learning Congestion Control Algorithm for Smart Grid Networks," *IEEE Access*, vol. 12, pp. 75 072–75 092, 2024.

[267] W. Li, N. Zhang, J. Zhu, Z. Liu, W. Wang, Y. Li, and W. Guo, "Noninvasive real-time traffic and congestion control algorithm based on policy," *Journal of Computational Methods in Sciences and Engineering*, vol. 22, no. 6, pp. 2185–2199, Dec. 2022.

[268] C. Wang, X. Xu, K. Shi, W. Qu, and J. Xu, "Research on Reinforcement Learning Algorithm for Dynamic Congestion Control of Power Internet of Things in Complex Environment," in *2024 13th International Conference of Information and Communication Technology (ICTech)*. IEEE, Apr. 2024, pp. 64–69.

[269] C. Wang, W. Qu, Y. Zhang, Y. Zou, and L. Zhang, "TransAL-CC: An Asynchronous Reinforcement Learning Approach for Multipath Transmission Congestion Control in Power IoT," *IEEE Transactions on Network Science and Engineering*, vol. 13, pp. 2998–3016, 2026.

[270] E. Abedini and M. Nikray, "Digital Twin-Assisted Robust Reinforcement Learning for Detection, Intelligent Congestion Control, and Autonomous Recovery from GOOSE Flooding Attacks in IEC 61850-Based Smart Substations," 2026.

[271] X. An, H. She, L. Zhong, J. Yang, and Y. Guo, "An MPTCP congestion control method for heterogeneous IIoT networks based on multi-agent deep reinforcement learning," *Ad Hoc Networks*, vol. 194, p. 104371, Jan. 2027.

[272] P. Kutsevol, Y. Deshpande, and W. Kellerer, "Learning Semantic Congestion Control for Cyber Physical Systems," in *2025 21st International Conference on Network and Service Management (CNSM)*. IEEE, Oct. 2025, pp. 1–9.

[273] S. Yan, X. Wang, X. Zheng, Y. Xia, D. Liu, and W. Deng, "ACC: automatic ECN tuning for high-speed datacenter networks," in *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*. ACM, Aug. 2021, pp. 384–397.

[274] G. Bernárdez, J. Suárez-Varela, X. Shi, S. Xiao, X. Cheng, P. Barlet-Ros, and A. Cabellos-Aparicio, "GraphCC: A practical graph learning-based approach to Congestion Control in datacenters," *Computer Networks*, vol. 257, p. 110981, Feb. 2025.

[275] K. Zaeri and A. M. Abdelmoniem, "EMPRN: Reinforcement Learning-based ECN Tuning Using Message Passing Graph Recurrent Networks for Datacenters," in *ICC 2024 - IEEE International Conference on Communications*. IEEE, 2024, pp. 4668–4673.

[276] J. Hu, Z. Zhou, and J. Zhang, "Lightweight Automatic ECN Tuning Based on Deep Reinforcement Learning With Ultra-Low Overhead in Datacenter Networks," *IEEE Transactions on Network and Service Management*, vol. 21, no. 6, pp. 6398–6408, Dec. 2024.

[277] Y. Zhang, Y. Fan, Y. Xiao, H. Chen, P. Xie, L. Liu, and H. Ma, "BBQ: Dynamic-Buffer-Driven Automatic ECN Tunning in Datacenter," in *2024 IEEE/ACM 32nd International Symposium on Quality of Service (IWQoS)*. IEEE, 2024, pp. 1–6.

[278] A. Chaurasiya, H. K. Verma, and M. Sharma, "Cross-Layer Dual-Agent Reinforcement Learning for Intelligent Congestion Control in P4-Enabled Data Planes," in *2026 IEEE 1st International Conference on Intelligent Technologies for a Sustainable and Inclusive Future (ICITSIF)*. IEEE, May 2026, pp. 274–279.

[279] G. Diel, C. C. Miers, M. A. Pillon, and G. P. Koslovski, "RSCAT: Towards zero touch congestion control based on actor–critic reinforcement learning and software-defined networking," *Journal of Network and Computer Applications*, vol. 215, p. 103639, 2023.

[280] J. Liu, H. Li, J. Huang, Z. Li, P. Zhong, B. Huang, P. Dong, W. Tang, W. Jiang, J. Wang, and Y. Cui, "Automatic Dual Threshold Tuning for Switch Buffer Sharing in Datacenter Networking," *IEEE Transactions on Networking*, vol. 33, no. 6, pp. 3535–3550, Dec. 2025.

[281] C. Tessler, Y. Shpigelman, G. Dalal, A. Mandelbaum, D. Haritan Kazakov, B. Fuhrer, G. Chechik, and S. Mannor, "Reinforcement Learning for Datacenter Congestion Control," *ACM SIGMETRICS Performance Evaluation Review*, vol. 49, no. 2, pp. 43–46, Jan. 2022.

[282] B. Fuhrer, Y. Shpigelman, C. Tessler, S. Mannor, G. Chechik, E. Zahavi, and G. Dalal, "Implementing Reinforcement Learning Datacenter Congestion Control in NVIDIA NICs," in *2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. IEEE, May 2023, pp. 331–343.

[283] J. Ye, D. Lin, K. Cai, C. Zhou, J. He, and J. C. Lui, "Data-Driven Rate Control for RDMA Networks: A Lightweight Online Learning Approach," in *2023 IEEE 43rd International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2023, pp. 1–11.

[284] T. Wang, Y. Zhang, A. Zhou, and S. Wang, "An RDMA Congestion Control Enhancement Framework for Edge Datacenters," *IEEE Internet of Things Journal*, vol. 12, no. 24, pp. 52 162–52 178, Dec. 2025.

[285] Y. Liu, J. Han, K. Xue, J. Li, Q. Sun, and J. Lu, "DECC: Achieving Low Latency in Data Center Networks With Deep Reinforcement Learning," *IEEE Transactions on Network and Service Management*, vol. 20, no. 4, pp. 4313–4324, Dec. 2023.

[286] C. Yang, R. Wang, and J. Hao, "DuetCC: A Hybrid Data Driven and Model Guided Congestion Control for Data Center Networks," in *2025 5th International Conference on Wireless Communication, Networking and Internet of Things (WCNIoT)*. IEEE, Nov. 2025, pp. 107–110.

[287] H. Wang, K. Zheng, C. Reiss, and H. Shen, "NCC: Neighbor-aware Congestion Control based on Reinforcement Learning for Datacenter Networks," in *Proceedings of the 51st International Conference on Parallel Processing*. ACM, Aug. 2022, pp. 1–10.

[288] Z. Xiao, A. Harwood, and M. A. Rodriguez, "Optimal Rate Control for Latency-constrained High Throughput Big Data Applications," in *2022 IEEE International Conference on Big Data (Big Data)*. IEEE, Dec. 2022, pp. 234–241.

[289] A. Hassan and S. S. Heydari, "TCP Congestion Avoidance in Data Centres using Reinforcement Learning," in *2021 23rd International Conference on Advanced Communication Technology (ICACT)*. IEEE, Feb. 2021, pp. 306–311.

[290] T. Huang, X. Lu, D. Zhang, H. Cheng, P. Dong, and L. Zhang, "ACC-RL: Adaptive Congestion Control Based on Reinforcement Learning in Power Distribution Networks with Data Centers," *Energies*, vol. 16, no. 14, p. 5385, 2023.

[291] B. Chen, J. Pan, A. Huang, J. Fu, and D. Liao, "Research on Congestion Adaptive Control in Power Data Centers Based on Reinforcement Learning and Bayesian Networks," *International Journal of High Speed Electronics and Systems*, vol. 35, no. 05, 2025.

[292] S. Ketabi, H. Chen, H. Dong, and Y. Ganjali, "A Deep Reinforcement Learning Framework for Optimizing Congestion Control in Data Centers," in *NOMS 2023-2023 IEEE/IFIP Network Operations and Management Symposium*. IEEE, May 2023, pp. 1–7.

[293] J. Hu, Z. Zhou, and J. Wang, "Hierarchical Adaptive Learning-Based Congestion Control With Low Training Overhead for Datacenter Networks," *IEEE Transactions on Network and Service Management*, vol. 22, no. 5, pp. 4061–4069, Oct. 2025.

[294] R. Jin, J. Li, X. Tuo, W. Wang, and X. Li, "A congestion control method of SDN data center based on reinforcement learning," *International Journal of Communication Systems*, vol. 31, no. 17, 2018.

[295] P. Guo, L. Liang, H. Liu, and L. Du, "A congestion control algorithm based on deep reinforcement learning in SDN data center networks," in *International Conference on Neural Networks, Information, and Communication Engineering (NNICE 2022)*, R. Tiwari, Ed. SPIE, 2022, p. 53.

[296] F. Ruffy, M. Przystupa, and I. Beschastnikh, "Iroko: A Framework to Prototype Reinforcement Learning for Data Center Traffic Control," arXiv:1812.09975, 2018.

[297] P. Gawłowicz and A. Zubow, "ns-3 meets OpenAI Gym: The Playground for Machine Learning in Networking Research," in *Proceedings of the 22nd International ACM Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems*. ACM, Nov. 2019, pp. 113–120. [Online]. Available: http://dx.doi.org/10.1145/3345768.3355908

[298] H. Mao, P. Negi, A. Narayan, H. Wang, J. Yang, H. Wang, R. Marcus, R. Addanki, M. K. Shirkoohi, S. He, V. Nathan, F. Cangialosi, S. Venkatakrishnan, W.-H. Weng, S. Han, T. Kraska, and M. Alizadeh, "Park: An Open Platform for Learning-Augmented Computer Systems," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[299] L. Giacomoni, B. Benny, and G. Parisis, "RayNet: A Simulation Platform for Developing Reinforcement Learning-Driven Network Protocols," *ACM Transactions on Modeling and Computer Simulation*, vol. 34, no. 3, pp. 1–25, 2024.

[300] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: rapid prototyping for software-defined networks," in *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks*. ACM, Oct. 2010, pp. 1–6. [Online]. Available: http://dx.doi.org/10.1145/1868447.1868466

[301] R. Netravali, A. Sivaraman, S. Das, A. Goyal, K. Winstein, J. Mickens, and H. Balakrishnan, "Mahimahi: Accurate Record-and-Replay for HTTP," in *2015 USENIX Annual Technical Conference (USENIX ATC)*, 2015, pp. 417–429.

[302] S. Ashok, S. S. Duvvuri, N. Natarajan, V. N. Padmanabhan, S. Sellamanickam, and J. Gehrke, "iBox: Internet in a Box," in *Proceedings of the 19th ACM Workshop on Hot Topics in Networks*. ACM, Nov. 2020, pp. 23–29. [Online]. Available: http://dx.doi.org/10.1145/3422604.3425935

[303] S. Ashok, S. Tiwari, N. Natarajan, V. N. Padmanabhan, and S. Sellamanickam, "Data-Driven Network Path Simulation with iBox," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 6, no. 1, pp. 1–26, Feb. 2022.

[304] S. Abbasloo, "Internet Congestion Control Benchmarking," arXiv:2307.10054, 2023.

[305] Z. Chen, S. S. Ahmed, C. Wang, B. Godfrey, and G. Wang, "CCLab: Adversarial Testing of Learning- and Non-Learning-Based Congestion Controllers," arXiv:2605.21915, 2026.

[306] P. Pazhooheshy, S. Abbasloo, and Y. Ganjali, "Mahak: An Automated and Efficient Assessment Framework for Internet Control Algorithms," in *2025 IEEE 33rd International Conference on Network Protocols (ICNP)*. IEEE, 2025, pp. 1–13.

[307] A. Ghermezian, K. Karpov, D. Kachan, V. Kirova, and E. Siemens, "PPO-Driven Reinforcement Learning Congestion Control Under High- BDP Wide-Area Deployment: A Scaling Analysis," 2026. [Online]. Available: https://opendata.uni-halle.de//handle/1981185920/125740

[308] T. Gilad, N. H. Jay, M. Shnaiderman, B. Godfrey, and M. Schapira, "Robustifying Network Protocols with Adversarial Examples," in *Proceedings of the 18th ACM Workshop on Hot Topics in Networks*. ACM, Nov. 2019, pp. 85–92.

[309] H. Chen, Y. Zhang, X. Zhao, Y. Xiao, P. Xie, L. Liu, and H. Ma, "Exploring Potential Vulnerabilities in DRL-Based Congestion Control with Adversarial Policy," in *2025 IEEE 33rd International Conference on Network Protocols (ICNP)*. IEEE, 2025, pp. 1–11.

[310] Z. Yang, J. Cao, Z. Liu, X. Zhang, K. Sun, and Q. Li, "Good Learning, Bad Performance: A Novel Attack Against RL-Based Congestion Control Systems," *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 1069–1082, 2022.

[311] S. Patel, J. Zhang, S. A. Jyothi, and N. Narodytska, "Plume: A Framework for High Performance Deep RL Network Controllers via Prioritized Trace Sampling," arXiv:2302.12403, 2023.

[312] Z. Zheng, Z. Xia, Y.-C. Chou, and Y. Chen, "TRCC: Transferable Congestion Control With Reinforcement Learning," *IEEE Internet of Things Journal*, vol. 11, no. 2, pp. 2273–2285, Jan. 2024.

[313] H. Bai, H. Li, J. Que, M. Zhang, P. H. J. Chong, K. L. K. Sudheera, and X. Pei, "MetaCon: Revitalizing Internet Congestion Control with Meta-Reinforcement Learning," in *ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, Apr. 2025, pp. 1–5.

[314] Z. Chen, N. Kan, C. Li, W. Dai, J. Zou, and H. Xiong, "A Generalizable and Expressive Meta-Diffusion Policy for RTC Bandwidth Prediction," in *2025 IEEE International Conference on Multimedia and Expo (ICME)*. IEEE, 2025, pp. 1–6.

[315] J. Zhou, X. Qiu, Z. Li, G. Tyson, Q. Li, J. Duan, and Y. Wang, "Antelope: A Framework for Dynamic Selection of Congestion Control Algorithms," in *2021 IEEE 29th International Conference on Network Protocols (ICNP)*. IEEE, Nov. 2021, pp. 1–11.

[316] W. Yang, Y. Liu, C. Tian, J. Jiang, and L. Guo, "Gemini: Divide-and-Conquer for Practical Learning-Based Internet Congestion Control," in *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications*. IEEE, May 2023, pp. 1–10.

[317] D. Yuan, W. Zhang, Y. Qiu, H. Huang, M. Yang, P. Chen, K. Xiao, H. Yan, Y. He, and Y. Zhang, "Context-Aware Cross-Layer Congestion Control for Large-Scale Live Streaming," *IEEE/ACM Transactions on Networking*, vol. 32, no. 5, pp. 3743–3759, Oct. 2024.

[318] D. Yuan, Y. Qiu, W. Zhang, M. Yang, H. Du, Z. Wang, and Y. Zhang, "A Concern-Decoupled Architecture for Scenario-Optimized Congestion Control in Large-Scale Live Video CDNs," *IEEE Transactions on Networking*, vol. 34, pp. 2717–2732, 2026.

[319] M. Zangooei, J. Weil, A. Rizk, M. T. Arashloo, and R. Boutaba, "Analyzing Symbolic Properties for DRL Agents in Systems and Networking," *ACM SIGMETRICS Performance Evaluation Review*, vol. 54, no. 1, pp. 40–42, 2026.

[320] N. H. Rotman, M. Schapira, and A. Tamar, "Online Safety Assurance for Learning-Augmented Systems," in *Proceedings of the 19th ACM Workshop on Hot Topics in Networks*. ACM, Nov. 2020, pp. 88–95.

[321] Y. Fan, B. Hou, P. Xie, L. Liu, and H. Ma, "EchoCC: Refining Learning-Based Congestion Control With WordBook," *IEEE Transactions on Networking*, vol. 33, no. 5, pp. 2553–2568, Oct. 2025.

[322] J. Zhang, C. Zeng, H. Zhang, S. Hu, and K. Chen, "LiteFlow: towards high-performance adaptive neural networks for kernel datapath," in *Proceedings of the ACM SIGCOMM 2022 Conference*. ACM, Aug. 2022, pp. 414–427.

[323] A. Narayan, F. Cangialosi, D. Raghavan, P. Goyal, S. Narayana, R. Mittal, M. Alizadeh, and H. Balakrishnan, "Restructuring endpoint congestion control," in *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*. ACM, Aug. 2018, pp. 30–43. [Online]. Available: http://dx.doi.org/10.1145/3230543.3230553

[324] D. Wei, J. Zhang, X. Zhang, and C. Huang, "Plume: Lightweight and generalized congestion control with deep reinforcement learning," *China Communications*, vol. 19, no. 12, pp. 101–117, Dec. 2022.

[325] X. Ji, B. Han, Y. Xu, and J. Su, "ICCP: Toward Congestion Control Agent via Controlling Logic Decoupling and Algorithm Integration," *IEEE Transactions on Network and Service Management*, vol. 23, pp. 6463–6479, 2026.

[326] S. Patel, S. Abdu Jyothi, and N. Narodytska, "CrystalBox: Future-Based Explanations for Input-Driven Deep RL Systems," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 13, pp. 14 563–14 571, Mar. 2024.

[327] S. Patel, D. Han, N. Narodytska, and S. A. Jyothi, "Agua: A Concept-Based Explainer for Learning-Enabled Systems," in *Proceedings of the ACM SIGCOMM 2025 Conference*. ACM, Aug. 2025, pp. 329–346.

[328] Y. Gu, K. Zhang, Q. Liu, R. Yu, X. Lin, and X. Sun, "ProCC: Programmatic Reinforcement Learning for Efficient and Transparent TCP Congestion Control," in *Proceedings of the Eighteenth ACM International Conference on Web Search and Data Mining*. ACM, Mar. 2025, pp. 963–972.

[329] P. Luo and G. Yang, "A Novel Distributed Reinforcement Learning Training System for Network Congestion Control," in *2023 China Automation Congress (CAC)*. IEEE, Nov. 2023, pp. 7244–7249.

[330] Y. Liu, D. Wei, C. Zhang, and W. Li, "Distributed Bandwidth Allocation Strategy for QoE Fairness of Multiple Video Streams in Bottleneck Links," *Future Internet*, vol. 14, no. 5, p. 152, May 2022.

[331] V. Arun, M. Alizadeh, and H. Balakrishnan, "Starvation in end-to-end congestion control," in *Proceedings of the ACM SIGCOMM 2022 Conference*. ACM, Aug. 2022, pp. 177–192. [Online]. Available: http://dx.doi.org/10.1145/3544216.3544223

[332] L. Peng, X. Xie, Y. Bao, and Y. Cui, "NetRL: Powering Network Decisions via NetGRPO-Driven Reinforcement Learning," arXiv:2610.16226, 2026.

[333] R. Dwivedula, D. Saxena, A. Akella, S. Chaudhuri, and D. Kim, "Man-Made Heuristics Are Dead. Long Live Code Generators!" in *Proceedings of the 24th ACM Workshop on Hot Topics in Networks*. ACM, Nov. 2025, pp. 51–60.

[334] Z. He, A. Gottipati, L. Qiu, Y. Yang, and F. Y. Yan, "Congestion Control System Optimization with Large Language Models," arXiv:2508.16074, 2025.

[335] N. Rozen-Schiff, L. Schiff, and S. Schmid, "Utility Function is All You Need: LLM-based Congestion Control," arXiv:2603.10357, 2026.

[336] ——, "GenCC: Heterogeneous Network Congestion Control using LLMs," in *Proceedings of the ACM SIGCOMM 2026 Conference*. ACM, Aug. 2026, pp. 1994–2000.

[337] M. F. Zhani, Y. Korbi, and Y. Mkadem, "FlexNGIA 2.0: Redesigning the Internet with Agentic AI – Protocols, Services, and Traffic Engineering Designed, Deployed, and Managed by AI," arXiv:2509.02124, 2025.

[338] Z. He, A. Gottipati, L. Qiu, X. Luo, K. Xu, Y. Yang, and F. Y. Yan, "Designing Network Algorithms via Large Language Models," in *Proceedings of the 23rd ACM Workshop on Hot Topics in Networks*. ACM, Nov. 2024, pp. 205–212. [Online]. Available: http://dx.doi.org/10.1145/3696348.3696868

[339] M. Zhang, H. Bagheri, and L. Xu, "Toward Non-Expert Customized Congestion Control," in *ICC 2025 - IEEE International Conference on Communications*. IEEE, 2025, pp. 3606–3612.

[340] X. Qin, Y. Wang, and L. Shangguan, "CCA Reimagined: An Exploratory Study of Large Language Models for Congestion Control," arXiv:2604.03857, 2026.

[341] D. Wu, L. Kang, Z. Wang, F. Wang, W. Zhang, C. Sun, X. Tao, W. Yang, L. Zhang, W. Zhu, P. Cui, and Z. Wang, "Grounding Large Language Models as Generalizable Policies in Network Control," arXiv:2512.11839, 2025.

[342] S. K. Shrestha, S. R. Pokhrel, and J. Kua, "Adapting Large Language Models for Improving TCP Fairness over WiFi," arXiv:2412.18200, 2024.

[343] R. D. Silva, S. R. Pokhrel, and J. Kua, "Small Language Model-based Control for BBR over Low Earth Orbit Satellite Internet," arXiv:2607.07142, 2026.

[344] S. Varkey, S. Galvankar, and S. Choi, "POSTER: Fine-Tuning Large Language Models via Reinforcement Learning with Verifiable Rewards for Congestion Control," in *2026 Silicon Valley Cybersecurity Conference (SVCC)*. IEEE, 2026, pp. 1–3.

[345] S. Varkey, "Reasoning in the Network: Large Language Models for Congestion Control and Agent-Defined Networking," Scholar Commons (Santa Clara University), 2026.

[346] D. Wu, X. Wang, Y. Qiao, Z. Wang, J. Jiang, S. Cui, and F. Wang, "NetLLM: Adapting Large Language Models for Networking," in *Proceedings of the ACM SIGCOMM 2024 Conference*. ACM, Aug. 2024, pp. 661–678. [Online]. Available: http://dx.doi.org/10.1145/3651890.3672268

[347] L. Jia, T. Huang, and L. Sun, "ZiXia: A Reinforcement Learning Approach via Adjusted Ranking Reward for Internet Congestion Control," in *ICC 2022 - IEEE International Conference on Communications*. IEEE, May 2022, pp. 365–370.

[348] A. P. Silva, K. Obraczka, S. Burleigh, and C. M. Hirata, "Smart Congestion Control for Delay- and Disruption Tolerant Networks," in *2016 13th Annual IEEE International Conference on Sensing, Communication, and Networking (SECON)*. IEEE, 2016, pp. 1–9.

[349] N. Jay, N. H. Rotman, P. B. Godfrey, M. Schapira, and A. Tamar, "Internet Congestion Control via Deep Reinforcement Learning," arXiv:1810.03259, 2018.

[350] W. Wang, Y. Shao, and K. Zheng, "Muses: Enabling Lightweight and Diversity for Learning Based Congestion Control," in *Proceedings of the ACM SIGCOMM 2018 Conference on Posters and Demos*. ACM, Aug. 2018, pp. 141–143.

[351] X. Nie, Y. Zhao, D. Pei, G. Chen, K. Sui, and J. Zhang, "Reducing Web Latency Through Dynamically Setting TCP Initial Window with Reinforcement Learning," in *2018 IEEE/ACM 26th International Symposium on Quality of Service (IWQoS)*. IEEE, 2018, pp. 1–10.

[352] I. Nascimento, R. Souza, S. Lins, A. Silva, and A. Klautau, "Deep Reinforcement Learning Applied to Congestion Control in Fronthaul Networks," in *2019 IEEE Latin-American Conference on Communications (LATINCOM)*. IEEE, Nov. 2019, pp. 1–6.

[353] R. Xie, X. Jia, and K. Wu, "Adaptive Online Decision Method for Initial Congestion Window in 5G Mobile Edge Computing Using Deep Reinforcement Learning," *IEEE Journal on Selected Areas in Communications*, vol. 38, no. 2, pp. 389–403, Feb. 2020.

[354] T. Mai, H. Yao, X. Zhang, Z. Xiong, and D. Niyato, "A Distributed Reinforcement Learning Approach to In-network Congestion Control," in *2020 IEEE/CIC International Conference on Communications in China (ICCC)*. IEEE, Aug. 2020, pp. 817–822.

[355] F. Naeem, G. Srivastava, and M. Tariq, "A Software Defined Network Based Fuzzy Normalized Neural Adaptive Multipath Congestion Control for the Internet of Things," *IEEE Transactions on Network Science and Engineering*, vol. 7, no. 4, pp. 2155–2164, Oct. 2020.

[356] H. Ma, D. Xu, Y. Dai, and Q. Dong, "An intelligent scheme for congestion control: When active queue management meets deep reinforcement learning," *Computer Networks*, vol. 200, p. 108515, Dec. 2021.

[357] M. Chen, R. Li, Z. Zhao, and H. Zhang, "RAN Information-assisted TCP Congestion Control via DRL with Reward Redistribution," in *2021 IEEE International Conference on Communications Workshops (ICC Workshops)*. IEEE, 2021, pp. 1–7.

[358] S. Zhang, W. Lei, W. Zhang, and Y. Guan, "LearningCC: An online learning approach for congestion control," *Transactions on Emerging Telecommunications Technologies*, vol. 32, no. 11, Aug. 2021.

[359] S. K. Kasi, S. Das, and S. Biswas, "TCP Congestion Control with Multiagent Reinforcement and Transfer Learning," in *2021 IEEE 11th Annual Computing and Communication Workshop and Conference (CCWC)*. IEEE, Jan. 2021, pp. 1507–1513.

[360] Y. Ma, H. Tian, X. Liao, J. Zhang, W. Wang, K. Chen, and X. Jin, "Multi-Objective Congestion Control," arXiv:2107.01427, 2021.

[361] Z. Xia, Y. Chen, L. Wu, Y.-C. Chou, Z. Zheng, H. Li, and B. Li, "A Multi-objective Reinforcement Learning Perspective on Internet Congestion Control," in *2021 IEEE/ACM 29th International Symposium on Quality of Service (IWQOS)*. IEEE, 2021, pp. 1–10.

[362] H. Yu, J. Zheng, Z. Du, and G. Chen, "MPLibra: Complementing the Benefits of Classic and Learning-based Multipath Congestion Control," in *2021 IEEE 29th International Conference on Network Protocols (ICNP)*. IEEE, Nov. 2021, pp. 1–11.

[363] A. Sacco, M. Flocco, F. Esposito, and G. Marchetto, "Partially Oblivious Congestion Control for the Internet via Reinforcement Learning," *IEEE Transactions on Network and Service Management*, vol. 20, no. 2, pp. 1644–1659, 2023.

[364] W. Li, S. Gao, X. Li, Y. Xu, and S. Lu, "TCP-NeuRoc: Neural Adaptive TCP Congestion Control With Online Changepoint Detection," *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 8, pp. 2461–2475, Aug. 2021.

[365] Y. Zheng, H. Chen, Q. Duan, L. Lin, Y. Shao, W. Wang, X. Wang, and Y. Xu, "Leveraging Domain Knowledge for Robust Deep Reinforcement Learning in Networking," in *IEEE INFOCOM 2021 - IEEE Conference on Computer Communications*. IEEE, May 2021, pp. 1–10.

[366] T. Huang, R.-X. Zhang, and L. Sun, "Zwei: A Self-Play Reinforcement Learning Framework for Video Transmission Services," *IEEE Transactions on Multimedia*, vol. 24, pp. 1350–1365, 2022.

[367] H. Li, B. Lu, J. Xu, L. Song, W. Zhang, L. Li, and Y. Yin, "Reinforcement Learning Based Cross-Layer Congestion Control for Real-Time Communication," in *2022 IEEE International Symposium on Broadband Multimedia Systems and Broadcasting (BMSB)*. IEEE, 2022, pp. 01–06.

[368] S.-J. Seo and Y.-Z. Cho, "Fairness Enhancement of TCP Congestion Control Using Reinforcement Learning," in *2022 International Conference on Artificial Intelligence in Information and Communication (ICAIIC)*. IEEE, Feb. 2022, pp. 288–291.

[369] j. lei and Y. Zhou, "Deep reinforcement learning based sensing bidirectional nodes congestion control mechanism in wireless sensor networks," in *5th International Conference on Computer Information Science and Application Technology (CISAT 2022)*, F. Zhao, Ed. SPIE, Oct. 2022, p. 130.

[370] W. Yang, Y. Li, P. Jiang, Z. Zhou, and Q. Liu, "GDMD: A Transmission Control Scheme with Block Information-aware for Delay Sensitive Multimedia," in *2022 IEEE 24th Int Conf on High Performance Computing & Communications; 8th Int Conf on Data Science & Systems; 20th Int Conf on Smart City; 8th Int Conf on Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys)*. IEEE, Dec. 2022, pp. 446–455.

[371] Z. Xia, Y. Zhou, F. Y. Yan, and J. Jiang, "Automatic Curriculum Generation for Learning Adaptation in Networking," arXiv:2202.05940, 2022.

[372] Z. Xia, L. Wu, F. Wang, X. Liao, H. Hu, J. Wu, and D. Wu, "Glider: rethinking congestion control with deep reinforcement learning," *World Wide Web*, vol. 26, no. 1, pp. 115–137, 2022.

[373] J. Zhang, C. Zeng, H. Zhang, S. Hu, and K. Chen, "LiteFlow: Toward High-Performance Adaptive Neural Networks for Kernel Datapath," *IEEE/ACM Transactions on Networking*, vol. 32, no. 1, pp. 627–642, Feb. 2024.

[374] J. Bai, T. Zhang, C. Wang, and G. Xie, *MA-CC: Cross-Layer Congestion Control via Multi-agent Reinforcement Learning*. Springer Nature Switzerland, 2023, pp. 659–671.

[375] X. Jiang, G. Gong, and G. Jin, "Combining Heuristic and Reinforcement Learning to Achieve the Low-latency and High-throughput Receiver-side Congestion Control," arXiv:2502.16498, 2025.

[376] D. V. Satinge and M. H. Dakhore, "Q Learning Algorithm Is ImplementedImprove Network Qos Using Adaptive Congestion Control," *International Journal of Innovations in Engineering and Science*, vol. 7, no. 8, pp. 31–35, Aug. 2022.

[377] G. Diel, C. C. Miers, M. A. Pillon, and G. P. Koslovski, "Data classification and reinforcement learning to avoid congestion on SDN-based data centers," in *GLOBECOM 2022 - 2022 IEEE Global Communications Conference*. IEEE, Dec. 2022, pp. 2861–2866.

[378] R. Yerushalmi, G. Amir, A. Elyasaf, D. Harel, G. Katz, and A. Marron, "Scenario-assisted Deep Reinforcement Learning," in *Proceedings of the 10th International Conference on Model-Driven Engineering and Software Development*. SCITEPRESS - Science and Technology Publications, 2022, pp. 310–319.

[379] H. Tian, X. Liao, C. Zeng, J. Zhang, and K. Chen, "Spine: an efficient DRL-based congestion control with ultra-low overhead," in *Proceedings of the 18th International Conference on emerging Networking EXperiments and Technologies*. ACM, Nov. 2022, pp. 261–275.

[380] Z. Xing, H. Qi, X. Di, J. Liu, and L. Cong, *Deep Reinforcement Learning Based Congestion Control Mechanism for SDN and NDN in Satellite Networks*. Springer Nature Switzerland, 2023, pp. 13–29.

[381] H. He, D. Chen, Z. Zhuang, Q. Qi, L. Zhang, T. Xu, and J. Wang, "Fine-Grained Flow Control Agent on Path MTU for IoT Software," in *Proceedings of the 14th Asia-Pacific Symposium on Internetware*. ACM, Aug. 2023, pp. 144–153.

[382] H. A. Naqvi, M. H. Hilman, and B. Anggorojati, "Implementability improvement of deep reinforcement learning based congestion control in cellular network," *Computer Networks*, vol. 233, p. 109874, 2023.

[383] P. Luo, Y. Liu, Z. Wang, J. Chu, and G. Yang, "A novel Congestion Control algorithm based on inverse reinforcement learning with parallel training," *Computer Networks*, vol. 237, p. 110071, Dec. 2023.

[384] J. Du, C. Zhang, S. He, and W. Qu, "Learning-Based Congestion Control Assisted by Recurrent Neural Networks for Real-Time Communication," in *2023 IEEE Symposium on Computers and Communications (ISCC)*. IEEE, 2023, pp. 323–328.

[385] R. Galliera, M. Zaccarini, A. Morelli, R. Fronteddu, F. Poltronieri, N. Suri, and M. Tortonesi, "Learning to Sail Dynamic Networks: The MARLIN Reinforcement Learning Framework for Congestion Control in Tactical Environments," in *MILCOM 2023 - 2023 IEEE Military Communications Conference (MILCOM)*. IEEE, Oct. 2023, pp. 424–429.

[386] Z. Wu, C.-Y. Huang, and P. Ramanathan, "MIA: A Transport-Layer Plugin for Immersive Applications in Millimeter Wave Access Networks," in *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications*. IEEE, May 2023, pp. 1–10.

[387] Z. Pan, J. Zhou, X. Qiu, W. Li, H. Pan, and W. Zhang, "Marten: A Built-in Security DRL-Based Congestion Control Framework by Polishing the Expert," in *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications*. IEEE, May 2023, pp. 1–10.

[388] M. Lakshmi and M. B. Arrama, "Comparison and Implementation of Protocols to Compensate for Congestion in Networks with Reinforcement Learning," *International Journal of Communication Networks and Information Security*, vol. 15, no. 04, pp. 572–580, 2023.

[389] Shahzad, E.-S. Jung, and H. S. Kim, "RLECN—A learning based dynamic threshold control of ECN," *ICT Express*, vol. 9, no. 6, pp. 1007–1012, Dec. 2023.

[390] D. Markudova and M. Meo, "ReCoCo: Reinforcement learning-based Congestion control for Real-time applications," in *2023 IEEE 24th International Conference on High Performance Switching and Routing (HPSR)*. IEEE, 2023, pp. 68–74.

[391] G. Amir, O. Maayan, T. Zelazny, G. Katz, and M. Schapira, *Verifying Generalization in Deep Learning*. Springer Nature Switzerland, 2023, pp. 438–455.

[392] M. H. Ali and S. Öztürk, "Efficient congestion control in communications using novel weighted ensemble deep reinforcement learning," *Computers and Electrical Engineering*, vol. 110, p. 108811, 2023.

[393] X. Liao, H. Tian, C. Zeng, X. Wan, and K. Chen, "Towards Fair and Efficient Learning-based Congestion Control," arXiv:2403.01798, 2024.

[394] H. Tian, X. Liao, C. Zeng, X. Wan, J. Zhang, B. Xia, and K. Chen, "Towards Fair and Efficient Congestion Control Through Multi-Agent Deep Reinforcement Learning," *IEEE Transactions on Networking*, vol. 34, pp. 2966–2980, 2026.

[395] L. Yang, J. A. Fraire, K. Zhao, R. Wang, W. Li, and H. Yang, "Optimizing deep-space DTN congestion control via deep reinforcement learning," *Computer Networks*, vol. 255, p. 110865, Dec. 2024.

[396] M. Alipio and M. Bures, "Enhancing WSN Performance with DRL-CaCC: An Adaptive Cache-Aware Congestion Control Mechanism," in *2024 15th International Conference on Information and Communication Technology Convergence (ICTC)*. IEEE, Oct. 2024, pp. 1903–1908.

[397] T. Yang, Q. Li, and F. Hu, "Intelligent Congestion Control in QUIC for Reliable E2E Communication Network: A Digital Twin-Based Approach," in *2024 IEEE/CIC International Conference on Communications in China (ICCC Workshops)*. IEEE, Aug. 2024, pp. 282–287.

[398] J. Hu, Z. Zhou, S. Rao, Y. Peng, B. Bao, and C. Ruan, "Learning-Based Hierarchical Adaptive Congestion Control with Low Training Overhead," in *2024 IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA)*. IEEE, Oct. 2024, pp. 262–267.

[399] Z. Xia, H. Li, and J. Jiang, "Loss-tolerant neural video codec aware congestion control for real time video communication," arXiv:2411.06742, 2024.

[400] X. Dai, Z. Wang, J. Ye, and J. C. Lui, "Quantifying the Merits of Network-Assist Online Learning in Optimizing Network Protocols," in *2024 IEEE/ACM 32nd International Symposium on Quality of Service (IWQoS)*. IEEE, 2024, pp. 1–10.

[401] Y. Zhu, "Intelligent congestion control of data transmission in low Earth orbit satellite networks based on reinforcement learning: Analysis and optimization," *Information Sciences*, vol. 694, p. 121692, Mar. 2025.

[402] A. Kumar and N. Hemrajani, "Comparative Analysis of Different Transport Layer Protocol Techniques Incognitive Network," *Recent Advances in Computer Science and Communications*, vol. 18, no. 3, Feb. 2025.

[403] V. Komanduri, C. Wang, and R. Rojas-Cessa, "Comparing Link Sharing and Flow Completion Time in Traditional and Learning-based TCP," in *2024 IEEE 25th International Conference on High Performance Switching and Routing (HPSR)*. IEEE, 2024, pp. 167–172.

[404] J. S. Sidhu and A. Bentaleb, "Aero: A Pluggable Congestion Control for QUIC," in *Proceedings of the 35th edition of the Workshop on Network and Operating System Support for Digital Audio and Video*. ACM, Mar. 2025, pp. 29–35.

[405] C. Wang, X. Ren, Y. He, F. Yu, and V. C. M. Leung, "Congestion Control for Blockchain-enabled SDN in Web 4.0: A Reinforcement Learning Approach through Active Inference," in *2025 IEEE 45th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2025, pp. 177–187.

[406] D. Zala, A. K. Vyas, N. Khatri, Y. Patidar, and K. Zala, *Effective Congestion Control Algorithm for Wireless Networks Based on Deep Reinforcement Learning*. Springer Nature Singapore, 2025, pp. 109–123.

[407] C. Pan, Y. Wang, H. Shi, and W. Tian, "DRL-ABS: Deep Reinforcement Learning-Based Adaptive Buffer Sizing for Edge Routers in AIoT Networks," *IEEE Internet of Things Journal*, vol. 12, no. 15, pp. 30 475–30 492, Aug. 2025.

[408] K. R. Rao and R. R. Kalangi, "Enhancing Efficiency with Adaptive Optimized Balancing Factor for MPTCP Congestion Control Using Deep Deterministic Policy Gradient: EE-AOBF-MPTCP-DDPG," *International Journal of Computational and Experimental Science and Engineering*, vol. 11, no. 2, Apr. 2025.

[409] L. Kiarie, M. Derakhshani, and K. Kyriakopoulos, "Expert-Driven Jumpstart Reinforcement Learning for Managing Network Congestion Control," 2025.

[410] L. Pappone, A. Sacco, and F. Esposito, "Online Reinforcement Learning From Existing Heuristics for TCP Congestion Control," *IEEE Transactions on Networking*, vol. 34, pp. 6922–6937, 2026.

[411] Y. Takesada, H. N. Aung, R. Nakamura, and H. Ohsaki, "Q-AIMD: AIMD-based Window Flow Control with Reinforcement Learning," in *2025 International Conference on Information Networking (ICOIN)*. IEEE, Jan. 2025, pp. 179–184.

[412] Q. Jiao and L. Mhamdi, "Bandwidth Fair Share Using Reinforcement Learning for IoT Networks," in *ICC 2025 - IEEE International Conference on Communications*. IEEE, 2025, pp. 3491–3496.

[413] A. B. Gandrakota, "AutoNetFlow: An Intelligent Reinforcement Learning Framework for Adaptive Process Automation in Network Traffic Control," in *2026 International Conference on Current Research in Artificial Intelligence and Data Science (ICCRAIDS)*. IEEE, Apr. 2026, pp. 1–6.

[414] V. T. Nguyen, K. Q. Nguyen, D. Nguyen, V. C. Ho, and L. Huynh, *Deep Reinforcement Learning Framework with Least Squares Congestion Minimization for Intelligent Network Congestion Control and Latency–Bandwidth Optimization*. Springer Nature Switzerland, 2026, pp. 576–589.

[415] H. Zhang, M. Wang, and H. Zhang, "A Delay-Based Reinforcement Learning Approach for RTT Jitter-Aware Congestion Control," *International Journal of Ad Hoc and Ubiquitous Computing*, vol. 1, no. 1, 2026.

[416] J. Zhang, J. Zhou, and J. Yuan, "Active queue management based on fuzzy logic and reinforcement learning," *Computer Networks*, vol. 282, p. 112272, 2026.

[417] K. S. Dara, "Autonomous Congestion Control in High-Speed Networks Via Multiagent Deep Q Learning," in *2026 International Conference on Multidisciplinary Innovations For Smart & Sustainable Future (MISSF)*. IEEE, Apr. 2026, pp. 1–6.

[418] J. Cui, Y. Huang, Q. Zeng, B. Hu, and X. Wu, "PRO-SAC: Deep Reinforcement Learning-Based Congestion Control for Real-Time Video Streaming With Delayed RTP/RTCP Feedback," in *2026 11th International Symposium on Advances in Electrical, Electronics and Computer Engineering (ISAEECE)*. IEEE, 2026, pp. 700–705.

[419] D. Kanojia, D. C. Misra, and I. Verma, "Reinforcement Learning-Assisted Hybrid Congestion Control for 6G Ultra-Low Latency Networks," in *2026 International Conference on Computing, Sciences and Communications (ICCSC)*. IEEE, Feb. 2026, pp. 1–6.

[420] Q. Guo, Z. Bao, S. Wang, C. Dong, S. Jiang, L. Zhu, Y. Sun, and X. Xu, "Deep Reinforcement Learning Enabled Adaptive Cross-Layer Semantic Congestion Control for Real-Time Semantic Tasks," *IEEE Transactions on Network Science and Engineering*, vol. 13, pp. 10 044–10 064, 2026.

[421] J. Pan, A. Agarwal, I. Dillig, and V. Arun, "Syntra: Synthesizing Cross-Layer Controllers for Low-Latency Video Streaming," in *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2026.

[422] L. Zhang, S. Fu, X. Duan, J. Yan, and Y. Jiang, "Collaborative optimization method for TT&C microburst traffic using deep learning," in *Eleventh Symposium on Novel Optoelectronic Detection Technology and Applications (NDTA 2025)*, C. Ping, Ed. SPIE, May 2026, p. 460.

[423] J. Cohen, "A Coefficient of Agreement for Nominal Scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, Apr. 1960. [Online]. Available: http://dx.doi.org/10.1177/001316446002000104